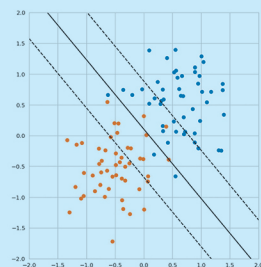
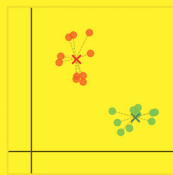
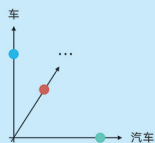


TURING

图灵程序
设计丛书



基础
知识

朴素贝叶斯
核方法
SVM
逻辑回归
正则化
线性回归

152张图表

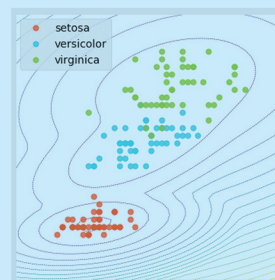
轻松掌握17种
常用算法

NMF
LDA
KNN
PCA
LSA

图解 机器学习 算法

随机森林
神经网络
k-means 算法
混合高斯分布
LLE
t-SNE

算法
详解



模型
评估

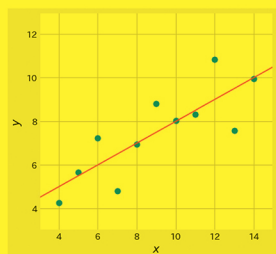
没有复杂公式，零基础也能懂！

[日]秋庭伸也 杉山阿圣 寺田学 著

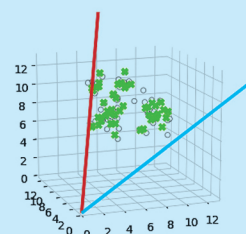
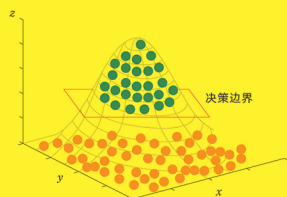
[日]加藤公一 审校

郑明智 译

全彩印刷



环境
搭建



中国工信出版集团

人民邮电出版社
POSTS & TELECOM PRESS

| 作者简介

秋庭伸也

日本早稻田大学硕士毕业，目前在 Recruit Communications 公司担任技术负责人。

杉山阿圣

具有多年研发经验，目前在AI创业公司 SENSY担任机器学习研究员。

寺田学

CMS Communications公司董事长、日本 PyCon 会议组织者、Plone 基金会大使、Python 工程师发展协会顾问理事、PSF（Python 软件基金会）贡献成员。

| 译者简介

郑明智

智慧医疗工程师，翻译经验丰富，有《白话机器学习的数学》《用Python动手机器学习》等多部译著。

数字版权声明

图灵社区的电子书没有采用专有客户端
您可以在任意设备上，用自己喜欢的浏览器和PDF阅读器进行阅读。

但您购买的电子书仅供您个人使用，未经授权，不得进行传播。

我们愿意相信读者具有这样的良知和觉悟，与我们共同保护知识产权。

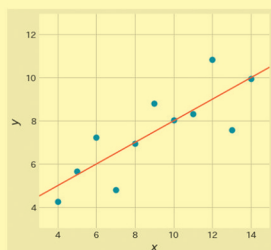
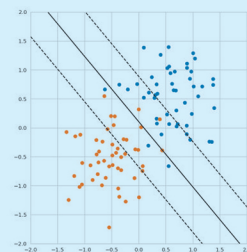
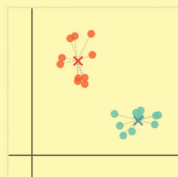
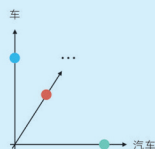
如果购买者有侵权行为，我们可能对该用户实施包括但不限于关闭该帐号等维权措施，并可能追究法律责任。

站在巨人肩上
Standing on Shoulders of Giants



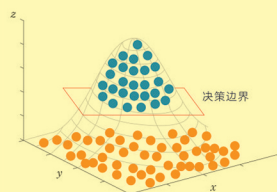
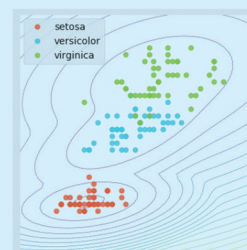


图灵程序
设计丛书

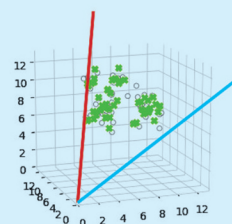


图解 机器学习 算法

[日]秋庭伸也 杉山阿圣 寺田学 著
[日]加藤公一 审校
郑明智 译



人民邮电出版社
北京



图书在版编目(CIP)数据

图解机器学习算法 / (日) 秋庭伸也, (日) 杉山阿圣, (日) 寺田学著; 郑明智译. -- 北京: 人民邮电出版社, 2021.6

(图灵程序设计丛书)

ISBN 978-7-115-56356-9

I. ①图… II. ①秋… ②杉… ③寺… ④郑… III. ①机器学习—算法 IV. ①TP181

中国版本图书馆CIP数据核字(2021)第065748号

内 容 提 要

本书基于丰富的图示, 详细介绍了有监督学习和无监督学习的17种算法, 包括线性回归、正则化、逻辑回归、支持向量机、核方法、朴素贝叶斯、随机森林、神经网络、KNN、PCA、LSA、NMF、LDA、*k-means* 算法、混合高斯分布、LLE 和 *t-SNE*。书中针对各算法均用 Python 代码进行了实现, 读者可一边运行代码一边阅读, 从而加深对算法的理解。

-
- ◆ 著 [日] 秋庭伸也 杉山阿圣 寺田学
审 校 [日] 加藤公一
译 郑明智
责任编辑 杜晓静
责任印制 周昇亮
- ◆ 人民邮电出版社出版发行 北京市丰台区成寿寺路11号
邮编 100164 电子邮件 315@ptpress.com.cn
网址 <https://www.ptpress.com.cn>
北京 印刷
- ◆ 开本: 880×1230 1/24
印张: 8
字数: 248千字 2021年6月第1版
印数: 1-2 500册 2021年6月北京第1次印刷
- 著作权合同登记号 图字: 01-2019-7534号
-

定价: 79.80元

读者服务热线: (010)84084456 印装质量热线: (010)81055316

反盗版热线: (010)81055315

广告经营许可证: 京东市监广登字20170147号

前言

近十年来，大数据、人工智能和深度学习开始流行，与它们相关联的机器学习技术也已经被人们所熟知。

机器学习算法并非只有数据分析师才需要理解并在工作中应用，软件工程师和项目管理人员等也会用到。因此，本书将为大家讲解各种机器学习算法。

刚开始接触机器学习的人可能会对不熟悉的数学公式和统计术语感到头疼。这时，我们可以借助图来理解，这有助于我们把握机器学习的全貌。为了帮助非科班出身的机器学习初学者理解，本书在讲解时尽可能少地使用数学公式，主要使用的是图，所以各种算法的特征和不同一目了然。

如果你在尝试学习机器学习时因不熟悉数学公式和统计术语而受挫，那么一定要读一读本书。

本书代码是使用 Python 实现的。Python 是极具人气的编程语言，它有丰富的机器学习和统计相关的库。请务必一边实际地运行代码，一边阅读本书。

目标读者

本书的目标读者如下。

- 对机器学习感兴趣，并且已开始学习的人。
- 了解一些机器学习算法，想学习更多机器学习算法的人。
- 不擅长数学公式，在阅读机器学习专业书时感到吃力的人。
- 希望能够根据要解决的问题选择恰当的机器学习算法的人。
- 有一定的编程经验，能够运行示例代码的人。

本书不会详细解释机器学习的数学背景知识以及进行调优计算时的具体步骤。如果读者想深入理解，请阅读参考文献或相关图书。

本书的主要内容

各章内容如下。

首先，第 1 章是对机器学习基础知识的说明。如果想先了解一下机学习的整体情况，请从第 1 章开始按顺序阅读。

第 2 章和第 3 章讲解了一些机器学习算法。如果读者对机器学习有一定程度的了解或者想学习某个算法，可以从这两章的算法开始读起。

第 4 章综合介绍了机器学习的评估方法。在实际使用机器学习时，这一章的内容一定会对读者有所帮助。

第 5 章介绍了 Python 的环境搭建方法。

关于数学公式和符号

针对本书涉及的数学公式和符号，说明如下。

- 向量

向量用小写字母粗斜体表示。向量的第 i 个元素表示为 x_i 。

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

- 矩阵

矩阵用大写字母粗斜体表示。矩阵第 i 行第 j 列的元素表示为 w_{ij} 。

$$\boldsymbol{W} = \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{pmatrix}$$

- 总和

元素的总和用 Σ 符号表示。计算 n 个值 $(x_1, x_2, \dots, x_n)$ 的总和的数学式表示为：

$$\sum_{i=1}^n x_i$$

本书的示例代码

■ 本书示例代码的运行环境

经过验证，本书各章的示例代码可以在以下环境正常运行。

- Python : 3.7
- scikit-learn : 0.20.3

■ 示例代码的下载地址

本书的示例代码可从以下网址下载。

ituring.cn/book/2761^①

■ 免责声明

我们已确认在正常使用示例代码时不会发生任何问题，但是对于应用本书示例代码所产生的一切损失，作译者和出版方概不负责。

^① 请至“随书下载”处下载本书示例代码文件。——编者注

目 录

第 1 章

机器学习基础 1

1.1 机器学习概要 2

- ▶ 什么是机器学习 2
- ▶ 机器学习的种类 3
- ▶ 机器学习的应用 8

1.2 机器学习的步骤 9

- ▶ 数据的重要性 9
- ▶ 有监督学习（分类）的例子 11
- ▶ 无监督学习（聚类）的例子 16
- ▶ 可视化 18
- ▶ 图形的种类和画法：使用 Matplotlib 显示图形的方法 22
- ▶ 使用 pandas 理解和处理数据 30
- ▶ 本章小结 36

第 2 章

有监督学习 37

2.1 算法 1：线性回归 38

- ▶ 概述 38
- ▶ 算法说明 39
- ▶ 详细说明 41

2.2	算法 2：正则化	45
	▶ 概述	45
	▶ 算法说明	48
	▶ 详细说明	50
2.3	算法 3：逻辑回归	52
	▶ 概述	52
	▶ 算法说明	53
	▶ 详细说明	55
2.4	算法 4：支持向量机	58
	▶ 概述	58
	▶ 算法说明	59
	▶ 详细说明	60
2.5	算法 5：支持向量机（核方法）	63
	▶ 概述	63
	▶ 算法说明	64
	▶ 详细说明	65
2.6	算法 6：朴素贝叶斯	68
	▶ 概述	68
	▶ 算法说明	70
	▶ 详细说明	74
2.7	算法 7：随机森林	76
	▶ 概述	76
	▶ 算法说明	77
	▶ 详细说明	80
2.8	算法 8：神经网络	81
	▶ 概述	81
	▶ 算法说明	83
	▶ 详细说明	86

2.9 算法 9: KNN 88

- ▶ 概述 88
- ▶ 算法说明 89
- ▶ 详细说明 90

第 **3** 章

无监督学习 93

3.1 算法 10: PCA 94

- ▶ 概述 94
- ▶ 算法说明 95
- ▶ 详细说明 98

3.2 算法 11: LSA 99

- ▶ 概述 99
- ▶ 算法说明 100
- ▶ 详细说明 104

3.3 算法 12: NMF 105

- ▶ 概述 105
- ▶ 算法说明 106
- ▶ 详细说明 108

3.4 算法 13: LDA 111

- ▶ 概述 111
- ▶ 算法说明 112
- ▶ 详细说明 114

3.5 算法 14: *k*-means 算法 117

- ▶ 概述 117
- ▶ 算法说明 117

▶ 详细说明	119
3.6 算法 15: 混合高斯分布	122
▶ 概述	122
▶ 算法说明	123
▶ 详细说明	126
3.7 算法 16: LLE	127
▶ 概述	127
▶ 算法说明	128
▶ 详细说明	131
3.8 算法 17: <i>t</i>-SNE	133
▶ 概述	133
▶ 算法说明	134
▶ 详细说明	136

第 **4** 章

评估方法和各种数据的处理 **139**

4.1 评估方法	140
▶ 有监督学习的评估	140
▶ 分类问题的评估方法	140
▶ 回归问题的评估方法	148
▶ 均方误差和决定系数指标的不同	152
▶ 与其他算法进行比较	152
▶ 超参数的设置	154
▶ 模型的过拟合	155
▶ 防止过拟合的方法	155
▶ 将数据分为训练数据和验证数据	156
▶ 交叉验证	158
▶ 搜索超参数	160

4.2 文本数据的转换处理 163

- ▶ 基于单词出现次数的转换 163
- ▶ 基于 tf-idf 的转换 164
- ▶ 应用于机器学习模型 165

4.3 图像数据的转换处理 167

- ▶ 直接将像素信息作为数值使用 167
- ▶ 将转换后的向量数据作为输入来应用机器学习模型 168

第 5 章

环境搭建 171

5.1 Python 3 的安装 172

- ▶ Windows 172
- ▶ macOS 172
- ▶ Linux 173
- ▶ 使用 Anaconda 在 Windows 上安装 174

5.2 虚拟环境 175

- ▶ 通过官方安装程序安装 Python 的情况 175
- ▶ 通过 Anaconda 安装 Python 的情况 177

5.3 第三方包的安装 178

- ▶ 什么是第三方包 178
- ▶ 安装第三方包的方法 178

参考文献 180

第 1 章

机器学习基础

2016 年春天，围棋界受到了冲击。

这一年发生了顶级棋手完败于计算机的事件，这一事件给人工智能热潮添了一把火。取得比赛胜利的是 DeepMind 公司开发的 AlphaGo 程序，该程序使用了最新的机器学习技术。

现如今，人工智能和深度学习等词语时常在新闻中出现。为了理解这些词语，我们需要先了解一下什么是机器学习。

1.1 机器学习概要

► 什么是机器学习

机器学习指的是计算机根据给定的问题、课题或环境进行学习，并利用学习结果解决问题或课题等的一整套机制（图 1-1）。



▲图 1-1 分类的示意图

与机器学习共同成为热门话题的还有人工智能和深度学习。这里梳理一下它们之间的关系（图 1-2）。**人工智能**的含义很广泛，是一个综合性系统。机器学习是实现人工智能的一种方法。也就是说，机器学习并非实现人工智能的唯一方法，但是近年来人工智能的研究一般使用机器学习。实现人工智能的方法还有很多，比如根据事先定好的规则进行数理统计预测等方法。

近年来，一种被称为**深度学习**的机器学习算法备受关注，以至于出现了这样的误解：提起人工智能，人们想到的就是深度学习。然而，深度学习只不过是机器学习算法的其中一种而已。深度学习在图像识别领域取得了划时代的成果，是当前热门的算法。目前在图像识别领域之外，深度学习也取得了许多成绩。

在进行机器学习时会用到各种各样的算法，需要根据机器学习对象的不同进行选择。本书的目标就是帮助读者学会选择合适的算法。在理解了各算法的特性后，读者就能实际处理机器学习问题了。



▲图 1-2 机器学习的包含关系

机器学习的种类

机器学习包含不同的种类。根据不同的输入数据，分类如下。

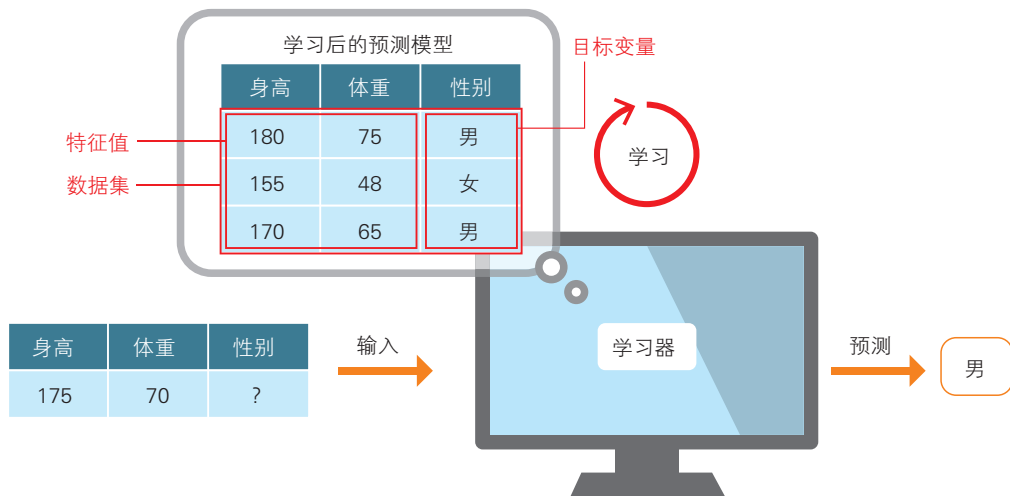
- 有监督学习
- 无监督学习
- 强化学习

下面我们依次详细地看一下。

有监督学习

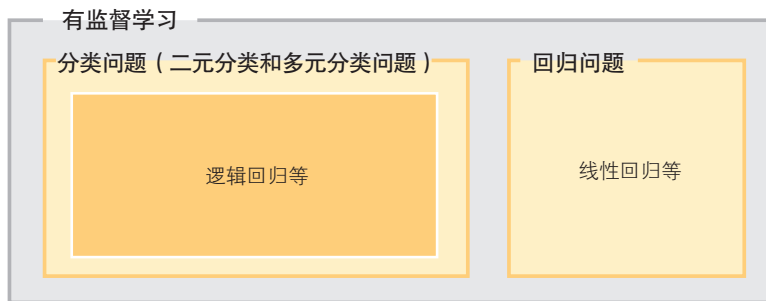
有监督学习是将问题的答案告知计算机，使计算机进行学习并给出机器学习模型的方法。这种方法要求数据中包含表示特征的数据和作为答案的目标数据。

如图 1-3 所示，已有表示特征的身高和体重数据，作为答案的数据是性别（男 / 女）。我们向计算机提供这些数据的组合，使计算机进行学习并给出预测模型。然后，将新的身高和体重数据提供给模型，由模型预测出性别。



▲图 1-3 有监督分类的示意图

预测性别这样的类别的问题叫作**分类问题**。由于这次是将数据分类到两个类别中，所以叫**二元分类**；还有将数据分类到更多类别，比如 10 个类别的情况，这样的情况叫作**多元分类**。这种答案变量不是连续值，而是作为类别数据的离散值的问题就是分类问题（图 1-4）。



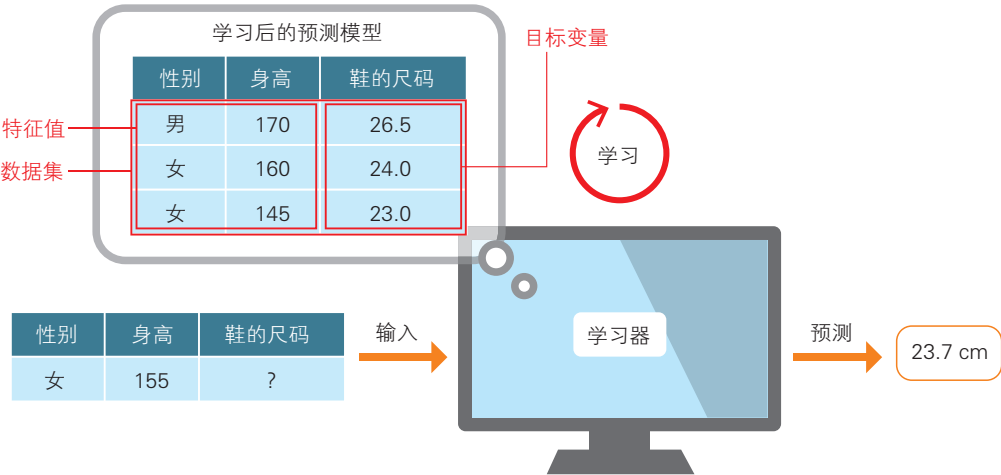
▲图 1-4 有监督学习的分类和回归的关系

此外，表示特征的数据叫作**特征值**或者**特征变量**，作为答案的数据叫作**目标变量**或者**标签**。

我们身边的一个分类问题的例子就是垃圾邮件过滤。用户判断邮件是否为垃圾邮件的过程就是打标签的过程，标签数据就是目标变量，邮件的发件人和内容则为特征值。打了标签的数据越多，机器学习的效果就越好，得到的结果的精度越高。

除了分类问题之外，有监督学习还包括**回归问题**。如图 1-5 所示，已有表示特征的性别和身高

数据，以及答案数据——鞋的尺码。在分类问题中，男和女的标签分别被数值化为 0 和 1，这两个数值之间的大小关系是没有意义的。与之相对，鞋的尺码 26.5 cm 和 24 cm 之间的大小关系则是有意
义的。对这样的数据进行预测的问题就是回归问题。在回归问题中，目标变量是作为连续值处理的，所以预测值有可能是 23.7 cm 这种不存在的尺码。



▲图 1-5 回归问题的示意图

第 2 章将详细介绍如表 1-1 所示的有监督学习的算法。

▼表 1-1 有监督学习算法与分类和回归的应用范围

算法编号	算 法 名	分 类	回 归
1	线性回归	×	✓
2	正则化	×	✓
3	逻辑回归	✓	×
4	支持向量机	✓	✓
5	支持向量机（核方法）	✓	✓
6	朴素贝叶斯	✓	×
7	随机森林	✓	✓
8	神经网络	✓	✓
9	KNN	✓	✓

无监督学习

有监督学习是将特征值和目标变量（答案）作为一套数据进行学习的方法，而无监督学习的数据中没有作为答案的目标变量。

有人可能会产生疑问：没有答案，该如何去学习呢？无监督学习将表示特征的数据作为输入，通过将数据变形为其他形式或者找出数据中的部分集合，来理解输入数据的构造。此外，与有监督学习相比，无监督学习的结果难以解释，或者要求分析者基于经验加以主观的解释。有监督学习以“能否正确预测目标变量”为指标，相比之下，为了能够对结果进行解释，在进行无监督学习时，用户需要对输入数据的前提知识有一定程度的了解。

这里举一个无监督学习的例子。我们思考一下对某个中学的学生成绩进行分析的场景。假设各科目之间存在着这样的关联性：擅长数学的学生也擅长理科，但不擅长语文和文科。

对于这样的输入数据，在使用无监督学习的代表性算法主成分分析（Principal Component Analysis, PCA）时，我们引入了新的轴，以说明被称为第一主成分的数据（关于 PCA，详见 3.1 节）。在第一主成分上的坐标可以解释为“小值表明该生擅长理科，大值表明该生擅长文科”，如表 1-2 所示，可以将数学、理科、语文、文科这 4 个特征值归纳在 1 个轴上加以展示。

▼表 1-2 PCA 的例子

数学	理科	语文	文科		第一主成分上的坐标
76	89	30	20	PCA →	-56.1
88	92	33	29		-54.3
52	35	84	89		49.1
45	33	90	91		56.6
60	65	55	70		4.6

这个例子使用 PCA 通俗易懂地解释了分析结果，但是需要根据输入数据选择合适的算法。近年来，无监督学习的研究在图像和自然语言处理方面取得了进展，是当前备受瞩目的领域（图 1-6）。这里介绍的 PCA 属于降维算法。降维是以更少的特征值来理解数据的算法。无监督学习中也包括聚类算法。聚类是将数据分类为几个簇（相似数据的集合）的算法。人类很难直接理解多变量数据（由 3 个以上的变量构成的数据），通过聚类，数据能够以簇这种简单的形式进行展现。



▲图 1-6 无监督学习的降维和聚类的关系

第 3 章将详细介绍如表 1-3 所示的无监督学习的算法。通过表 1-3，还可以了解每个算法适合的任务。

▼表 1-3 无监督学习与降维和聚类的应用范围

算法编号	算 法 名	降 维	聚 类
10	PCA	✓	×
11	LSA	✓	×
12	NMF	✓	×
13	LDA	✓	×
14	k -means 算法	×	✓
15	混合高斯模型	×	✓
16	LLE	✓	×
17	t -SNE	✓	×

强化学习

强化学习是以在某个环境下行动的智能体获得的奖励最大化为目标而进行学习的方法。本书未涉及强化学习。

这里简单地介绍一下强化学习。在主机游戏（环境）中，玩家（智能体）为了获得赛点（奖励）并取得最终的胜利，会无数次地重复尝试。我们也可以把强化学习看作有监督学习的目标变量被作为奖励提供的情况。拿主机游戏的例子来说，由于全部场景下所有操作的组合实在太多，很难通过人力进行评估，所以可以将游戏的场景和操作作为特征值，将游戏赛点作为目标变量，玩家无

须依赖人力，通过无数次的游戏即可自行收集特征值和目标变量的数据组。强化学习在重复地玩游戏、查看结果中不断学习更恰当的行动。

机器学习的应用

机器学习在各领域的应用都取得了较大进展，比如自动驾驶领域的研究就很有名。在文章的自动分类和自动翻译方面，机器学习成果颇丰。在医疗领域，机器学习对 X 射线影像的分析有助于疾病的早期发现。此外，很早之前人们就将机器学习应用在气象信息领域了。

近年来，随着计算机越来越便宜，数量越来越多，机器学习的研究也在不断加速。互联网产业的发达和物联网（Internet of Things, IoT）等技术的应用也为机器学习带来了丰富的数据。

根据数据的特性选择合适的算法，就会有前所未有的发现，这是机器学习领域有意思的地方。从下一节开始，我们将学习具体的机器学习步骤，目标是达到能对实际的数据应用机器学习的水平。

1.2 机器学习的步骤

机器学习具体要做哪些事情呢？本节的目标就是使读者了解机器学习的大致步骤。通过阅读本节，读者将对作为机器学习算法基础的处理流程有所理解，并学到机器学习的基本概念。

数据的重要性

在使用机器学习时，必须要有汇总并整理到一定程度的数据。以数据为基础，按规定的法则进行学习，最终才能进行预测。

没有数据，就不能进行机器学习。换言之，收集数据是首先要做的事情。

本节将说明机器学习的训练过程的一系列流程。为了便于理解，本节基于示例数据进行讲解，使用的是主流机器学习库 **scikit-learn** 包内置的数据，这个数据便于入手，可自由使用。

专栏 数据收集、数据预处理的重要性

在实际用机器学习解决问题之前，要先收集数据，有时还需要做问卷调查，甚至购买数据。然后，需要为收集到的数据人工标注答案标签，或者将其加工为机器学习算法易于处理的形式，删除无用的数据，加入从别的数据源获得的数据等。另外，基于平均值和数据分布等统计观点查看数据，或者使用各种图表对数据进行可视化，把握数据的整体情况也很重要。此外，有时还需要对数据进行正则化处理。

这些操作被称为数据预处理。有这样一种说法：机器学习工作 80% 以上的时间花在了数据预处理上。

专栏 scikit-learn 包

scikit-learn 是一个机器学习库，包含了各种用于机器学习的工具。

这个库以 BSD 许可证开源，谁都可以免费、自由地使用。在写作本书时（2019 年 3 月），它的最新版本是 0.20.3。scikit-learn 实现了许多有监督学习和无监督学习的算法，是一套包含了用于评估的工具、方便的函数、示例数据集等的工具套件。在机器学习领域，scikit-learn 已成为事实上的标准库，它具有两大优点：一是操作方法统一；二是易于在 Python 中使用。关于 Python 环境的设置和 scikit-learn 的安装方法，请参考第 5 章。

数据和学习种类

前面说过，没有数据，就不能进行机器学习。具体来说，机器学习需要的是什么样的数据呢？

机器学习需要的是二维的表格形式的数据（根据解决问题的目的不同，存在例外的情况）。表格的列中含有表示数据本身特征的多种信息，行则是由多个信息构成的数据集。接下来，我们看一个更具体的例子：学校的某个社团有 4 名学生，下面的表 1-4 是每个学生的姓名、身高、体重、出生日期和性别信息的数据。

▼表 1-4 表格形式的学生数据

姓 名	身高 (cm)	体重 (kg)	出生日期	性 别
赵小刚	165	60	1995-10-02	男
钱小花	150	45	1996-01-20	女
孙小明	170	70	1995-05-29	男
李小芳	160	50	1995-08-14	女

我们思考一下用机器学习进行性别预测的问题。

因为要预测的是性别，所以性别列的男或女的数据就是预测对象。本书把预测对象的数据称为**目标变量**。不过，根据分类的场景的不同，有时也称为**标签或类别标签数据**，对应的英文单词为 **target**。

除了性别之外的 4 个列（姓名、身高、体重、出生日期）是用于预测的原始数据。本书将用于预测的原始数据称为**特征值**，根据场景的不同，有时也称为**特征变量或输入变量**，对应的英文单词为 **feature**。

了解示例数据

我们看一下 **scikit-learn** 包中内置的示例数据。这里显示了部分鸢尾花（**iris**）数据（表 1-5）。Python 生态圈中用于处理数据的工具有 **pandas**，它常与 **scikit-learn** 搭配使用。关于使用 **pandas** 处理数据的方法，请参考后文的“使用 **pandas** 理解和处理数据”部分。

下面输出数据的基本信息。

▼示例代码

```
import pandas as pd
from sklearn.datasets import load_iris
data = load_iris()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.DataFrame(data.target, columns=["Species"])
df = pd.concat([X, y], axis=1)
df.head()
```

▼表 1-5 部分鸢尾花数据

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

列方向上有 sepal length (cm)、sepal width (cm)、petal length (cm)、petal width (cm)、Species 这 5 种信息，意思分别是鸢尾花的萼片长度、萼片宽度、花瓣长度、花瓣宽度、品种。前面 4 列是表示特征的特征值，最后 1 列是目标变量。在这个数据集中，目标变量的值为 0、1、2 这 3 个值之一。

本书在讲解的过程中使用了基于 scikit-learn 库编写的代码。下面将讲解 scikit-learn 的大致用法。不过本书不会全面讲解 scikit-learn 的功能。关于 scikit-learn 的详细信息，请参考官方文档和其他图书。

▶ 有监督学习（分类）的例子

本节将介绍基于有监督学习解决分类问题的实现方法。

下面依次来看例题和实现方法。

■ 例题

例题采用的是美国威斯康星州乳腺癌数据集。这个数据集中包含 30 个特征值，目标变量的值为“良性”或者“恶性”。数据数量有 569 条，其中“恶性”（M）数据 212 条，“良性”（B）数据 357 条。换言之，这是根据 30 个特征值判断结果是恶性还是良性的二元分类问题。

下面看一下数据长什么样子（表 1-6）。

▼表 1-6 部分乳腺癌数据

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension
0	17.99	10.38	122.80	1001.0	0.118 40	0.277 60	0.3001	0.147 10	0.2419	0.078 71
1	20.57	17.77	132.90	1326.0	0.084 74	0.078 64	0.0869	0.070 17	0.1812	0.056 67
2	19.69	21.25	130.00	1203.0	0.109 60	0.159 90	0.1974	0.127 90	0.2069	0.059 99
3	11.42	20.38	77.58	386.1	0.142 50	0.283 90	0.2414	0.105 20	0.2597	0.097 44
4	20.29	14.34	135.10	1297.0	0.100 30	0.132 80	0.1980	0.104 30	0.1809	0.058 83
...	...	...	...	...	...	...	...	...	...	...
...	worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension
...	25.38	17.33	184.60	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601	0.118 90
...	24.99	23.41	158.80	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.089 02
...	23.57	25.53	152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.8758
...	14.91	26.50	98.87	567.7	0.2098	0.8663	0.6869	0.2575	0.6638	0.173 00
...	22.54	16.67	152.20	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364	0.076 78

这份数据可以通过 `scikit-learn` 包读取。

▼示例代码

```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
```

这段代码用于导入 `scikit-learn` 内置的读取数据集的函数，并将所读取的数据保存在变量 `data` 中。

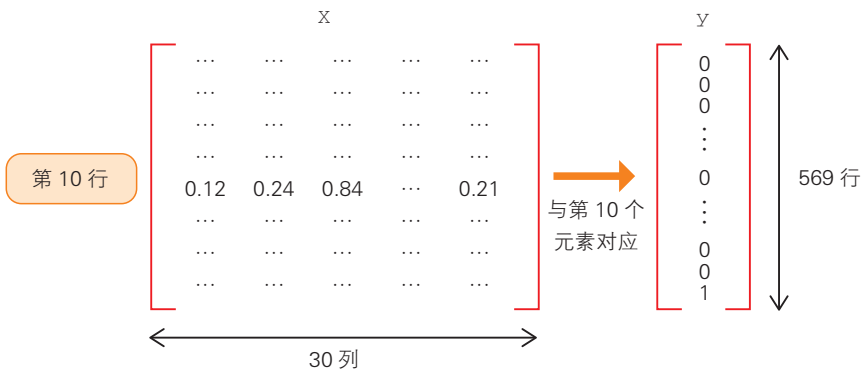
接下来，从数据集中取出特征值赋给 `x`，取出目标变量赋给 `y`。

▼示例代码

```
X = data.data
y = data.target
```

`x` 由多个特征值向量构成，我们将其作为矩阵处理，因此遵照惯例使用大写字母作为变量名。`y` 是目标变量的向量，其元素值的含义为：0 表示恶性（M），1 表示良性（B）。

`x` 是大小为 569×30 的数据，可将其看作 569 行 30 列的矩阵。虽然 `y` 是向量，但把它当作 569 行 1 列的矩阵后，`x` 和 `y` 的行就能一一对应了。比如特征值 `x` 的第 10 行与目标变量 `y` 的第 10 个元素相对应（图 1-7）。



▲图 1-7 与 y 的第 10 个元素对应

要想详细了解这个数据集，需要具备相应的医学知识，但是这里我们仅将其作为数值，对其进行有监督学习的二元分类。特征值共有 30 个，分为平均值、误差值、最差值 3 类，每类包括 10 项，分别为半径、纹理、面积等。这次我们着眼于平均值、误差值、最差值这 3 类数据中的平均值（图 1-8）。

平均值										接下一行
mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	
误差值										接下一行
radius error	texture error	perimeter error	area error	smoothness error	compactness error	concavity error	concave points error	symmetry error	fractal dimension error	
最差值										接下一行
worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension	

▲图 1-8 特征值的种类

▼示例代码

```
X = X[:, :10]
```

这行操作使得只有平均值被重新赋值给了变量 x，用作特征值的数据现在缩减到了 10 项。

实现方法

下面趁热打铁，基于美国威斯康星州乳腺癌数据集创建并训练进行二元分类的模型。这里使用

的分类算法是逻辑回归。虽然算法名中有“回归”二字，却能用于分类，详细内容请参考 2.3 节。

▼ 示例代码

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
```

注意 在使用 scikit-learn 进行模型的初始化和训练时，读者有可能会看到输出的警告信息。警告信息是 FutureWarning，即对将来有可能会变更的功能的通知，在训练不收敛时可能会出现。本书没有提及警告的输出，如果读者在实践中发现有警告输出，请根据警告内容采取相应的措施。

为了使用逻辑回归模型，上面的代码导入了 scikit-learn 的 LogisticRegression 类，然后创建了 LogisticRegression 类的实例，并将已初始化的模型赋给了 model。

▼ 示例代码

```
model.fit(X, y)
```

上面的代码使用 model（LogisticRegression 的实例）的 fit 方法训练模型，方法的参数是特征值 x 和目标变量 y 。

在调用 fit 方法后，model 成为学习后的模型。

▼ 示例代码

```
y_pred = model.predict(X)
```

上面的代码使用学习后的模型 model 的 predict 方法对学习时用到的特征值 x 进行预测，并将预测结果赋给变量 y_pred。

■ 评估方法

下面介绍分类的评估方法。

首先看一下正确率（详见第 4 章）。这里使用 scikit-learn 的 accuracy_score 函数查看正确率。

▼ 示例代码

```
from sklearn.metrics import accuracy_score
accuracy_score(y, y_pred)
```

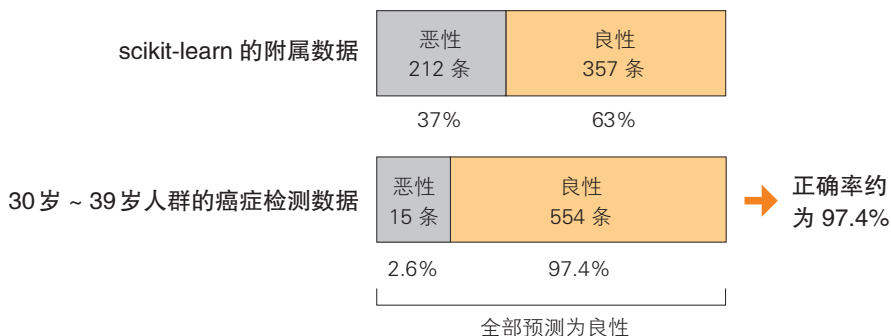
```
0.9086115992970123
```

代码的输出结果是学习后的模型预测的结果 y_{pred} 相对于作为正确答案的目标变量 y 的正确率。

这个验证通常应使用另外准备的一些不用于学习的数据来进行，否则会产生过拟合（overfitting）问题。过拟合是有监督学习的一个严重问题。有监督学习追求的是正确预测未知的数据，但是现在输出的正确率是使用学习时用过的数据计算出来的。这就意味着我们不知道模型对于未用于学习的未知数据的预测性能的好坏，不知道得到的学习后的模型是不是真正优秀。关于过拟合，详见 4.1 节的“模型的过拟合”部分。

关于评估方法，还有其他问题需要考虑。比如，只看正确率就能判断结果是否正确吗？根据数据的特性不同，有些情况下不能保证分类是正确的。这次用的数据中有“恶性”数据 212 条，“良性”数据 357 条，可以说是在一定程度上均衡的数据。

对于“良性”“恶性”极不均衡的数据，光看正确率无法判断结果是否正确。我们再以另外一组数据为例，看一下 30 岁 ~ 39 岁人群的癌症检测数据。通常来说，诊断为恶性的数据只占整体的百分之几，大多数人没有肿瘤或者肿瘤是良性的。对于这样的数据，如果模型将所有的样本都判断为良性的，那么尽管正确率很高，但光看正确率也不能正确评估这个模型（图 1-9）。



▲ 图 1-9 光看正确率无法正确评估模型

关于这些内容，第 4 章会详细介绍。

无监督学习（聚类）的例子

下面看一下无监督学习的聚类问题的实现方法的各个步骤。与前面一样，这里我们也使用 `scikit-learn` 包。

例题

例题采用的是 `scikit-learn` 包内置的与葡萄酒种类有关的数据集。这个数据集有 13 个特征值，目标变量是葡萄酒种类（表 1-7）。由于这次介绍的是无监督学习的聚类算法，所以不使用目标变量。简单起见，本次只使用 13 个特征值中的 `alcohol`（酒精度）和 `color_intensity`（色泽）两个特征值（表 1-8）。我们对这个数据集应用 *k*-means 聚类算法，将其分割为 3 个簇。

▼表 1-7 葡萄酒数据的特征值

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_of_diluted_wines	proline
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065.0
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050.0
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185.0
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480.0
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735.0

▼表 1-8 本次使用的两个特征值

	alcohol	color_intensity
0	14.23	5.64
1	13.20	4.38
2	13.16	5.68
3	14.37	7.80
4	13.24	4.32

下面使用 `scikit-learn` 包加载这个数据集。

▼ 示例代码

```
from sklearn.datasets import load_wine
data = load_wine()
```

上面的代码用于导入 `scikit-learn` 内置的读取葡萄酒数据集的函数，并将读取的数据保存在变量 `data` 中。

接着，仅从数据集中选择 `alcohol` 列和 `color_intensity` 列作为特征值赋给 `x`。这么做是为了在显示结果时，只用二维图形对结果进行可视化。

▼ 示例代码

```
X = data.data[:, [0, 9]]
```

特征值 `x` 是 178 行 2 列的数据。

■ 实现方法

下面使用 *k*-means 算法实现聚类。

▼ 示例代码

```
from sklearn.cluster import KMeans
n_clusters = 3
model = KMeans(n_clusters=n_clusters)
```

上面的代码导入并使用了实现 *k*-means 算法的 `KMeans` 类。

初始化 `KMeans` 类，把它作为学习前的模型赋给变量 `model`。通过 `n_clusters` 参数，指示模型将数据分为 3 个簇。

▼ 示例代码

```
pred = model.fit_predict(X)
```

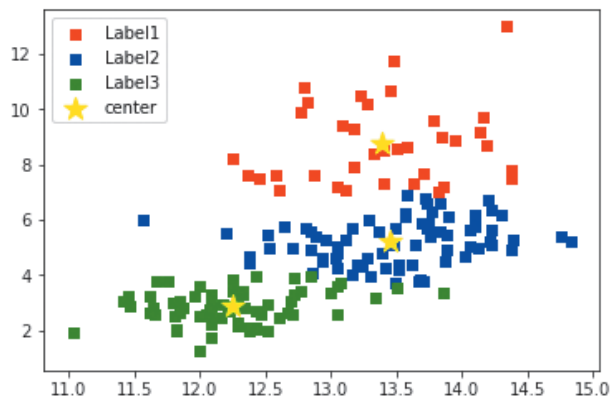
上面的代码用于向学习前的模型 `model` 的 `fit_predict` 方法传入特征值数据。预测结果赋给变量 `pred`。

下面看一下赋给 `pred` 的数据是如何聚类的。

查看结果

这里将聚类的结果可视化。

由于本次使用的特征值只有两种，所以绘制二维图形即可实现结果的可视化。图 1-10 是以图形展示的聚类结果。图形中每个数据点对应的是一种葡萄酒。从数据点的颜色可以看出每种酒属于哪个簇。3 个黄色的星星是各个簇的重心，是这 3 个簇的代表点。



▲ 图 1-10 特征值的可视化

原本以酒精度、色泽变量表示的葡萄酒，现在以“属于哪个簇”这种简洁直观的形式展示了出来。此外，要想了解各个簇具有什么特征，只需查看作为代表点的重心的值即可。比如，第 3 个簇的特点是“酒精度低、色泽淡”。

通过 k -means 算法实现的聚类是以“将酒精度百分之多少以上的数据分到第 1 个簇”之类的规则进行聚类的，这些规则不是由人预先设置的，而是由算法自动进行聚类得出的。这一点很重要，说明这个算法具有通用性，可应用于葡萄酒之外的数据。

无监督学习的评估方法将在第 3 章介绍各个算法时进行说明，请参考相应内容。

可视化

可视化是利用图形等把握数据的整体情况的方法。在机器学习领域中，许多场景下需要进行可视化。有时用于了解数据的概况，有时用于以图形展示机器学习的结果。

这里介绍一下使用 Python 进行可视化的方法，书中也将展示作为可视化结果的图形等。

工具介绍

这里使用常用的 Python 可视化工具 Matplotlib。Matplotlib 具有许多可视化功能。在可视化时，为了使图形美观，需要编写多行 Python 代码来设置坐标轴、标签、布局和配色等。代码行数的增加容易让人觉得晦涩难懂，但其实用于输出图形的重要部分的代码只有寥寥几行。

在使用 Matplotlib 实现可视化后，我们就能很容易地把握数据的偏差和特征等信息，所以要掌握它的用法。

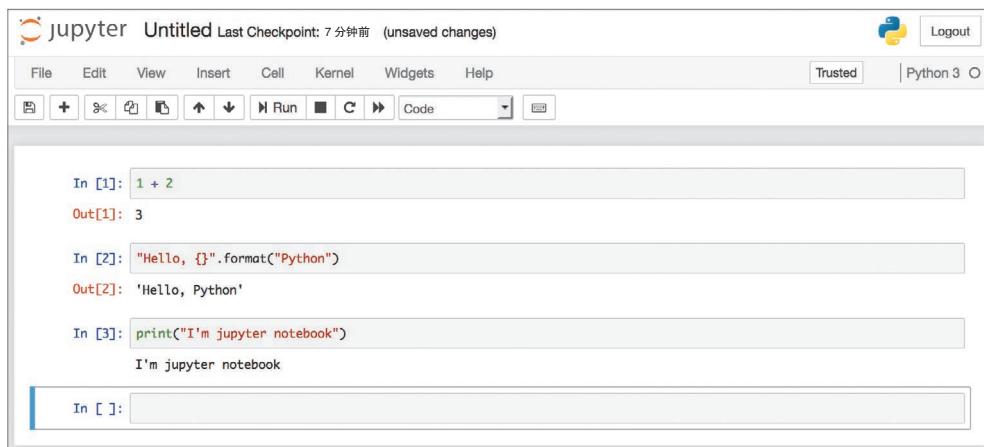
Python 的可视化工具不只有 Matplotlib，还有以下几种。

- pandas
pandas 是处理数组数据的库，也具有可视化功能。
- seaborn
seaborn 在 Matplotlib 的基础上强化了表现力，用起来更简单。
- Bokeh
Bokeh 使用了 JavaScript，可用于显示动态图形。

在浏览器上显示

使用 Jupyter Notebook 可以简单地在浏览器上显示数据的可视化结果。

Jupyter Notebook 是在 Web 浏览器上运行 Python 等语言代码的环境（图 1-11）。



▲ 图 1-11 在 Jupyter Notebook 上运行 Python 代码

下面依次来看一下例题及其实现。

在 Jupyter Notebook 上运行 Python 程序，进行机器学习的实验。Jupyter Notebook 不仅能运行 Python，还可以作为 R 等语言的运行环境。

关于 Jupyter Notebook 的安装和环境设置方法，请参考第 5 章。

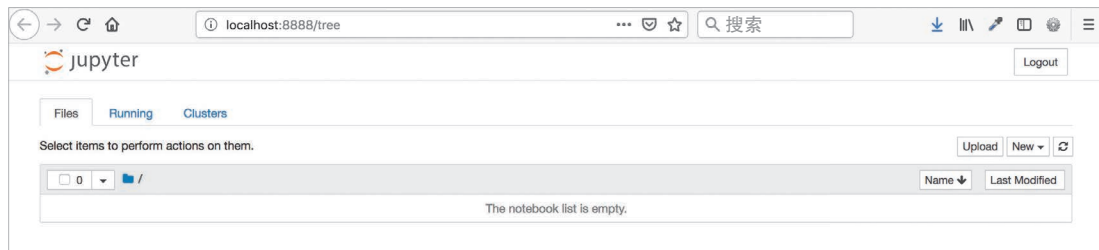
启动方法

在 Jupyter Notebook 安装成功后，就可以使用 `jupyter` 命令了。我们可以从命令提示行和终端运行 `jupyter notebook` 命令。

```
$ jupyter notebook
```

在命令运行后，浏览器将自动打开，显示已运行的当前目录的文件或文件夹（图 1-12）。在 Web 浏览器上的单元格内输入程序代码后，页面上将显示运行结果。运行结果为 notebook 格式的文件，以 `.ipynb` 扩展名保存。用户还可以在自己的计算机上打开其他人创建的 `.ipynb` 文件，按顺序执行每个单元格并查看其运行结果。也可以查看中间处理的变量，改变条件再次运行以查看不同的结果。

此外，把这种文件提交到 GitHub 后，不仅可以分享运行结果，还可以在 GitHub 上以图形显示运行结果。



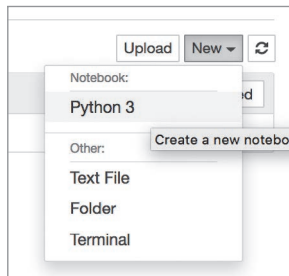
▲图 1-12 显示文件列表

用法

从右上角的 New 菜单选择 Python 3，可新建 notebook 格式的文件（图 1-13）。

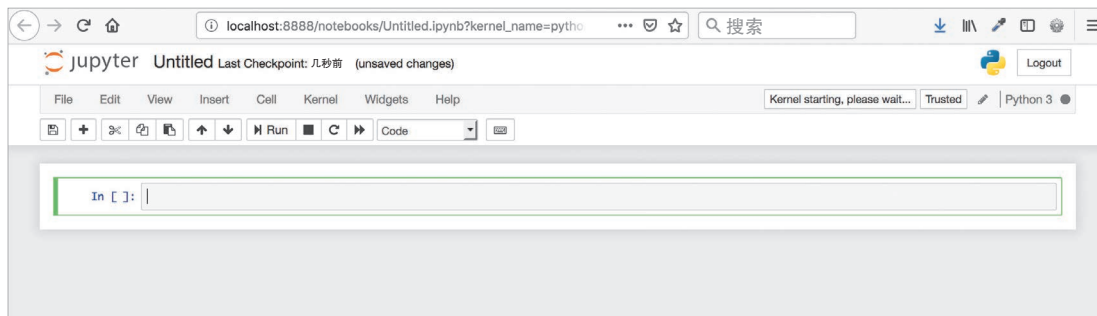
Web 浏览器将打开新建的文件。

在叫作单元格的输入框里编写程序代码。此外，单元格有不同的



▲图 1-13 新建 notebook 文件

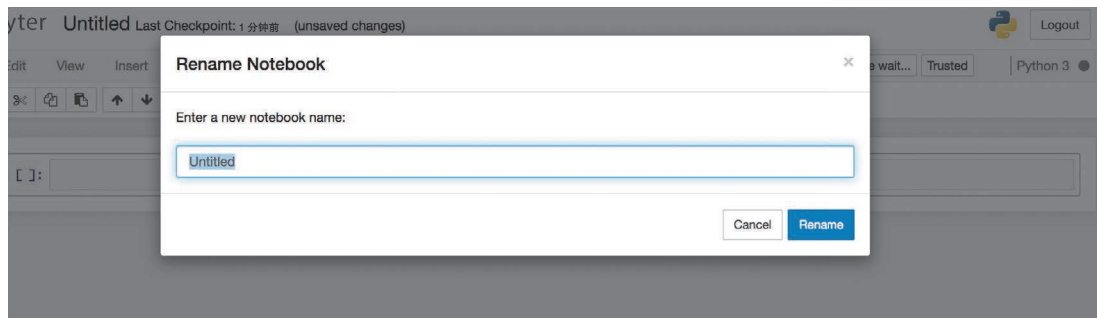
种类，可通过界面上的 Code 和 Markdown 等下拉菜单决定单元格的作用。默认选项是 Code，系统将其识别为可运行的单元格（图 1-14）。



▲图 1-14 可运行的 notebook

在单元格内输入程序代码后，使用 Enter 键在单元格内换行，使用 Ctrl 和 Enter 组合键运行程序代码。另外，可使用 Shift 和 Enter 组合键运行程序代码，并移动到下一个单元格。

点击上部的标题 Untitled，可修改文件的标题（图 1-15）。Jupyter Notebook 会自动在此处确定的标题后附加扩展名 .ipynb，以此作为文件名创建文件。



▲图 1-15 修改 notebook 的标题

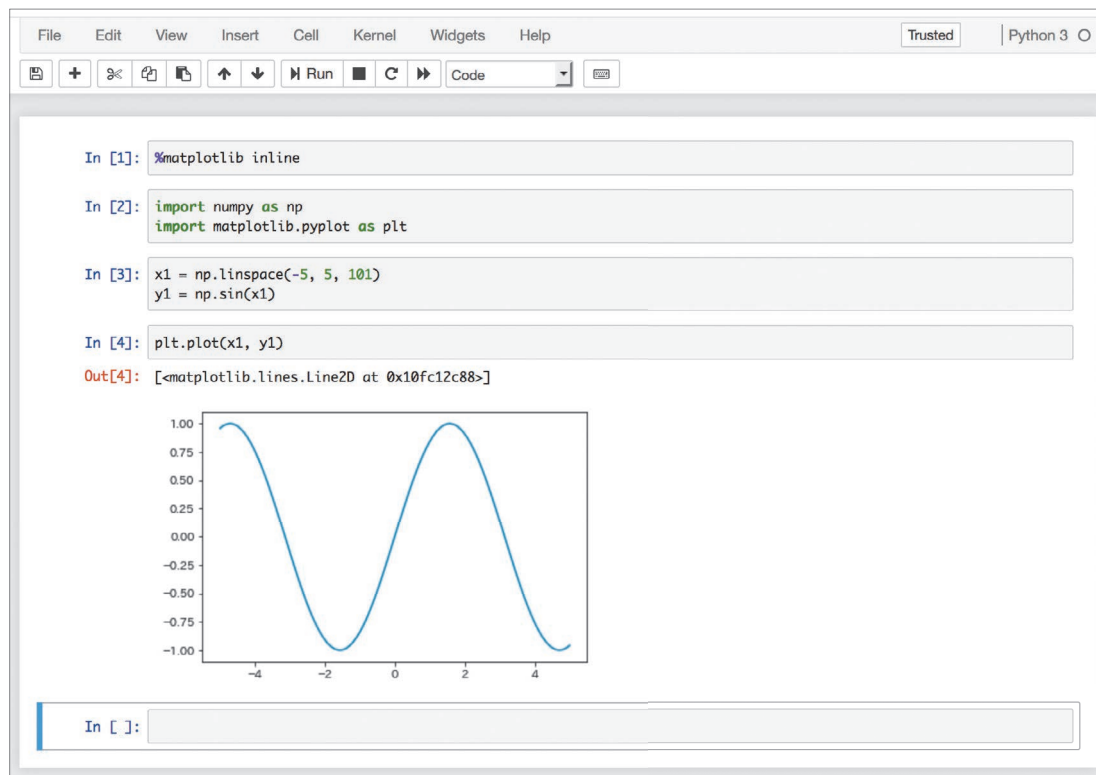
从菜单选择 Save and Checkpoint，可保存当前的状态。

上面简要地介绍了 Jupyter Notebook 的使用方法。除此之外，它还有很多方便用法，感兴趣的读者可参考官网、相关的文章和图书。

图形的种类和画法：使用 Matplotlib 显示图形的方法

在页面上显示图形

在 Jupyter Notebook 页面上可显示图形。在 Code 单元格内，运行 `%matplotlib inline` 这个以 `%` 开始的“魔法”命令后，即使不运行后面将介绍的 `show` 方法，页面上也会输出图形（图 1-16）。



▲图 1-16 在 notebook 内显示图形

下面看一下图 1-16 中的代码。

▼示例代码

```
import numpy as np
import matplotlib.pyplot as plt
```

上面的代码为了生成数据而导入了 `numpy`，为了显示图形而导入了 `matplotlib`。`NumPy` 是以数组形式处理数据并进行高效计算的 `Python` 第三方包。习惯上分别以别名 `np` 和 `plt` 调用 `numpy` 和 `matplotlib` 的 `pyplot`。

下面以 `sin` 曲线为例，显示其图形。

▼ 示例代码

```
x1 = np.linspace(-5, 5, 101)
y1 = np.sin(x1)
```

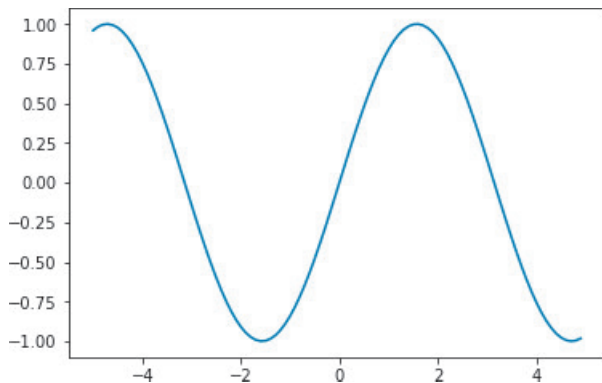
- `x1` 中保存的是为显示 `sin` 曲线而生成的从 `-5` 到 `5` 的 `101` 个数据。
- `y1` 中保存的是使用 `NumPy` 的 `sin` 函数生成的数据。

下面以用 `Matplotlib` 绘制 `sin` 曲线的图形为例进行说明。

最简单的显示图形的方法是 `plt.plot(x1, y1)`（图 1-17）。

▼ 示例代码

```
plt.plot(x1, y1)
```

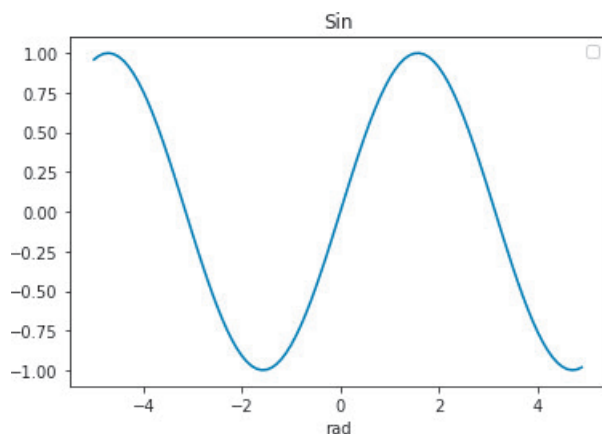


▲ 图 1-17 `sin` 曲线的图形

下面介绍使用 `Matplotlib` 绘制图形的代码的标准写法。前面只是简单地调用了 `plt.plot`，这种做法没有明确输出对象，只是声明“在这里绘图”，比较粗糙。严密的做法应为先创建要绘制的对象，再输出图形，代码及图形如下（图 1-18）。

▼ 示例代码

```
fig, ax = plt.subplots()
ax.set_title("Sin")
ax.set_xlabel("rad")
ax.plot(x1, y1)
handles, labels = ax.get_legend_handles_labels()
ax.legend(handles, labels)
plt.show()
```



▲ 图 1-18 先创建对象再输出图形

上面的代码包含了显示标签和坐标轴名称等信息的处理，因此变成了7行，但其中用于显示图形的主要代码只有 `ax.plot(x1, y1)` 一行。虽然这种方法更受欢迎，不过如果要输出简单的图形，可以使用 `plt.plot(x1, y1)` 形式。请记住，方法共有两种：一种是 `plt.plot` 这种简易方法；另一种是 `ax.plot` 这种严密地面向对象进行声明的方法。最后运行 `plt.show()`。如果已运行魔法命令 `%matplotlib inline`，那就不用特意调用 `show` 了。为了使代码在 Jupyter Notebook 之外的环境中也能输出图形，这里特意编写了这一行代码。不管写不写这行代码，图形都会在 Jupyter Notebook 中显示。

■ 绘制各种图形

首先生成用于显示图形的数据。

▼ 示例代码

```
x2 = np.arange(100)
y2 = x2 * np.random.rand(100)
```

x2 中保存的是元素为“从 0 到 99 的整数”的数组。

y2 中保存的是元素为“在从 0 到 1 的范围内随机选出的 100 个数据”的数组与刚才的变量 x2 相乘的结果。

下面将使用这两个变量绘制各种图形。

接下来要展示的是通过 plt 变量声明图形形式的简易方法的示例。对于前面代码中明示图形位置的 ax 变量，我们也可以用同样的方式输出其图形。

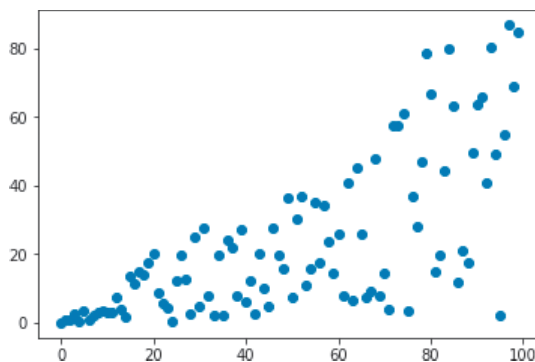
● 散点图

使用 scatter 方法绘制散点图。

▼ 示例代码

```
plt.scatter(x2, y2)
```

x2 和 y2 的散点图如图 1-19 所示。



▲ 图 1-19 散点图

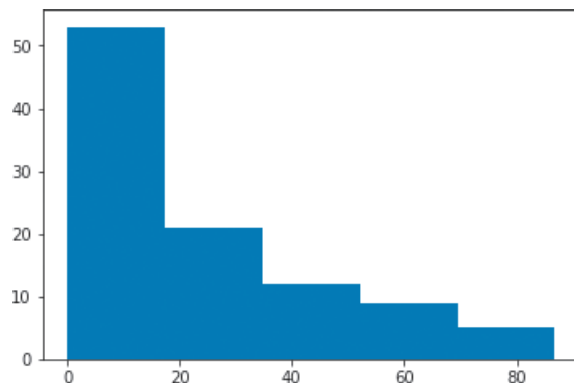
● 直方图

使用 hist 方法绘制直方图。

▼ 示例代码

```
plt.hist(y2, bins=5)
```

设 y_2 的直方图的 bin 为 5，输出的图形如图 1-20 所示。



▲图 1-20 直方图

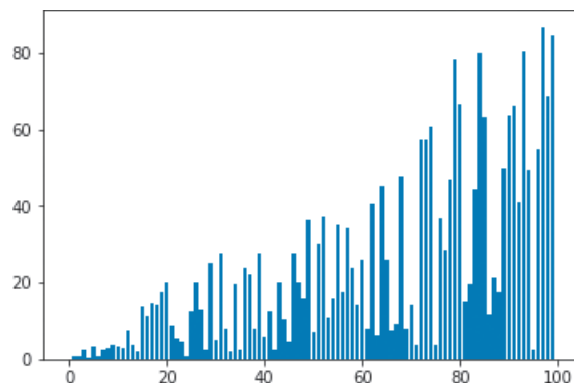
● 柱状图

使用 `bar` 方法绘制柱状图。

▼ 示例代码

```
plt.bar(x2, y2)
```

输出的柱状图如图 1-21 所示。



▲图 1-21 柱状图

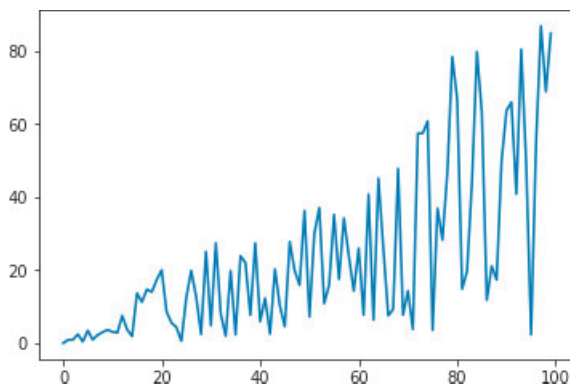
● 折线图

使用 `plot` 方法绘制折线图。

▼ 示例代码

```
plt.plot(x2, y2)
```

`x2` 和 `y2` 的折线图如图 1-22 所示。



▲ 图 1-22 折线图

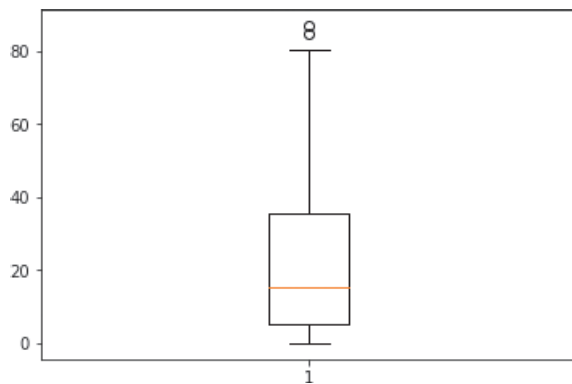
● 箱形图

使用 `boxplot` 方法绘制箱形图。

▼ 示例代码

```
plt.boxplot(y2)
```

用箱形图输出 `y2` 的数据，如图 1-23 所示。箱形图是查看数据分布的优秀的可视化方法。



▲图 1-23 箱形图

红酒数据集

下面对 scikit-learn 内置的红酒数据进行可视化。

▼示例代码

```
from sklearn.datasets import load_wine
data = load_wine()
```

加载与红酒有关的数据，并将数据保存在变量 data 中。

▼示例代码

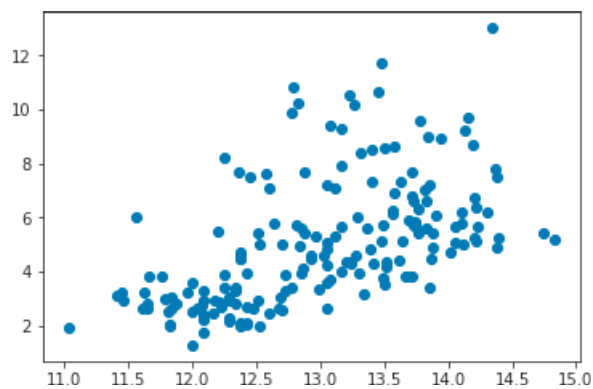
```
x3 = data.data[:, [0]]
y3 = data.data[:, [9]]
```

将要显示的索引为 0 的 alcohol（酒精度）和索引为 9 的 color_intensity（色泽）数据分别赋值给 x3 和 y3。

下面输出散点图（图 1-24）。

▼示例代码

```
plt.scatter(x3, y3)
```

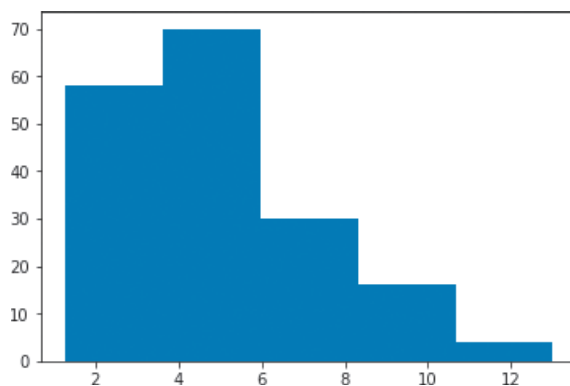


▲图 1-24 红酒数据集的散点图

下面输出 y_3 的直方图（图 1-25）。

▼示例代码

```
plt.hist(y3, bins=5)
```



▲图 1-25 y_3 的直方图

通过可视化查看与红酒数据集相关的两个图形，我们可以了解数据的特性。

使用 pandas 理解和处理数据

注意 在进行机器学习时，有时会查看特征值、对数据进行取舍选择或再加工等。下面将介绍如何使用 pandas 库的数据可视化的方便功能去了解数据的情况。已经知道 pandas 基本用法的读者和想先了解机器学习算法的读者，跳过这一部分也没关系。

下面介绍使用 pandas 库进行数据可视化的方便功能。

▼ 示例代码

```
import pandas as pd
```

上面的代码导入了常用于机器学习数据的变形等操作的 pandas。与 numpy 的 np 一样，习惯上用 pd 来调用它。

▼ 示例代码

```
from sklearn.datasets import load_wine
data = load_wine()
df_X = pd.DataFrame(data.data, columns=data.feature_names)
```

然后将红酒数据转换为 pandas 的 DataFrame。

DataFrame 可以方便地处理像 Excel 工作表那样的二维数据。df_X 是 DataFrame 形式的特征值。

下面的代码调用 head 方法输出了数据集前 5 行的数据（表 1-9），用于查看数据集中包含了什么样的数据。

▼ 示例代码

```
df_X.head()
```

▼表 1-9 红酒数据

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_of_diluted_wines	proline
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065.0
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050.0
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185.0
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480.0
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735.0

接着将红酒数据的目标变量转换为 pandas 的 DataFrame。

▼示例代码

```
df_y = pd.DataFrame(data.target, columns=["kind(target)"])
```

接下来看一下转换后的数据。做法和刚才相同：调用 head 方法。从表 1-10 中可以看出，df_y 是名副其实的目标变量数据。

▼示例代码

```
df_y.head()
```

为了便于使用，我们将这些数据合并在一起。下面使用 pandas 的 concat 将特征值 df_x 和目标变量 df_y 合并。

▼示例代码

```
df = pd.concat([df_x, df_y], axis=1)
```

输出数据的前几行看看。下面的代码使用 head 方法输出了合并结果的前 5 行（表 1-11）。这样就得到了包含特征值和目标变量的数据。

▼示例代码

```
df.head()
```

▼表 1-10 红酒数据的目标变量

	kind(target)
0	0
1	0
2	0
3	0
4	0

▼表 1-11 红酒数据的特征值和目标变量

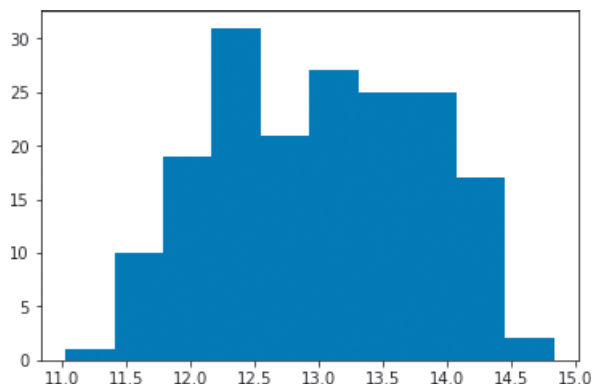
	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_of_diluted_wines	proline	kind(target)
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065.0	0
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050.0	0
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185.0	0
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480.0	0
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735.0	0

接下来，通过可视化和数理统计来分析这份数据。

图 1-26 是以直方图的形式输出的 alcohol 列的数据。由于下面的代码没有指定 bins 参数，所以程序使用默认的参数值 10，输出了有 10 个区间的直方图。

▼示例代码

```
plt.hist(df.loc[:, "alcohol"])
```

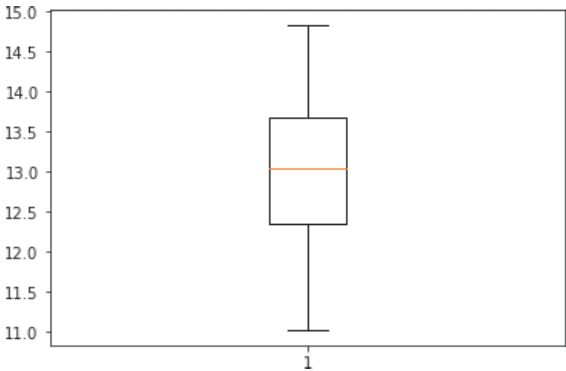


▲图 1-26 有 10 个区间的直方图

图 1-27 是以箱形图显示的同一个 alcohol 列的数据。接下来开始使用 pandas 的统计功能。

▼示例代码

```
plt.boxplot(df.loc[:, "alcohol"])
```



▲图 1-27 以箱形图显示的 alcohol 列的数据

下面的代码使用 `corr` 方法汇总计算并输出了相关系数（表 1-12）。相关系数越接近于 1，越表明数据之间是正相关关系；越接近于 -1，越表明数据之间是负相关关系。换言之，如果相关系数在 0 左右，表明数据列之间的相关性很低。

▼示例代码

```
df.corr()
```

▼表 1-12 相关系数

	alcohol	malic_acid	ash	alkalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280od315_of_diluted_wines	proline	kind(target)
alcohol	1.000 000	0.094 397	0.211 545	-0.310 235	0.270 798	0.289 101	0.236 815	-0.155 929	0.136 698	0.546 364	-0.071 747	0.072 343	0.643 720	-0.328 222
malic_acid	0.094 397	1.000 000	0.164 045	0.288 500	-0.054 575	-0.335 167	-0.411 007	0.292 977	-0.220 746	0.248 985	-0.561 296	-0.368 710	-0.192 011	0.437 776
ash	0.211 545	0.164 045	1.000 000	0.443 367	0.286 587	0.128 980	0.115 077	0.186 230	0.009 652	0.258 887	-0.074 667	0.003 911	0.223 626	-0.049 643
alkalinity_of_ash	-0.310 235	0.288 500	0.443 367	1.000 000	-0.083 333	-0.321 113	-0.351 370	0.361 922	-0.197 327	0.018 732	-0.273 955	-0.276 769	-0.440 597	0.517 859
magnesium	0.270 798	-0.054 575	0.286 587	-0.083 333	1.000 000	0.214 401	0.195 784	-0.256 294	0.236 441	0.199 950	0.055 398	0.066 004	0.393 351	-0.209 179
total_phenols	0.289 101	-0.335 167	0.128 980	-0.321 113	0.214 401	1.000 000	0.864 564	-0.449 935	0.612 413	-0.055 136	0.433 681	0.699 949	0.498 115	-0.719 163
flavanoids	0.236 815	-0.411 007	0.115 077	-0.351 370	0.195 784	0.864 564	1.000 000	-0.537 900	0.652 692	-0.172 379	0.543 479	0.787 194	0.494 193	-0.847 498
nonflavanoid_phenols	-0.155 929	0.292 977	0.186 230	0.361 922	-0.256 294	-0.449 935	-0.537 900	1.000 000	-0.365 845	0.139 057	-0.262 640	-0.503 270	-0.311 385	0.489 109
proanthocyanins	0.136 698	-0.220 746	0.009 652	-0.197 327	0.236 441	0.612 413	0.652 692	-0.365 845	1.000 000	-0.025 250	0.295 544	0.519 067	0.330 417	-0.499 130
color_intensity	0.546 364	0.248 985	0.258 887	0.018 732	0.199 950	-0.055 136	-0.172 379	0.139 057	-0.025 250	1.000 000	-0.521 813	-0.428 815	0.316 100	0.265 668
hue	-0.071 747	-0.561 296	-0.074 667	-0.273 955	0.055 398	0.433 681	0.543 479	-0.262 640	0.295 544	-0.521 813	1.000 000	0.565 468	0.236 183	-0.617 369
od280od315_of_diluted_wines	0.072 343	-0.368 710	0.003 911	-0.276 769	0.066 004	0.699 949	0.787 194	-0.503 270	0.519 067	-0.428 815	0.565 468	1.000 000	0.312 761	-0.788 230
proline	0.643 720	-0.192 011	0.223 626	-0.440 597	0.393 351	0.498 115	0.494 193	-0.311 385	0.330 417	0.316 100	0.236 183	0.312 761	1.000 000	-0.633 717
kind(target)	-0.328 222	0.437 776	-0.049 643	0.517 859	-0.209 179	-0.719 163	-0.847 498	0.489 109	-0.499 130	0.265 668	-0.617 369	-0.788 230	-0.633 717	1.000 000

下面再介绍一个查看数据情况的方法。

`describe` 方法用于输出每列的统计信息。输出的统计信息从上到下依次为行数、平均值、标准差、最小值、25 百分位数、中位数、75 百分位数、最大值（表 1-13）。从统计信息可以看出每列包含的数据具有何种特性、有没有缺损等信息。

▼ 示例代码

```
df.describe()
```

▼ 表 1-13 统计信息

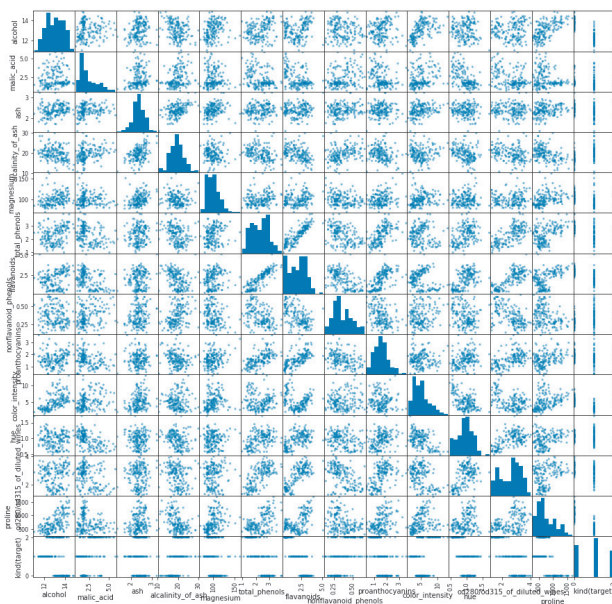
	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_of_diluted_wines	proline	kind(target)
count	178.000 000	178.000 000	178.000 000	178.000 000	178.000 000	178.000 000	178.000 000	178.000 000	178.000 000	178.000 000	178.000 000	178.000 000	178.000 000	178.000 000
mean	13.000 618	2.336 348	2.366 517	19.494 944	99.741 573	2.295 112	2.029 270	0.361 854	1.590 899	5.058 090	0.957 449	2.611 685	746.893 258	0.938 202
std	0.811 827	1.117 146	0.274 344	3.339 564	14.282 484	0.625 851	0.998 859	0.124 453	0.572 359	2.318 286	0.228 572	0.709 990	314.907 474	0.775 035
min	11.030 000	0.740 000	1.360 000	10.600 000	70.000 000	0.980 000	0.340 000	0.130 000	0.410 000	1.280 000	0.480 000	1.270 000	278.000 000	0.000 000
25%	12.362 500	1.602 500	2.210 000	17.200 000	88.000 000	1.742 500	1.205 000	0.270 000	1.250 000	3.220 000	0.782 500	1.937 500	500.500 000	0.000 000
50%	13.050 000	1.865 000	2.360 000	19.500 000	98.000 000	2.355 000	2.135 000	0.340 000	1.555 000	4.690 000	0.965 000	2.780 000	673.500 000	1.000 000
75%	13.677 500	3.082 500	2.557 500	21.500 000	107.000 000	2.800 000	2.875 000	0.437 500	1.950 000	6.200 000	1.120 000	3.170 000	985.000 000	2.000 000
max	14.830 000	5.800 000	3.230 000	30.000 000	162.000 000	3.880 000	5.080 000	0.660 000	3.580 000	13.000 000	1.710 000	4.000 000	1680.000 000	2.000 000

下面使用 `pandas` 的功能将所有列之间的关系可视化（图 1-28）。

使用 `scatter_matrix` 输出散点图矩阵。这里将 14 列全都输出了出来（图 1-28）。

▼ 示例代码

```
from pandas.plotting import scatter_matrix
_ = scatter_matrix(df, figsize=(15, 15))
```



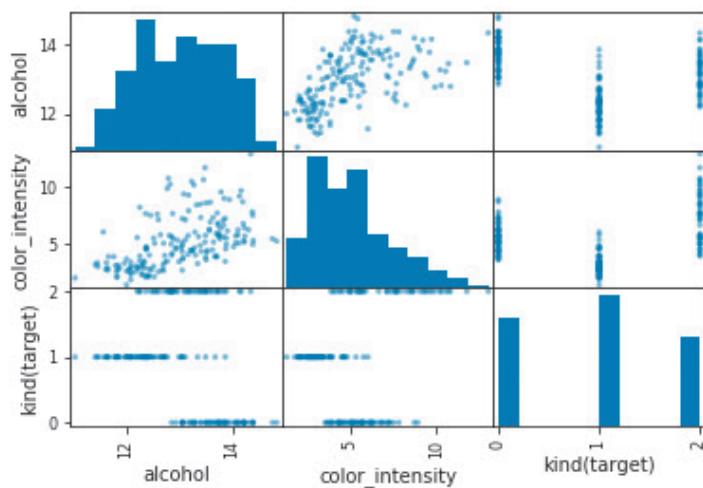
▲ 图 1-28 各列的关联性

下面只查看部分列的关联性。

下列代码输出的是从所有散点图矩阵中选出的索引为 0 的列、索引为 9 的列和最后一列的关联性（图 1-29）。通过像这样减少散点图矩阵输出的列，能够看出更细致的情况。

▼ 示例代码

```
_ = scatter_matrix(df.iloc[:, [0, 9, -1]])
```



▲ 图 1-29 显示重要的关联性

本章小结

本章前半部分介绍了机器学习的基础内容：机器学习包括有监督学习和无监督学习等，存在与分类问题、回归问题、降维和聚类对应的算法。

后半部分介绍了机器学习库 `scikit-learn`，并列举了有监督学习（分类）和无监督学习（聚类）的实现示例。最后，我们了解了使用可视化工具 `Matplotlib` 显示图形的方法。

到这里，事前准备工作就结束了。从下一章开始，我们将正式学习机器学习的算法。

第 2 章

有监督学习

序 号	算 法
1	线性回归
2	正则化
3	逻辑回归
4	支持向量机
5	支持向量机（核方法）
6	朴素贝叶斯
7	随机森林
8	神经网络
9	KNN

2.1 算法 1：线性回归

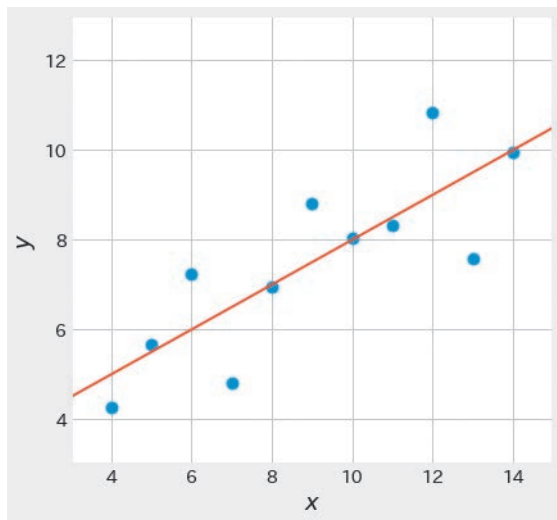
线性回归（linear regression）是用于预测回归问题的算法。该算法不难理解，算法中根据训练数据计算使损失最小的参数的做法是有监督学习算法的共同之处。

概述

线性回归是对“目标变量随着某个特征变量的增大而增大（或者减小）”这种关联性建模的方法。这里以表 2-1 所示的特征变量 x 和目标变量 y 的组合数据为例。图 2-1 是对该数据进行线性回归建模而得到的直线。

▼表 2-1 特征变量 x 和目标变量 y 的组合数据

i	x	y
0	10.0	8.04
1	8.0	6.95
2	13.0	7.58
3	9.0	8.81
4	11.0	8.33
5	14.0	9.96
6	6.0	7.24
7	4.0	4.26
8	12.0	10.84
9	7.0	4.82
10	5.0	5.68



▲图 2-1 线性回归

图 2-1 的直线可写为 $y = w_0 + w_1x$ 。这是我们在中学阶段学过的一次函数， w_1 是斜率（或者叫权重）， w_0 相当于在 y 轴上的截距。斜率 w_1 和截距 w_0 是由有监督学习的算法学到的参数，所以我们称之为学习参数。线性回归的学习参数是如何求得的呢？下面的“算法说明”部分将对此进行讲解。

另外，线性回归算法一般使用一个以上的特征变量创建模型，其中只有一个独立的特征变量的情况叫作一元回归。

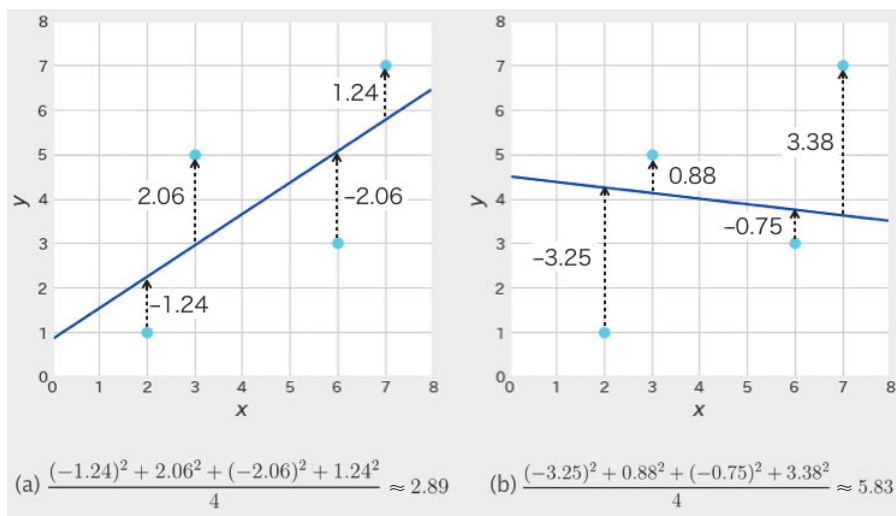
算法说明

对于直线 $y = w_0 + w_1x$ ，只要给定不同的两个点，我们就能求出唯一的 w_0 和 w_1 。但在线性回归中，我们需要根据不在一条直线上的点求出学习参数。下面使用表 2-2 的数据介绍学习参数的求法。

图 2-2a 和图 2-2b 是根据表 2-2 的数据绘制的两条不同的直线，分别是 $y = 0.706x + 0.823$ ， $y = -0.125x + 4.5$ 。这两条直线中的哪一条更好地表示了数据的关联性呢？我们可以通过均方误差进行定量判断。均方误差指的是目标变量和直线的差 $y_i - (w_0 + w_1x_i)$ 的平方的平均值。当存在 n 个数据时，可如下表示：

$$\frac{\sum_{i=1}^n [y_i - (w_0 + w_1x_i)]^2}{n}$$

对图 2-2 的数据计算均方误差，得到图 2-2a 的均方误差约为 2.89，图 2-2b 的均方误差约为 5.83，前者的均方误差更小，因此图 2-2a 能比图 2-2b 更好地表示数据的关联性。



▲ 图 2-2 均方误差的比较

现在我们知道图 2-2a、图 2-2b 这两条直线的均方误差的值是不同的。

▼ 表 2-2 用于线性回归的数据

i	x	y
1	2	1
2	3	5
3	6	3
4	7	7

这两条直线的区别就是学习参数 w_0 、 w_1 。也就是说，改变学习参数 w_0 、 w_1 ，那么计算出的均方误差也会发生变化。这种表示误差和学习参数之间关系的函数叫作**误差函数**（或损失函数）。线性回归需要在各条直线中找出使误差函数值最小的参数（更具体的计算方法请参考后文的“均方误差的最小化方法”部分）。这种计算出使误差函数值最小的参数的思路在其他有监督学习算法中也是通用的。

■ 示例代码

下面尝试对前面的表 2-1 的数据进行线性回归。使用 `LinearRegression` 类创建线性回归模型，并使用 `fit` 方法训练后，输出 `intercept_` 的值以查看截距，输出 `coef_` 的值以查看斜率。

▼ 示例代码

```
from sklearn.linear_model import LinearRegression
X = [[10.0], [8.0], [13.0], [9.0], [11.0], [14.0],
      [6.0], [4.0], [12.0], [7.0], [5.0]]
y = [8.04, 6.95, 7.58, 8.81, 8.33, 9.96,
      7.24, 4.26, 10.84, 4.82, 5.68]

model = LinearRegression()
model.fit(X, y)
print(model.intercept_) # 截距
print(model.coef_) # 斜率
y_pred = model.predict([[0], [1]])
print(y_pred) # 对x=0, x=1的预测结果
```

```
3.0000909090909094
```

```
[0.50009091]
```

```
[3.00009091 3.50018182]
```

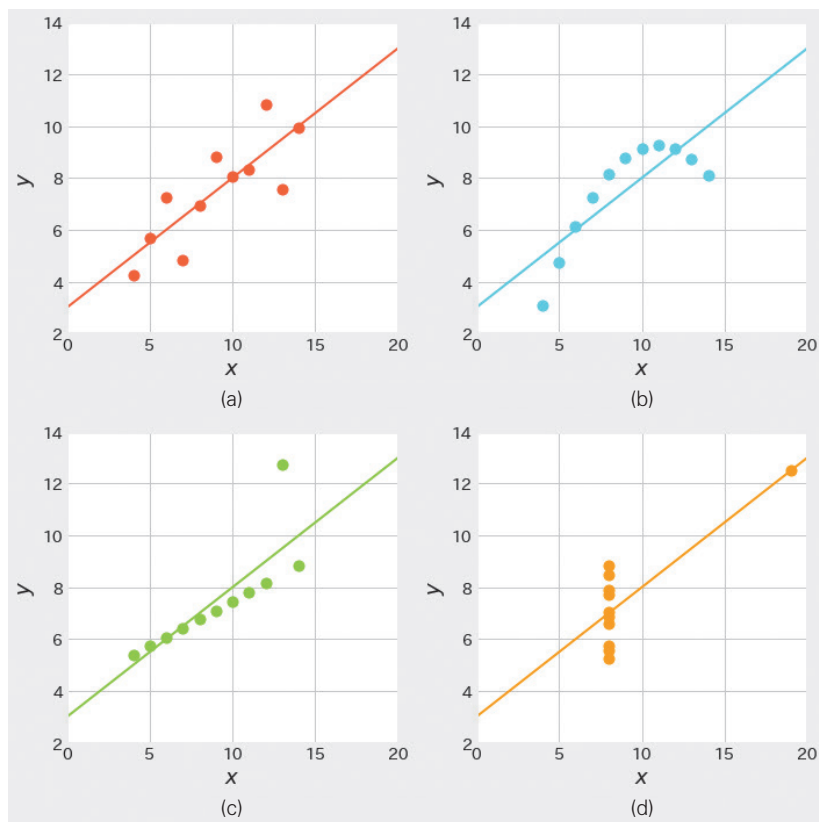
详细说明

线性回归不成功的例子

我们发现线性回归适用于表 2-1 的数据，但是对于有些数据，线性回归可能不成功。

表 2-1 及示例代码中使用的数据是由统计学家 F.J. 安斯库姆 (F.J. Anscombe) 制作的数据集安斯库姆四重奏 (Anscombe's Quartet) 中的一部分。这个数据集的目的是说明可视化的重要性，其中汇总了包含表 2-1 在内的 4 组数据。

图 2-3 显示了对安斯库姆四重奏数据集的每一组数据分别进行线性回归的结果。尽管图 2-3 中的 4 张图是不同的散点图，但线性回归的学习参数 (截距和斜率) 却相同。另外，这些数据的平均值、方差和相关系数也几乎相同。



▲ 图 2-3 对安斯库姆四重奏数据集应用线性回归

下面来仔细看一下各组数据。图 2-3b 的数据看上去是沿曲线分布的，所以假定这组数据是线性分布的就不合适。图 2-3c 中有离群值，应考虑对其进行去除离群值的预处理，或者应用便于处理离群值的其他方法。图 2-3d 是对数据两两之间没有相关关系但存在离群值的数据集应用线性回归而得到回归直线的例子。对原本不遵循线性分布的数据强行进行线性回归也得不到好的结果。拿到数据之后，首先应该进行可视化，再考虑是否进行线性回归。

均方误差的最小化方法

在“算法说明”部分，我们了解了均方误差可用于评估具有不同学习参数的直线。

然而，虽然通过如表 2-3 所示的数据比较了不同的学习参数产生的误差大小，但我们还不了解具体的学习参数的计算方法。

本节将介绍如何获得使均方误差最小的学习参数。

▼表 2-3 学习参数和均方误差的关系

线性回归	w_0	w_1	均方误差
(a)	0.823	0.706	2.89
(b)	4.5	-0.125	5.83

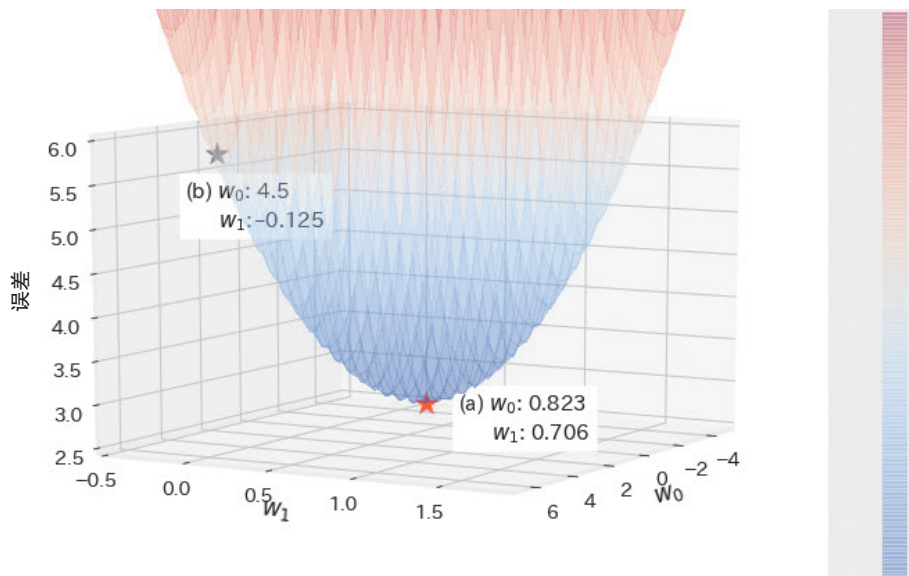
从表 2-3 可以看出，由于学习参数的变化，作为误差函数的均方误差也会发生变化。也就是说，均方误差可以使用学习参数的函数表示：

$$L(w_0, w_1) = \frac{\sum_{i=1}^n [y_i - (w_0 + w_1 x_i)]^2}{n}$$

这时将表 2-2 的数据代入目标变量 y_i 和特征变量 x_i ，就可以仅用 w_0 、 w_1 表示均方误差：

$$L(w_0, w_1) = \frac{\sum_{i=1}^4 [y_i - (w_0 + w_1 x_i)]^2}{4} = w_0^2 + 24.5w_1^2 + 9w_0w_1 - 8w_0 - 42w_1 + 21$$

该式是 w_0 、 w_1 的二次函数，函数图形如图 2-4 所示。从图 2-4 可以看出，当 w_0 、 w_1 发生变化时，误差的值也各不相同。另外，图 2-4 上标记星号的点对应于表 2-3 中的两种情况。表 2-3 中的 (a) 的学习参数与误差函数取最小值时的学习参数一致，可以看出这是数据点的最佳学习参数。



▲图 2-4 均方误差的最小化

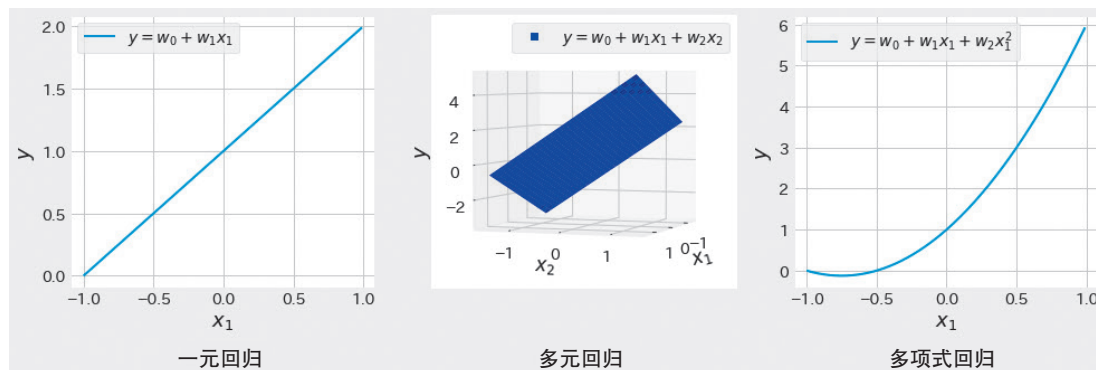
各种线性回归和非线性回归

本节主要说明了一元回归。一元回归是指独立特征变量只有一个时的线性回归，独立特征变量为两个及以上时的线性回归叫作多元回归。另外，尽管独立特征变量只有一个，但包含 x^2 、 x^3 这种特征变量的次方项的线性回归叫作多项式回归。表 2-4 列出了一元回归、多元回归、多项式回归的例子，图 2-5 展示了与这些例子相应的图形。多项式回归对于特征变量 x_i 不是线性的，所以称之为“线性”回归可能让人觉得不太合适。

但是，是否为线性回归不是从特征变量来看的。从学习参数（在这个例子中是 x_1^2 和 x_1 的系数）的角度来看是线性的回归，我们才称为线性回归，所以多项式回归也属于线性回归。

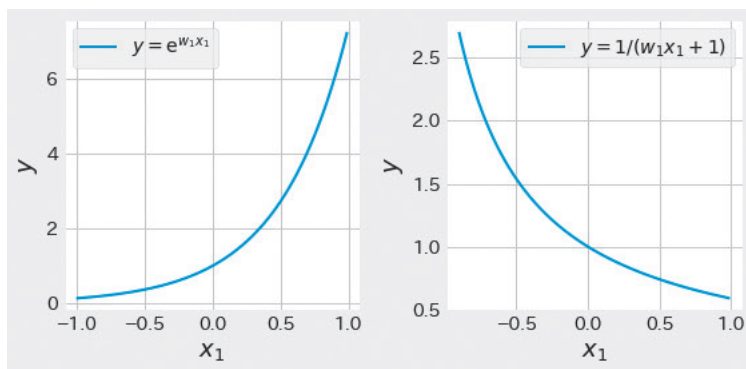
▼表 2-4 各种线性回归的例子

线性回归的种类	示 例
一元回归	$y = w_0 + w_1x_1$
多元回归	$y = w_0 + w_1x_1 + w_2x_2$
多项式回归	$y = w_0 + w_1x_1 + w_2x_1^2$



▲图 2-5 各种线性回归的例子

下面再看一下非线性回归的例子。图 2-6 是 $y = e^{w_1x_1}$ 和 $y = 1/(w_1x_1 + 1)$ 的图形。这些函数中的学习参数 w_1 和目标变量 y 之间的关系不是线性关系，所以被分类为非线性回归。



▲图 2-6 非线性回归的例子

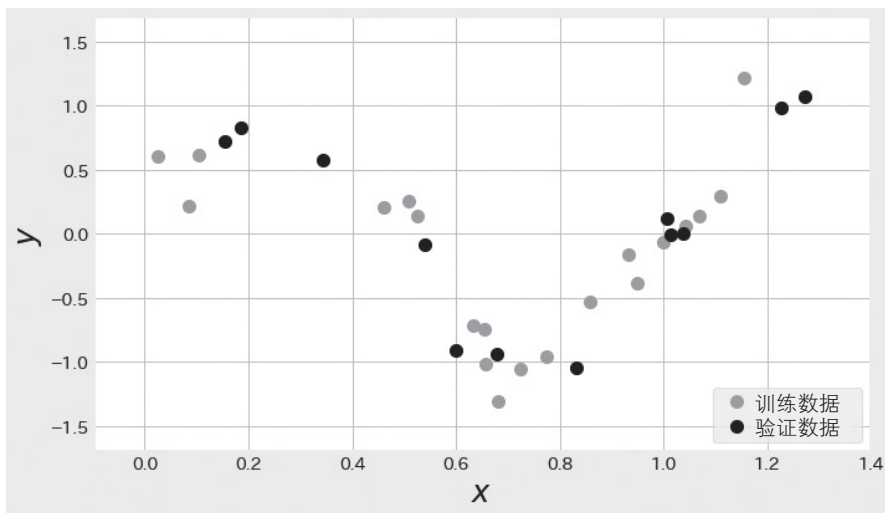
2.2 算法 2：正则化

正则化是防止过拟合的一种方法，与线性回归等算法配合使用。通过向损失函数增加惩罚项的方式对模型施加制约，有望提高模型的泛化能力。

概述

正则化是防止过拟合的方法，用于机器学习模型的训练阶段。过拟合是模型在验证数据上产生的误差比在训练数据上产生的误差（训练误差）大得多的现象。过拟合的一个原因是机器学习模型过于复杂。正则化可以降低模型的复杂度，有助于提高模型的泛化能力。

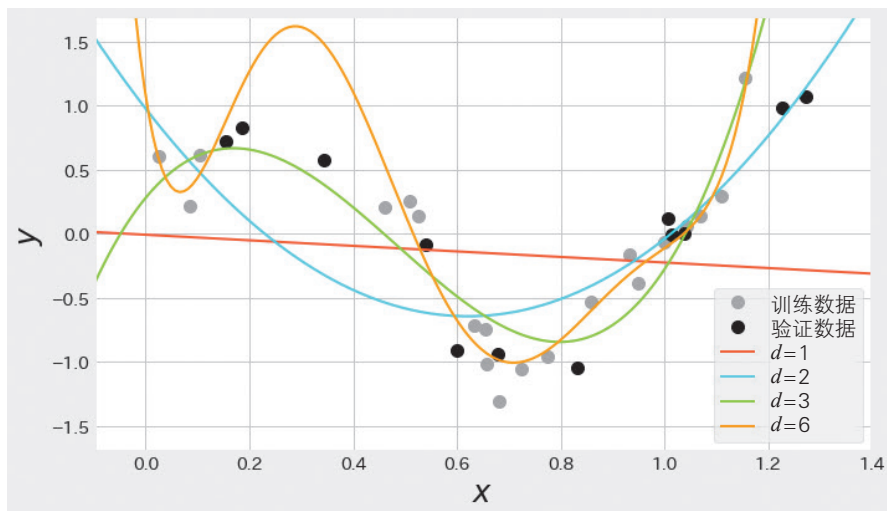
在详细了解正则化的方法之前，我们先看一下使用了正则化的模型如何防止过拟合。这里使用的数据是图 2-7 中的训练数据（灰色的数据点）和验证数据（黑色的数据点）。这些数据点是对函数 $y=\sin(2\pi x)$ 添加了遵循高斯分布的随机数而生成的。



▲图 2-7 使用 $y=\sin(2\pi x)$ 加随机数生成的数据

下面尝试用线性回归对这份数据建模。不断在线性回归中加入一次项、二次项……观察随着多项式的次数越来越大，训练误差和验证误差会如何变化。

不同次数 d 的训练结果如图 2-8 所示。此时，使用均方误差计算使误差最小的学习参数 w_i 。一次线性回归函数是 $y = w_0 + w_1x$ ，因此图形是直线；二次线性回归函数是 $y = w_0 + w_1x + w_2x^2$ ，因此图形是二次曲线；六次线性回归函数是 $y = w_0 + w_1x + w_2x^2 + \cdots + w_6x^6$ ，它的图形看上去是一个非常复杂的曲线。



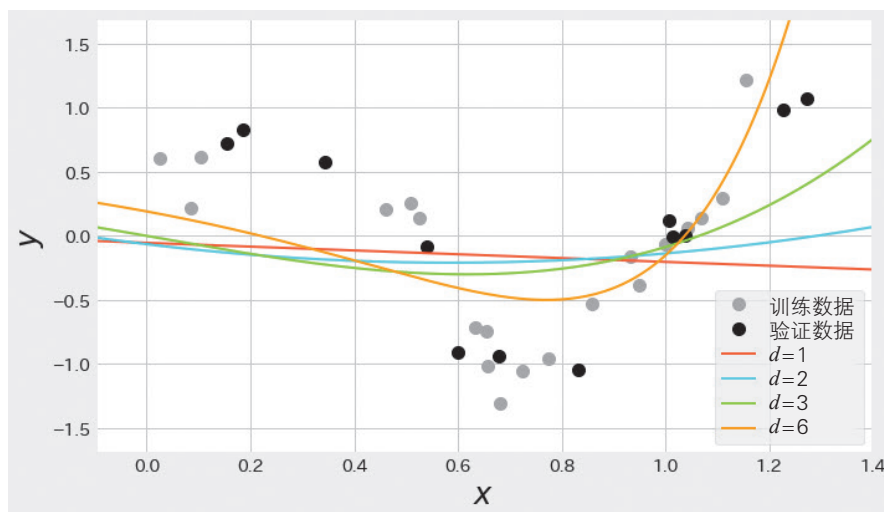
▲图 2-8 不同次数的线性回归的训练结果（对于同样的数据，复杂的模型会发生过拟合）

不同次数的训练误差和验证误差如表 2-5 所示。我们可以看出随着函数次数的增加，训练误差渐渐变小了。如果只看训练误差，那么六次线性回归的误差 0.024 是最小的，但此时的验证误差是 3.472，比训练误差大了很多。六次线性回归是一个复杂的模型，虽然它减小了训练误差，但是由于过拟合，它的泛化能力很低。

▼表 2-5 次数与训练误差、验证误差的关系

次 数	训练误差	验证误差
1	0.412	0.618
2	0.176	0.193
3	0.081	0.492
⋮	⋮	⋮
6	0.024	3.472

接下来看一下对线性回归应用正则化后的结果，如图 2-9 和表 2-6 所示。正则化可以通过向损失函数增加惩罚项的方式防止过拟合。从图 2-9 可以看出，正则化抑制了模型的复杂度，次数增加后验证误差也被抑制，从而防止了过拟合的出现。



▲图 2-9 对不同次数的线性回归应用正则化后的训练结果（模型的复杂度得到了抑制）

▼表 2-6 应用正则化后的次数与训练误差、验证误差的关系

次 数	训练误差	验证误差
1	0.412	0.612
2	0.372	0.532
3	0.301	0.394
⋮	⋮	⋮
6	0.159	0.331

现在有许多正则化的方法。本书前面使用的回归模型是被称为岭回归（ridge regression）的具有代表性的回归方法。下面的“算法说明”部分将介绍如何防止岭回归出现过拟合，从而提高它的泛化能力。

算法说明

在“概述”部分，我们了解了复杂模型的过拟合以及使用正则化防止过拟合的知识。复杂模型过拟合的一个原因是学习参数 w_i 的值太大（或太小）。表 2-7 列出了不同次数的线性回归的学习参数。随着次数的增加，学习参数的绝对值变大。

表 2-8 列出了使用正则化后的学习参数。使用正则化能够抑制学习参数随着次数增加而变大。

▼表 2-7 不同次数的学习参数

次 数	w_0	w_1	w_2	w_3	w_4	w_5	w_6
1	-0.007	-0.217	-	-	-	-	-
2	0.978	-5.222	4.204	-	-	-	-
3	0.281	4.927	-17.639	12.157	-	-	-
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
6	1.080	-26.324	287.431	-1034.141	1611.144	-1147.946	308.643

▼表 2-8 应用正则化后的不同次数的学习参数

次 数	w_0	w_1	w_2	w_3	w_4	w_5	w_6
1	-0.055	-0.149	-	-	-	-	-
2	-0.066	-0.493	0.421	-	-	-	-
3	-0.001	-0.716	-0.042	0.670	-	-	-
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
6	0.191	-0.751	-0.497	-0.182	0.109	0.370	0.607

为什么正则化可以抑制学习参数变大呢？这里以下面的岭回归的误差函数为例进行说明。简单起见，这里考虑对二次线性回归应用正则化的情况：

$$R(\mathbf{w}) = \sum_{i=1}^n \left[y_i - (w_0 + w_1 x_i + w_2 x_i^2) \right]^2 + \alpha (w_1^2 + w_2^2)$$

等号右边的第 1 项 $\sum_{i=1}^n \left[y_i - (w_0 + w_1 x_i + w_2 x_i^2) \right]^2$ 是线性回归的损失函数。第 2 项 $\alpha (w_1^2 + w_2^2)$ 被称为惩罚项（或者正则化项），是学习参数的平方和的形式。一般来说，惩罚项中不包含截距。

另外， α ($\alpha \geq 0$) 是控制正则化强度的参数， α 越大，对学习参数的抑制越强； α 越小，对训练数据过拟合的可能性越大。

下面思考岭回归的损失函数 $R(\mathbf{w})$ 的最小化。

等号右边的第 1 项 $\sum_{i=1}^n [y_i - (w_0 + w_1 x_i + w_2 x_i^2)]^2$ 其实是求使得与训练数据 $\mathbf{y}$ 之间的误差变小的任意 w_0 、 w_1 、 w_2 的问题，右边第 2 项（即惩罚项）是学习参数的平方和，因此学习参数的绝对值越大，损失函数整体的值就越大。由此可知，惩罚项具有“对绝对值大的学习参数给予损失变大的惩罚”的作用，这个作用可以抑制学习参数变大。

■ 示例代码

下面是对 $\sin$ 函数进行岭回归建模时的示例代码。代码中使用 `PolynomialFeatures` 方法创建了六次多项式。

▼ 示例代码

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error

train_size = 20
test_size = 12
train_X = np.random.uniform(low=0, high=1.2, size=train_size)
test_X = np.random.uniform(low=0.1, high=1.3, size=test_size)
train_y = np.sin(train_X * 2 * np.pi) + np.random.normal(0, 0.2, train_size)
test_y = np.sin(test_X * 2 * np.pi) + np.random.normal(0, 0.2, test_size)
poly = PolynomialFeatures(6) # 次数为6
train_poly_X = poly.fit_transform(train_X.reshape(train_size, 1))
test_poly_X = poly.fit_transform(test_X.reshape(test_size, 1))
model = Ridge(alpha=1.0)
model.fit(train_poly_X, train_y)
train_pred_y = model.predict(train_poly_X)
test_pred_y = model.predict(test_poly_X)
print(mean_squared_error(train_pred_y, train_y))
print(mean_squared_error(test_pred_y, test_y))
```

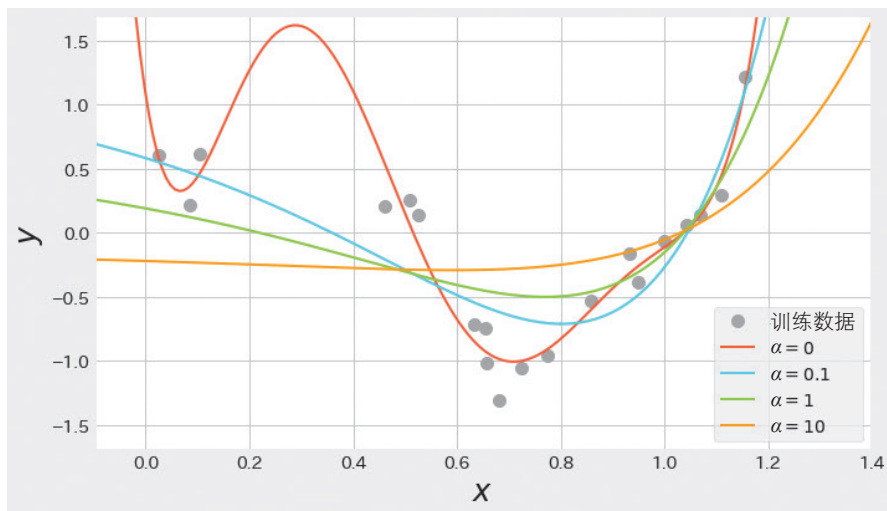
```
0.15917213330897523
```

```
0.3313327101623571
```

详细说

通过 α 控制正则化强度

下面来详细看一下控制正则化强度的超参数 α 。图 2-10 是使用不同的 α 值时模型的可视化图形。当 α 增大时，可以看出学习参数被抑制，图形变得简单。相反，当 α 变小时，对学习参数的绝对值变大的惩罚力度变缓，模型变复杂。另外，当 $\alpha=0$ 时，惩罚项始终为 0，因此等同于不使用正则化的线性回归。一般来说，应一边验证误差一边对 α 进行调整，最终得到合适的 α 。



▲图 2-10 α 变化时的岭回归

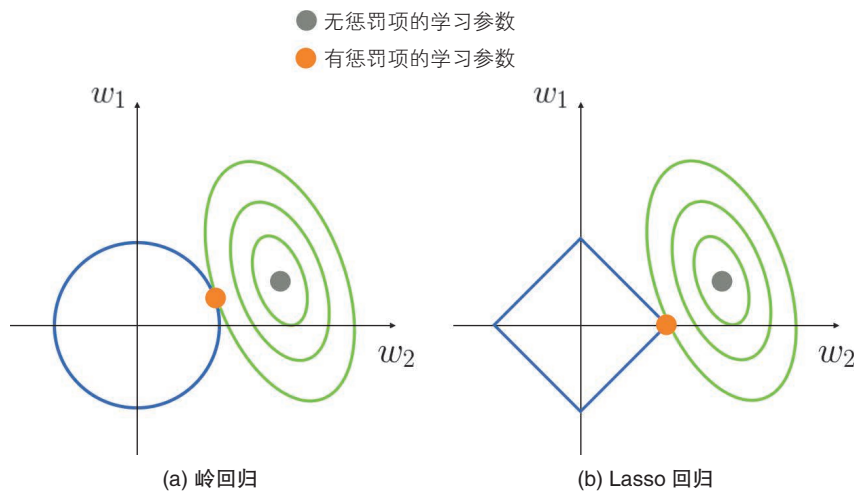
岭回归和 Lasso 回归

前面介绍了作为正则化方法的岭回归。岭回归的误差函数的惩罚项是学习参数的平方和的形式，通过将该惩罚项改为其他形式，可以实现不同特点的正则化。除了岭回归以外，还有一种具有

代表性的正则化方法——Lasso 回归。Lasso 回归的误差函数如下：

$$R(\mathbf{w}) = \sum_{i=1}^n [y_i - (w_0 + w_1 x_i + w_2 x_i^2)] + \alpha(|w_1| + |w_2|)$$

Lasso 回归的惩罚项是学习参数的绝对值之和，这一点与岭回归不同。岭回归和 Lasso 回归计算学习参数时的情况分别如图 2-11a 和图 2-11b 所示。



▲ 图 2-11 计算学习参数时的示意图

绿线是线性回归的误差函数，蓝线是惩罚项的相关函数。岭回归的惩罚项是学习参数的平方和，所以其图形是图 2-11a 所示的圆形；Lasso 回归的惩罚项是绝对值的和，所以其图形是图 2-11b 所示的四边形。原来的函数（线性回归的误差函数）与这些函数的交点就是带有正则化项的误差函数的最佳解。可以看出，在加入惩罚项后，图 2-11a 的岭回归的学习参数得到了抑制。图 2-11b 的 Lasso 回归的情况与岭回归相似，学习参数同样被抑制，但学习参数 w_2 变为了 0。

Lasso 回归计算的是函数与这种四边形函数的交点，因此具有学习参数容易变为 0 的特点。利用这个特点，我们可以使用学习参数不为 0 的特征来构建模型，从而达到利用 Lasso 回归选择特征的效果。这样不仅能提高模型的泛化能力，还能使模型的解释变容易。

参考文献

Christopher M. Bishop. Pattern Recognition and Machine Learning [M]. Berlin: Springer, 2007.
scikit-learn. 1.1 Generalized Linear Model [EB/OL]. [2019-2-21].

2.3 算法 3：逻辑回归

逻辑回归是一种用于有监督学习的分类任务的简单算法。虽然算法的名字中包含“回归”二字，但其实它是用于分类问题的算法。逻辑回归通过计算数据属于各类别的概率来进行分类。

概述

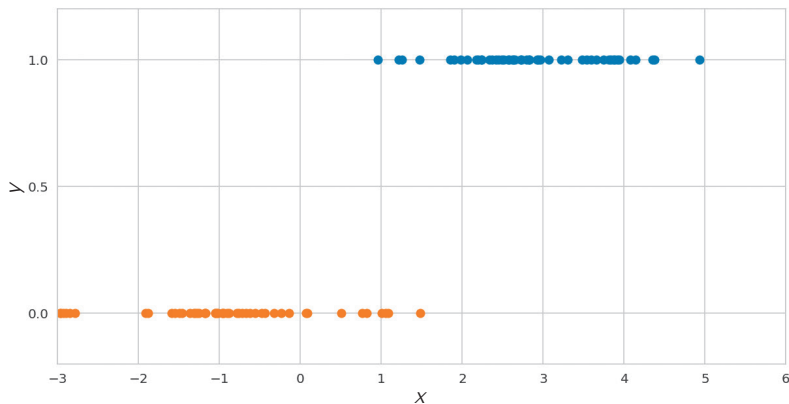
逻辑回归是一种学习某个事件发生概率的算法。利用这个概率，可以对某个事件发生或不发生进行二元分类。

虽然逻辑回归本来是二元分类的算法，但也可以用于三种类别以上的分类问题。

为了理解这个算法，请思考以下例子。

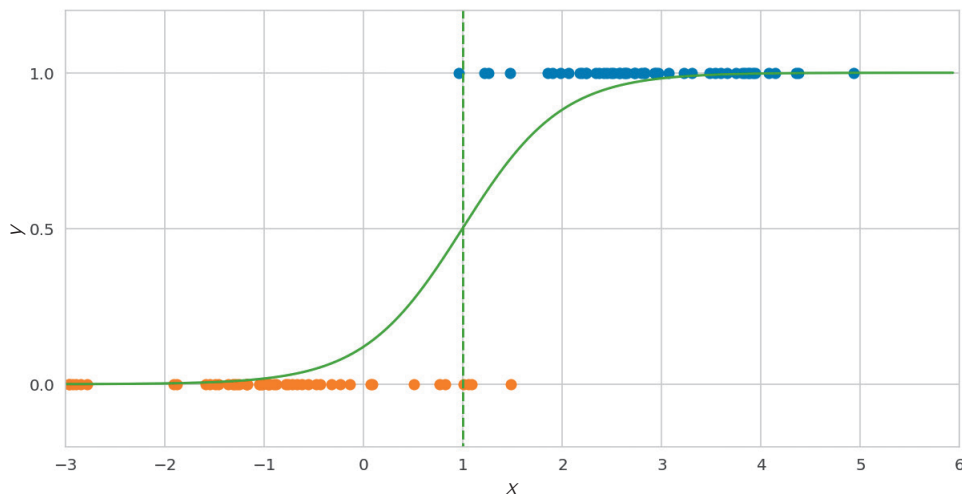
你在回家的路上发现下雪了。如果明天积雪，你就必须从鞋柜里翻出雪天穿的靴子做好准备。现在的气温是 2°C 。请问明天不积雪，你可以穿平时的鞋子出门的概率是多少呢？

这里使用虚构的 100 天的数据，通过逻辑回归求出可以穿平时的鞋子出门的概率。以气温为横轴 x ，当出现积雪而需要穿着雪天穿的靴子出门时，纵轴 y 的值为 0；当雪已经融化、可以穿着平时的鞋子外出时，纵轴 y 的值为 1，绘制得到的数据散点图如图 2-12 所示。



▲图 2-12 使用 100 天的数据绘制的散点图

在这里使用逻辑回归，我们就可以得到如图 2-13 中的绿线所示的曲线，并利用该曲线求出在不同的气温下，雪已经融化、可以穿着平时的鞋子外出的概率。



▲图 2-13 逻辑回归

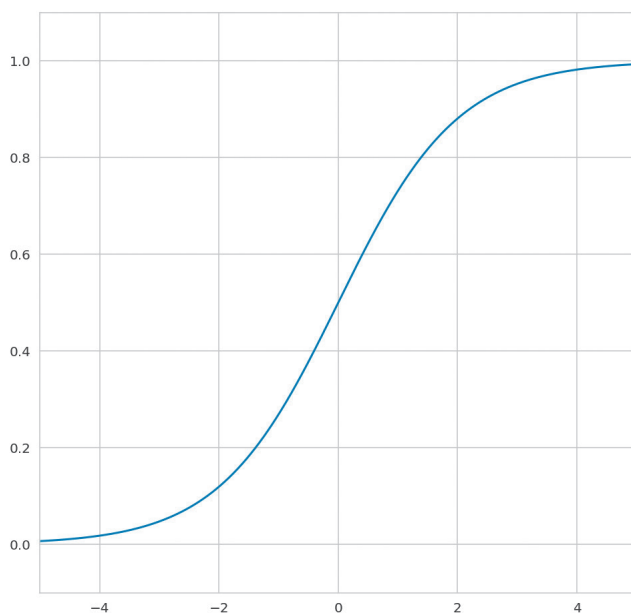
具体来说，计算得到的概率分别是：气温为 0 °C 时的概率为 12%，为 1 °C 时的概率为 50%，为 2 °C 时的概率为 88%。看起来明天可以穿平时的鞋子外出。

▶ 算法说明

如前所述，逻辑回归根据数据 $\mathbf{x}$ 和表示其所属类别的标签 y 进行学习，计算概率。数据 $\mathbf{x}$ 可以当作由特征值组成的向量处理。如果标签是二元分类，则可以使用前面的 $y=0, 1$ 这种二元数值表示。

逻辑回归的基本思想与线性回归一样，对数据 $\mathbf{x}$ 乘以权重向量 $\mathbf{w}$ ，再加上偏置 w_0 ，计算 $\mathbf{w}^T \mathbf{x} + w_0$ 的值。逻辑回归和线性回归在从数据中学习权重 $\mathbf{w}$ 和偏置 w_0 这一点上是相同的。

与线性回归不同的是，为了计算概率，逻辑回归的输出范围必须限制在 0 和 1 之间。逻辑回归使用 Sigmoid 函数 $\sigma(z) = 1/[1 + \exp(-z)]$ ，返回 0 和 1 之间的数值。Sigmoid 函数的图形如图 2-14 所示。



▲图 2-14 Sigmoid 函数

我们对输入数据 $\mathbf{x}$ 使用 Sigmoid 函数 $\sigma(z)$ ，即使用 $p = \sigma(\mathbf{w}^T \mathbf{x} + w_0)$ 计算标签为 y 的概率 p 。二元分类通常将预测概率 0.5 作为阈值进行分类。例如，当概率小于 0.5 时，将 y 的预测值分类为 0；当概率大于 0.5 时，将 y 的预测值分类为 1。根据问题的不同，有时会将阈值设置为大于或小于 0.5 的值。

在学习过程中，我们使用逻辑损失作为误差函数进行最小化。与其他误差函数一样，逻辑损失是在分类失败时返回大值，在分类成功时返回小值的函数。与在误差回归中引入的均方误差不同的是，我们无法通过式子变形来计算逻辑损失的最小值，因此需要采用梯度下降法通过数值计算来求解。

对于无法通过式子变形严密求解的情况，机器学习中经常会通过数值计算来近似求解。

■ 示例代码

下面使用与前面的“概述”部分相同的数据，计算在 0℃、1℃、2℃时积雪的概率。由于数据是使用随机数生成的，所以每次执行时结果都有所不同。

▼ 示例代码

```
import numpy as np
from sklearn.linear_model import LogisticRegression
X_train = np.r_[np.random.normal(3, 1, size=50),
                 np.random.normal(-1, 1, size=50)].reshape((100, -1))
y_train = np.r_[np.ones(50), np.zeros(50)]
model = LogisticRegression()
model.fit(X_train, y_train)
model.predict_proba([[0], [1], [2]])[:, 1]
```

```
array([ 0.12082515,  0.50296844,  0.88167486])
```

计算可知，在 0 °C、1 °C、2 °C 时概率的值（保留小数点后两位）分别约为 0.12、0.50、0.88。

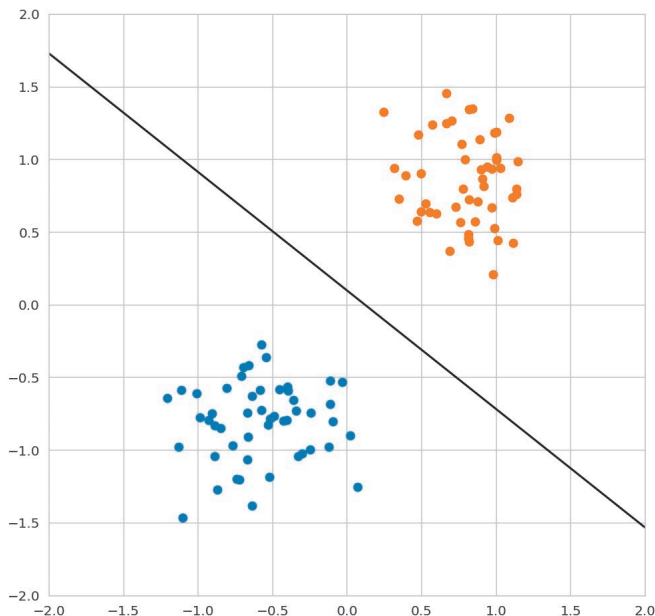
详细说明

决策边界

在解决分类问题时，如果让学习后的模型对未知数据分类，模型就会以某个地方为边界来区分分类结果，这个边界就叫作**决策边界**。逻辑回归的决策边界是计算出的概率正好为 50% 的地方。

下面看一下在平面的情况下的决策边界的图形。使用逻辑回归学习时的训练数据和决策边界如图 2-15 所示。

决策边界的形状因使用的算法不同而有很大的不同。在平面的情况下，逻辑回归的决策边界是直线。在其他算法中，比如 KNN 和



▲ 图 2-15 决策边界的图形

神经网络，决策边界是更复杂的形式。详细内容将在介绍相应算法时进行说明。

对特征的解释

我们可以查看逻辑回归中每个特征的系数。通过查看每个特征的系数的符号，可以知道它对概率是正影响还是负影响。这里以第1章的鸢尾花数据为例，只使用其中的 `setosa`（山鸢尾）和 `versicolor`（杂色鸢尾）这两种鸢尾花的数据来确认这一点。表 2-9 所示为逻辑回归学习得到的每个特征的权重的结果。对应于 `versicolor` 和 `setosa` 的目标变量分别为 1 和 0。

表 2-9 每个特征的权重

sepal length	sepal width	petal length	petal width
-0.407 317 45	-1.460 923 71	2.240 047 24	1.008 414 92

这些值表示在特征值发生变化时，特征值对模型将数据标签分类为 `versicolor` 的概率的影响程度。下面使用 `sepal width` 和 `petal length` 两个特征绘制散点图，来查看训练数据的分布。绘制结果如图 2-16 所示。

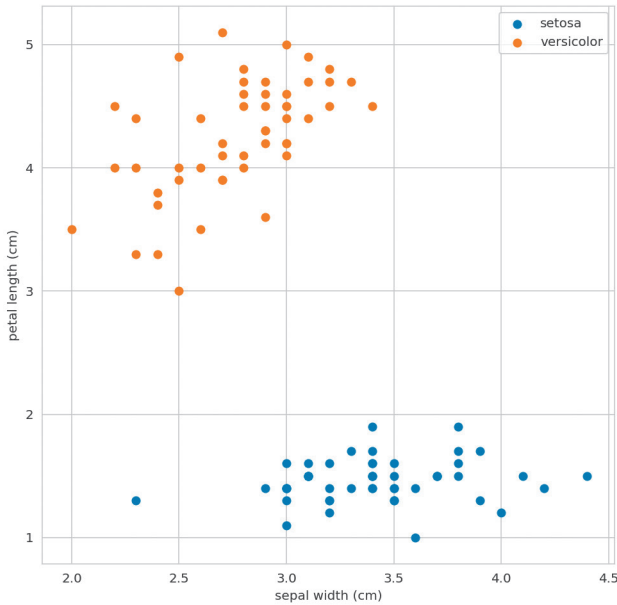


图 2-16 对特征的解释

一方面，由于纵轴的 petal length 的权重是正值，所以结合权重来看，如果 petal length 变大，数据被分类为 versicolor 的比例会变大。总的来看，在图 2-16 中的上方的数据被分类为 versicolor 的比例大，而下方的数据被分类为 setosa 的比例大。

另一方面，由于 sepal width 的权重是负值，所以如果 sepal width 变小，数据被分类为 versicolor 的比例会变大。我们可以得出如下结论：在图 2-16 中偏左的地方分布着 versicolor，在偏右的地方分布着 setosa。

2.4 算法 4：支持向量机

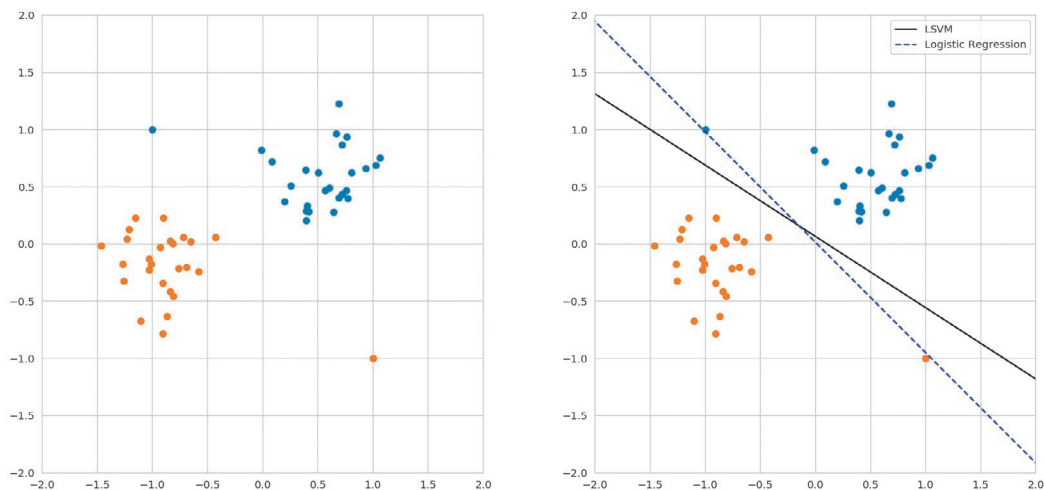
支持向量机（Support Vector Machine, SVM）是一种应用范围非常广泛的算法，既可以用于分类，也可以用于回归。

本节将介绍如何将线性支持向量机应用于二元分类问题，以间隔（margin）最大化为基准，得到更好的决策边界。

概述

本节使用线性支持向量机（Linear Support Vector Machine, LSVM）处理二元分类。线性支持向量机是以间隔最大化为基准，来学习得到尽可能地远离数据的决策边界的算法。虽然该算法的决策边界与逻辑回归一样是线性的，但有时线性支持向量机得到的结果更好。

下面对同一数据分别应用线性支持向量机和逻辑回归，并比较其结果（图 2-17）。



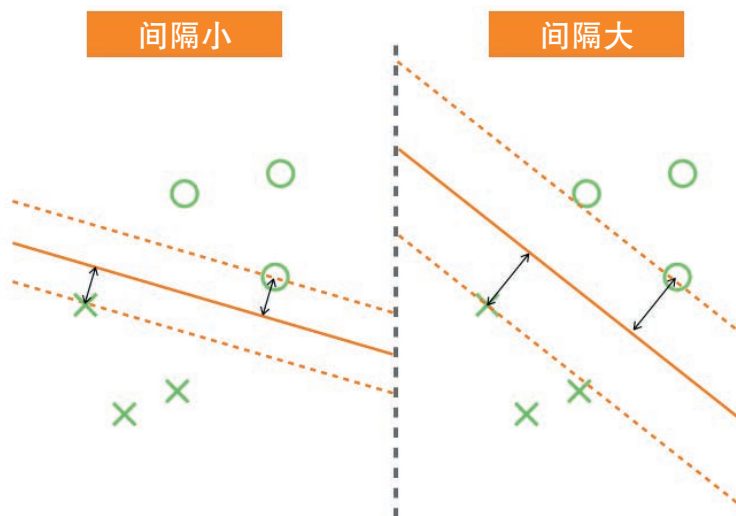
▲ 图 2-17 线性支持向量机和逻辑回归的比较

在图 2-17 中，左图是学习前的数据，右图是从数据中学习后的结果。右图中用黑色直线标记的是线性支持向量机的决策边界，用蓝色虚线标记的是逻辑回归的决策边界。

线性支持向量机也好，逻辑回归也好，都可以正确分类，但线性支持向量机的分类结果更佳。线性支持向量机的学习方式是：以间隔最大化为基准，让决策边界尽可能地远离数据。下面来看一下线性支持向量机是如何从数据中学习的。

▶ 算法说明

线性支持向量机通过最大化间隔来获得更好的用于分类的决策边界。这里首先说明一下间隔的定义。方便起见，我们以平面上的二元分类问题为例进行说明，并且假设数据可以完全分类。线性支持向量机通过线性的决策边界将平面一分为二，据此进行二元分类。此时，训练数据中最接近决策边界的数据与决策边界之间的距离就称为间隔（图 2-18）。



▲ 图 2-18 不同的间隔

在图 2-18 中，右图的间隔大于左图的间隔。支持向量机试图通过增大决策边界和训练数据之间的间隔来获得更合理的边界。

■ 示例代码

下面生成线性可分的数据，将其分割成训练数据和验证数据，使用训练数据训练线性支持向量机，使用验证数据评估正确率。另外，由于使用了随机数，所以每次运行的结果可能有所不同。

▼ 示例代码

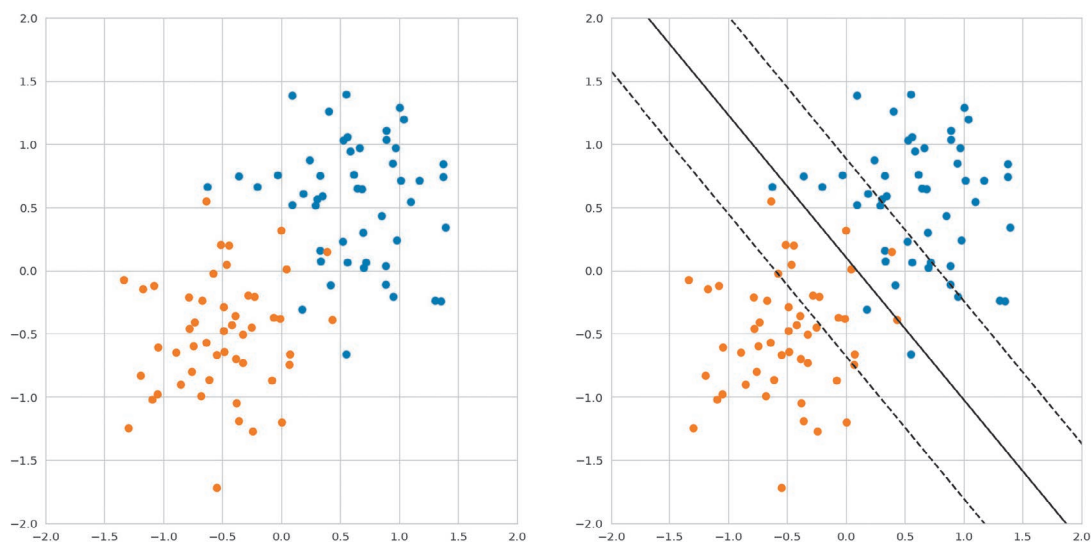
```
from sklearn.svm import LinearSVC
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
# 生成数据
centers = [(-1, -0.125), (0.5, 0.5)]
X, y = make_blobs(n_samples=50, n_features=2,
                  centers=centers, cluster_std=0.3)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
model = LinearSVC()
model.fit(X_train, y_train) # 训练
y_pred = model.predict(X_test)
accuracy_score(y_pred, y_test) # 评估
```

1.0

 详细说明

■ 软间隔和支持向量

到这里为止，我们了解的都是数据可以线性分离的情况，这种不允许数据进入间隔内侧的情况称为**硬间隔**。但一般来说，数据并不是完全可以线性分离的，所以要允许一部分数据进入间隔内侧，这种情况叫作**软间隔**。通过引入软间隔，无法线性分离的数据也可以如图 2-19 所示进行学习。



▲图 2-19 软间隔

基于线性支持向量机的学习结果，我们可以将训练数据分为以下 3 种。

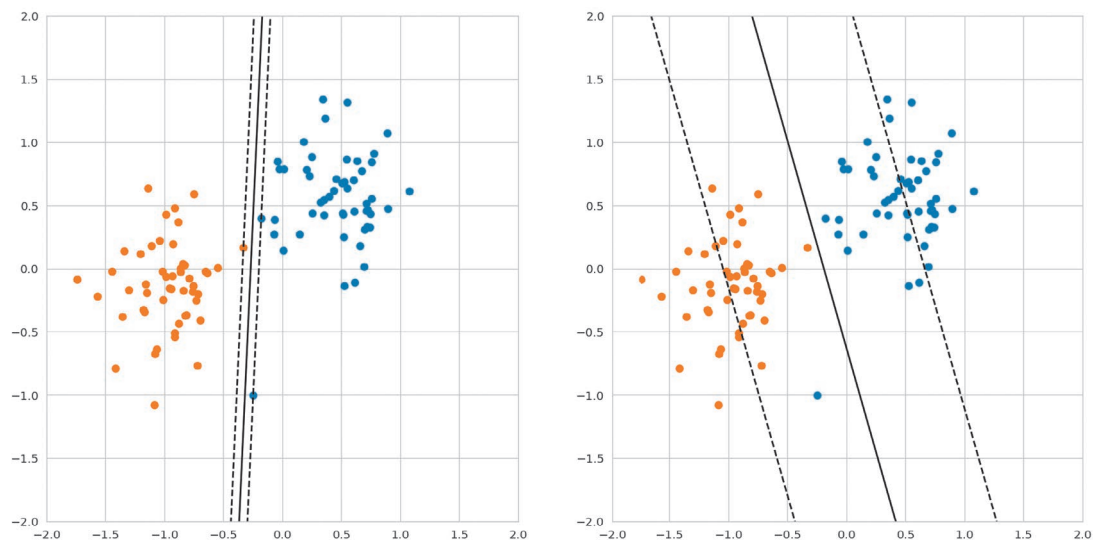
1. 与决策边界之间的距离比间隔还要远的数据：间隔外侧的数据。
2. 与决策边界之间的距离和间隔相同的数据：间隔上的数据。
3. 与决策边界之间的距离比间隔近，或者误分类的数据：间隔内侧的数据。

其中，我们将间隔上的数据和间隔内侧的数据特殊对待，称为**支持向量**。支持向量是确定决策边界的重要数据。间隔外侧的数据则不会影响决策边界的形状。

由于间隔内侧的数据包含被误分类的数据，所以乍看起来通过调整间隔，使间隔内侧不存在数据的做法更好。

但对于线性可分的数据，如果强制训练数据不进入间隔内侧，可能会导致学习结果对数据过拟合。

使用由两个标签组成的训练数据训练线性支持向量机而得到的结果如图 2-20 所示。



▲图 2-20 硬间隔和软间隔

图 2-20 的左图是不允许数据进入间隔内侧的硬间隔的情况，右图是允许数据进入间隔内侧的软间隔的情况。

另外，我们在蓝色点表示的训练数据中特意加上了偏离值。

比较两个结果可以发现，使用了硬间隔的图 2-20 中的左图上的决策边界受偏离值的影响很大；而在引入软间隔的图 2-20 中的右图上的学习结果不容易受到偏离值的影响。

在使用软间隔时，允许间隔内侧进入多少数据由超参数决定。与其他算法一样，在决定超参数时，需要使用网格搜索（grid search）和随机搜索（random search）等方法反复验证后再做决定。

2.5 算法 5: 支持向量机 (核方法)

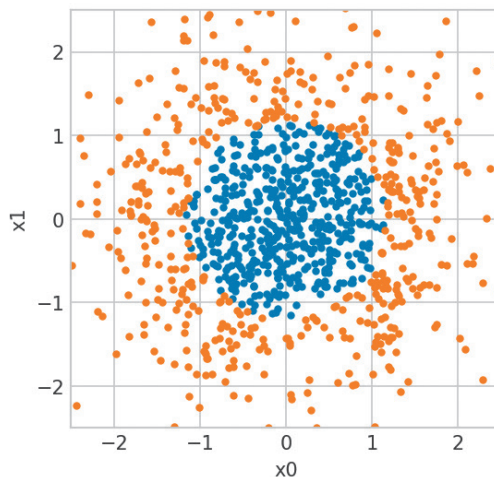
在深度学习出现之前,使用了核方法的支持向量机非常受欢迎。

通过在支持向量机中引入**核方法** (kernel methods) 这个技巧,那些无法人力标注特征值的复杂数据也能被处理。当然,这个算法现在也用于解决各种分类和回归问题。

概述

本节介绍通过在支持向量机中引入核方法,使模型可以学习复杂的决策边界的做法。这里以分类为例进行介绍,但是对于回归问题,同样可以使用核方法。

线性支持向量机通过最大化间隔,可以得到尽可能远离数据的“好的”决策边界。但是由于决策边界必定为直线,所以它很难对如图 2-21 所示的“每个标签的边界为曲线的数据”进行分类。



▲图 2-21 以某个点为中心分布的数据

为了分隔如图 2-21 所示的数据,模型需要学习曲线的线性决策边界。支持向量机可以使用核

方法学习复杂的决策边界。例如，对这次的数据使用核方法，模型可以学到如图 2-22 所示的圆形的决策边界。

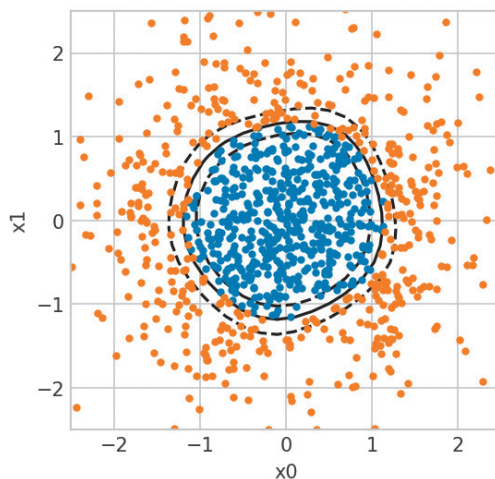
下面的“算法说明”部分将详细探讨核方法。

▶ 算法说明

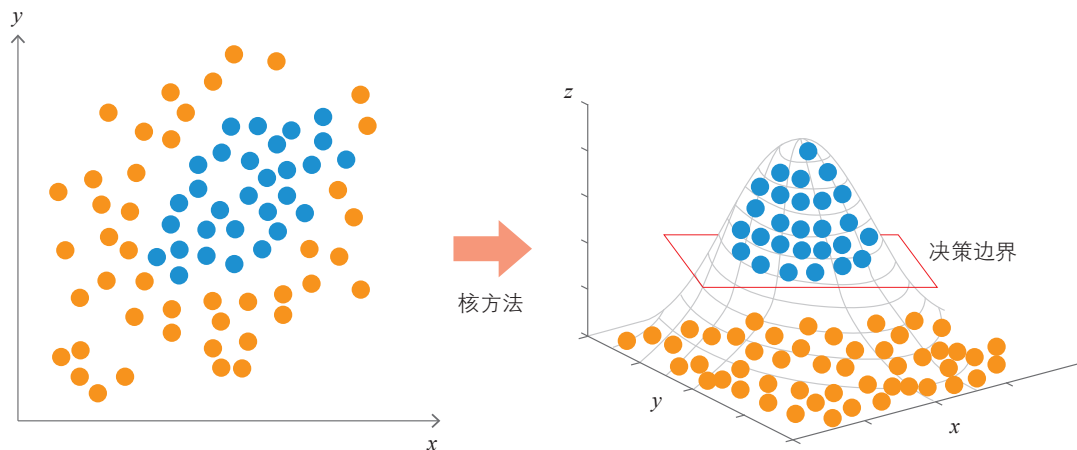
核方法的一个常见解释是“将数据移动到另一个特征空间，然后进行线性回归”。对于这个解释，我们再详细地看一下。

首先，思考一下如何使线性不可分数据成为线性可分数据。假设有一个比训练数据更高维的空间，训练数据中的每一个点都对应着这个高维空间中的一个点。在这个高维空间中，训练数据对应的点是可以线性分离的，实际的训练数据是来自该高维空间的投影。一旦有了这样的空间，模型就可以在高维空间中使用支持向量机来学习决策边界。最后，将高维空间的决策边界投影到由原始特征形成的向量空间上，得到决策边界。

图 2-23 所示为在高维空间中对数据进行线性分离的情形，该数据原本是线性不可分的。



▲图 2-22 支持向量机（核方法）



▲图 2-23 使原本线性不可分的数据变为线性可分

虽然构建线性分离的高维空间非常困难, 但通过一个叫作**核函数**的函数, 核方法就可以使用在高维空间中学习到的决策边界, 而无须构建具体的线性分离的高维空间。

“详细说明”部分将介绍具有代表性的核函数。

■ 示例代码

下面通过示例代码来看一下使用核方法的支持向量机如何学习呈圆形分布的数据的决策边界。我们生成呈圆形分布的数据, 将其拆分为训练数据和验证数据, 使用训练数据训练模型, 使用验证数据评估正确率。代码中没有明确指定使用哪个核方法, 这是因为代码默认使用 RBF (Radial Basis Function, 径向基函数) 核方法。

▼ 示例代码

```
from sklearn.svm import SVC
from sklearn.datasets import make_gaussian_quantiles
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

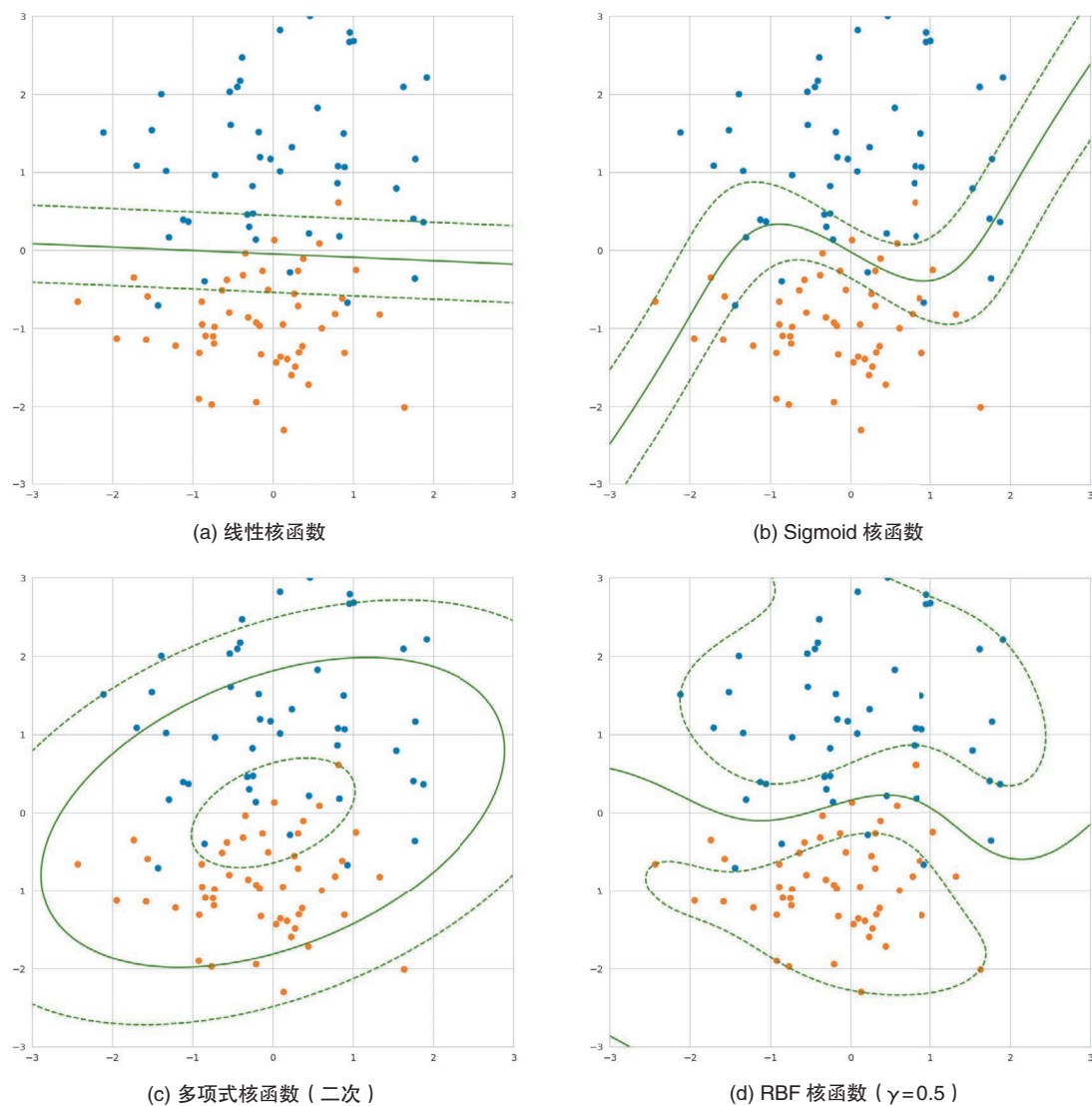
# 生成数据
X, y = make_gaussian_quantiles(n_features=2, n_classes=2, n_samples=300)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
model = SVC()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy_score(y_pred, y_test)
```

0.9777777777777775

► 详细说明

■ 学习结果因核函数而异

核方法中可以使用的核函数多种多样。使用不同的核函数, 得到的决策边界的形状也不同。图 2-24 所示为使用具有代表性的核函数在相同的数据上训练支持向量机的结果, 这些核函数分别是线性核函数、Sigmoid 核函数、多项式核函数和 RBF 核函数。

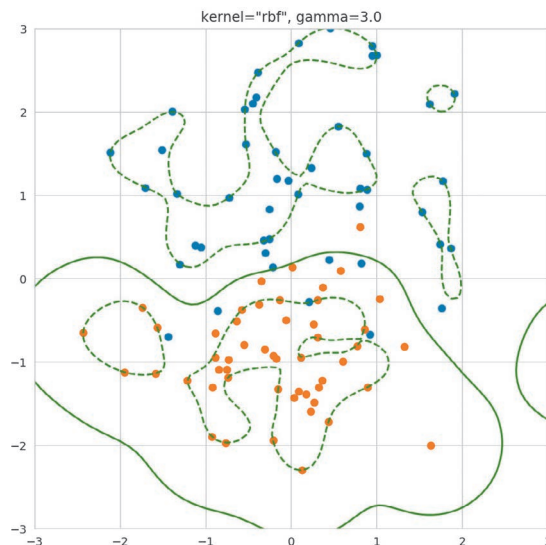


▲ 图 2-24 各种核函数

图 2-24a 中的线性核函数等价于线性支持向量机。图 2-24b 中使用 Sigmoid 函数作为核函数。从图中可以看出，通过改变核函数，模型学习到了曲线的决策边界。另外，图 2-24c 中的多项式核函数（二次）学习到了圆形的决策边界。从图 2-24d 中可以看到 RBF 核函数是如何学习更复杂的决策边界的。

注意

使用核方法后，我们就无法清楚地知道支持向量机使用的是哪些特征了。在将 RBF 作为核函数，并将 RBF 核函数的超参数 γ 设置为 3.0 时，模型学习的结果如图 2-25 所示。通过增加这个超参数的值，模型会从数据中学习更复杂的决策边界。



▲图 2-25 使用了 RBF 核函数的决策边界

图 2-25 中的模型学习的是一个复杂的决策边界，但我们已经不知道模型使用的是哪些特征了。核方法适用于“相比特征的可解释性更看重精度”的场景。还需要注意的是，得到的决策边界不一定是通过示例代码学习到的那种一目了然的决策边界。

由于这些特点，在使用支持向量机时，不宜立即使用非线性核函数，在此之前，应先使用线性核函数进行分析，以了解数据。

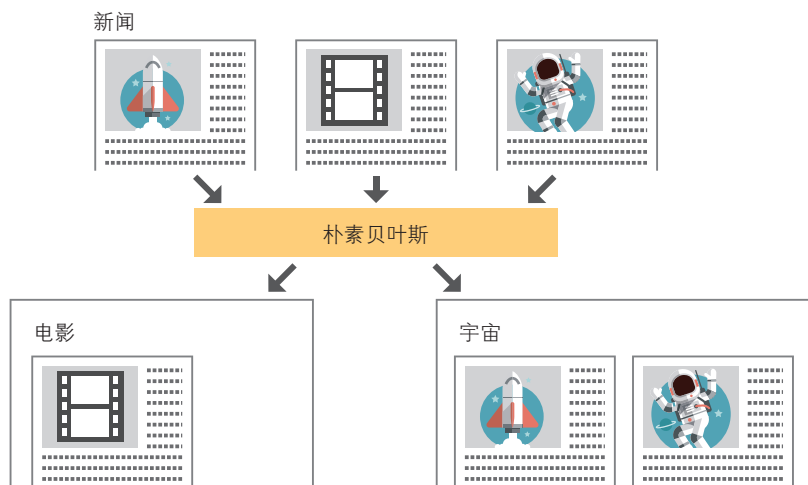
2.6 算法 6：朴素贝叶斯

朴素贝叶斯（Naive Bayes）是常用于自然语言分类问题的算法。它在垃圾邮件过滤上的应用非常有名。

概述

朴素贝叶斯是一个基于概率进行预测的算法，在实践中被用于分类问题。具体来说，就是计算数据为某个标签的概率，并将其分类为概率值最大的标签。朴素贝叶斯主要用于文本分类和垃圾邮件判定等自然语言处理中的分类问题。

下面使用朴素贝叶斯将虚构的新闻标题分类为“电影”和“宇宙”两个类别（图 2-26）。



▲ 图 2-26 类别的分类

使用的数据如表 2-10 和表 2-11 所示。

▼表 2-10 训练数据

训练数据	类 别
那部让人感动的电影名作重映	电影
华丽的动作电影首映	电影
复映的名作感动了世界	电影
沙尘暴笼罩着火星	宇宙
火星探测终于重新开始	宇宙
VR 中看到的火星沙尘暴让人感动	宇宙

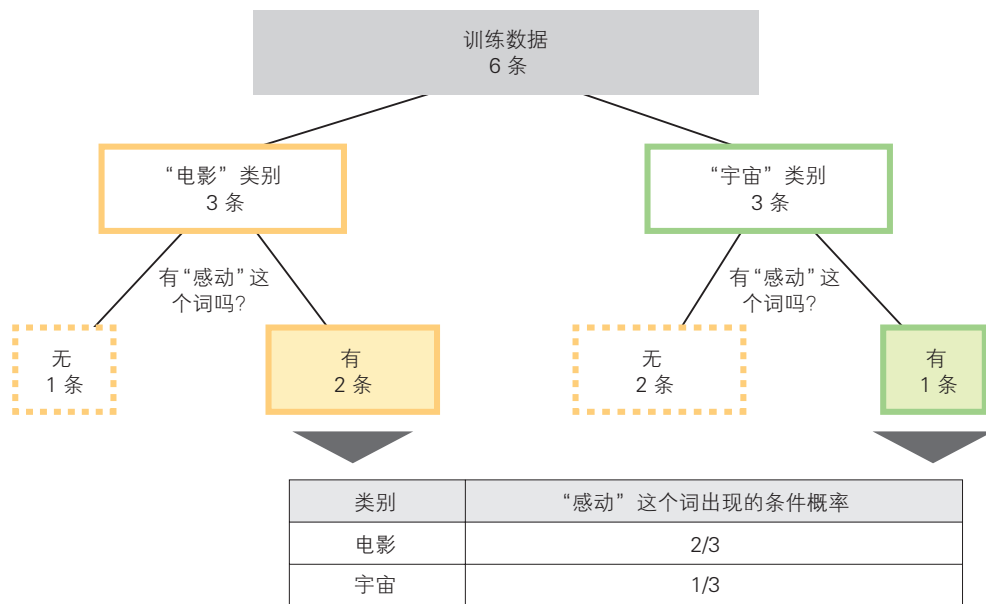
▼表 2-11 验证数据

验证数据	类 别
复映的动作电影名作让人感动	??

表 2-11 的验证数据是作为“电影”类别的新闻标题生成的，但这里我们当作不知道它的类别。

下面思考根据训练数据计算未知数据的类别是“电影”还是“宇宙”的概率的方法。训练数据中类别为“电影”的数据有 3 条，类别为“宇宙”的数据也有 3 条。简单地计算一下，可知未知数据的类别为“电影”的概率是 $3/6=50\%$ ，同样地，类别为“宇宙”的概率也是 50%。这个方法虽然很简单，但是如果采用这个计算方法，那么不管未知数据中的文本是什么，分类为各个类别的概率都是固定的。

朴素贝叶斯基于文本中包含的单词推测未知数据的类别。我们关注一下验证数据中包含的“感动”这个词。这个词在哪个类别里都会出现，但是出现的比例在每个类别里都不一样（图 2-27）。



▲图 2-27 单词出现的比例

在“电影”类别的数据中出现“感动”这个词的概率是 $2/3 \approx 67\%$ ，在“宇宙”类别的数据中则是 $1/3 \approx 33\%$ ，可见这个词更容易出现在“电影”类别的数据中。另外，在出现“感动”这个词的数据中，类别为“电影”的概率是 $2/3 \approx 67\%$ ，为“宇宙”的概率是 $1/3 \approx 33\%$ 。如果对使用验证数据中的“感动”这个词进行筛选后的数据计算比例，类别是“电影”的概率更高。此处求得的在某个条件（这次的条件是“感动”这个词出现）下的概率叫作**条件概率**。

朴素贝叶斯不仅使用了单词在文本中出现的比例，还使用了每个单词的条件概率，通过文本中单词的信息提高了计算精度。

算法说明

虽然最终要做的是自然语言处理的分类，但在应用朴素贝叶斯时，还需要将输入数据转换为由特征值构成的向量。我们先在预处理阶段将文本转换为由特征值构成的向量，然后使用朴素贝叶斯进行训练，最后查看结果。

预处理

在预处理阶段，我们将文本转换为 BoW（Bag of Words，词袋）的形式，形成由特征值构成的向量和标签的组合。

先从现有的训练数据的文本中只提取出名词，忽略名词在文本中的顺序，把它们作为集合处理^①。表 2-12 列出了只从训练数据中提取出名词的结果。

▼表 2-12 提取出训练数据的单词后形成的集合

训练数据	类 别
{ “感动”, “名作”, “电影” }	电影
{ “华丽”, “动作”, “电影” }	电影
{ “名作”, “世界”, “感动” }	电影
{ “沙尘暴”, “火星” }	宇宙
{ “火星”, “探测”, “重新开始” }	宇宙
{ “VR”, “火星”, “感动” }	宇宙

接下来，将训练数据和类别转换为易于处理的数据形式。当所有单词的集合包含训练数据的单词时，将该单词列的值设为 1，否则设为 0。另外，当类别为“电影”时，将类别的值替换为 1；当类别为“宇宙”时，将类别的值替换为 0，结果如表 2-13 所示。这些就是标签。

▼表 2-13 训练数据已成为特征值的向量

训练数据	名作	电影	华丽	动作	世界	感动	沙尘暴	火星	探测	重新开始	VR	类别
那部让人感动的电影名作重映	1	1	0	0	0	1	0	0	0	0	0	1
华丽的动作电影首映	0	1	1	1	0	0	0	0	0	0	0	1
复映的名作感动了世界	1	0	0	0	1	1	0	0	0	0	0	1
沙尘暴笼罩着火星	0	0	0	0	0	0	1	1	0	0	0	0
火星探测终于重新开始	0	0	0	0	0	0	0	1	1	1	0	0
VR 中看到的火星沙尘暴让人感动	0	0	0	0	0	1	1	1	0	0	1	0

通过以上处理，我们可以将以自然语言书写的文本转换为表示单词出现的特征值和标签的组

^① 为了从文本中提取出单词的集合，需要进行分词处理，这里假定这些处理已事先做完，正文中不进行介绍。

合，这种文本表示方式称为 BoW。下面采用同样的做法处理验证数据。验证数据中包含了训练数据中不包含的“复映”一词。这样的单词很难处理，简单起见，这次就不处理这个词了。文本处理后的结果如表 2-14 所示。

▼表 2-14 验证数据已成为特征值的向量

验证数据	名作	电影	华丽	动作	世界	感动	沙尘暴	火星	探测	重新开始	VR	类别
复映的动作电影名作让人感动	1	1	0	0	0	1	0	0	0	0	0	??

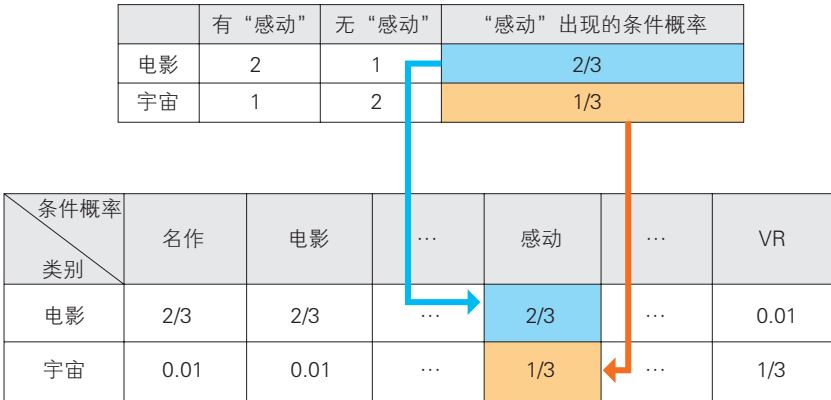
通过实施上述预处理，本次的问题就变为了预测验证数据 [1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0] 的类别的问题。

■ 概率的计算

朴素贝叶斯使用训练数据来学习与各标签对应的单词的出现概率。在分类时求出每个标签对应的概率，将概率最高的标签作为分类结果。朴素贝叶斯在训练时计算以下两种概率。

- 1. 每个标签出现的概率。
- 2. 在各标签下，每个单词出现的条件概率。

对朴素贝叶斯在训练时进行的处理进行汇总，如图 2-28 所示。注意，图中只展示了部分特征值的列。



▲图 2-28 训练时进行的处理

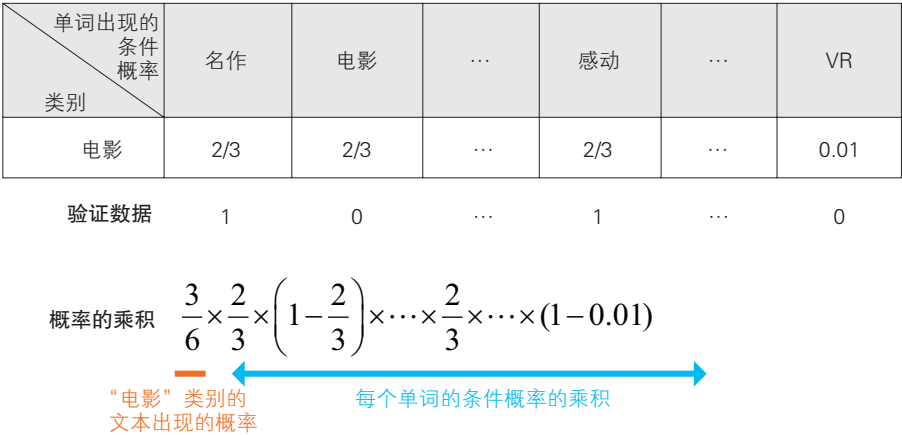
在图 2-8 中应该分配概率 0 的地方，我们分配了小的概率值 0.01，这种做法叫作平滑（smoothing）。概率为 0 的位置表示在对应的训练数据中没有出现对应的单词，但是如果使用更大的数据集作为训练数据，那个单词却真有可能出现。平滑处理考虑到了这种可能性，在没有出现单词的位置也分配小的概率值。

在分类时，朴素贝叶斯分别为每个标签值计算上述两个概率的乘积，通过比较进行分类。

例如，在对本次的验证数据 [1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0]（“复映的动作电影名作让人感动”）这个文本进行分类时，实际比较的是下列处理的结果。

- 1. 求出“电影”类别的文本出现的概率。
- 2. 从“电影”类别的文本中分别求出“名作”“动作”“感动”这几个单词出现的概率。
- 3. 对于文本中没有出现的单词，也要求出在“电影”类别的文本中没有出现的概率。
- 4. 计算所有概率的乘积。

上述处理如图 2-29 所示。按标签比较图 2-29 中的处理的结果，输出其中概率最大的标签作为分类结果。



▲图 2-29 处理的示意图

图 2-29 中分别计算了作为特征值的每个单词的概率，没有考虑单词的顺序和组合。朴素贝叶斯通过“每个单词都是独立的”这样一个简单的假设简化了学习过程。

示例代码

下面实际编写代码来查看计算结果。

▼ 示例代码

```
from sklearn.naive_bayes import MultinomialNB

# 生成数据
X_train = [[1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0],
            [0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0],
            [1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0],
            [0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1]]
y_train = [1, 1, 1, 0, 0, 0]

model = MultinomialNB()
model.fit(X_train, y_train) # 训练
model.predict([[1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0]]) # 评估
```

```
array([1])
```

这里得到的 `array([1])` 意味着类别是“电影”，说明这次的判断是正确的。

详细说明

注意

朴素贝叶斯是自然语言分类的基础算法，但是不适合预测天气预报中的降水概率那种预测值是概率本身的情况。前面的“算法说明”部分提到过，朴素贝叶斯基于“每个单词的概率可以独立计算”的假设，这是为了使概率的计算方法尽量简单。这个假设忽略了文本中单词之间的关联性。如

果要计算的概率值很重要，则应该避免直接将由朴素贝叶斯算出的值用作概率。

另外，朴素贝叶斯的“即使将各自的特征作为独立的变量来处理，也可以计算出概率”这一假设对于某些学习任务不成立。例如，如果单词在不同的上下文中含义不同，那么朴素贝叶斯的假设是不成立的。“踢”这个单词常常在格斗等运动的文本中出现，但是如果文本中出现了“踢了球”的用法，那么此时文本的主题很有可能是足球。这种单词含义在不同的上下文中有所变化的情况是不满足朴素贝叶斯“所有特征都是相互独立的”这一假设的。如果要考虑单词的上下文，就需要根据不同的上下文考虑使用其他模型。

2.7 算法 7：随机森林

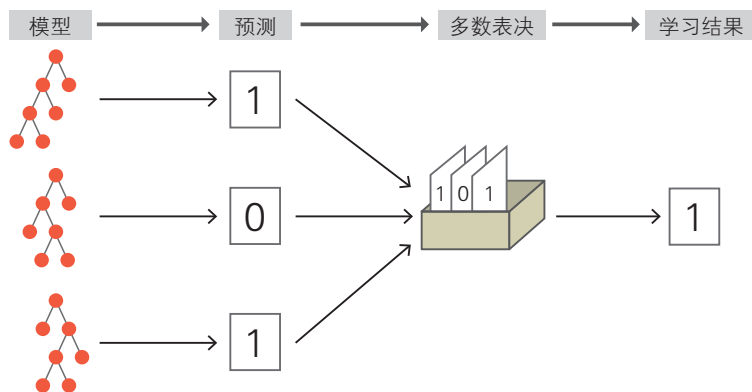
随机森林（random forest）是将多个模型综合起来创建更高性能模型的方法，既可用于回归，也可用于分类。同样的算法有梯度提升（gradient boosting）等在机器学习竞赛中很受欢迎的算法。

通过学习随机森林，我们可以学到在其他算法中也适用的基础知识。

概述

随机森林的目标是利用多个决策树模型，获得比单个决策树更高的预测精度。单个决策树的性能并不一定很高，但是多个决策树汇总起来，一定能创建出泛化能力更强的模型。

随机森林既可用于回归，也可用于分类，这里以分类问题为例进行说明。图 2-30 是随机森林分类算法的示意图。该算法从每个决策树收集输出，通过多数表决得到最终的分类结果。



▲ 图 2-30 随机森林的示意图

随机森林的多数表决就像找别人商量事情一样，不只听一个人的意见，而是在听取许多人的意见之后综合判断。机器学习也一样，通过创建多个模型，采取多数表决的方式，可以期待获得更为妥当的结果。

需要注意的是，如果使用同样的学习方法创建决策树，那么输出的就都是同样的东西，也就失去了采取多数表决的意义。在随机森林中，采用的决策树要具备多样性，这一点很重要。下面的“算法说明”部分将介绍决策树的算法，以及随机森林如何使决策树具有多样性。

算法说明

随机森林是综合决策树的结果的算法。本节先介绍决策树的基本内容，再介绍如何综合决策树的结果。

决策树

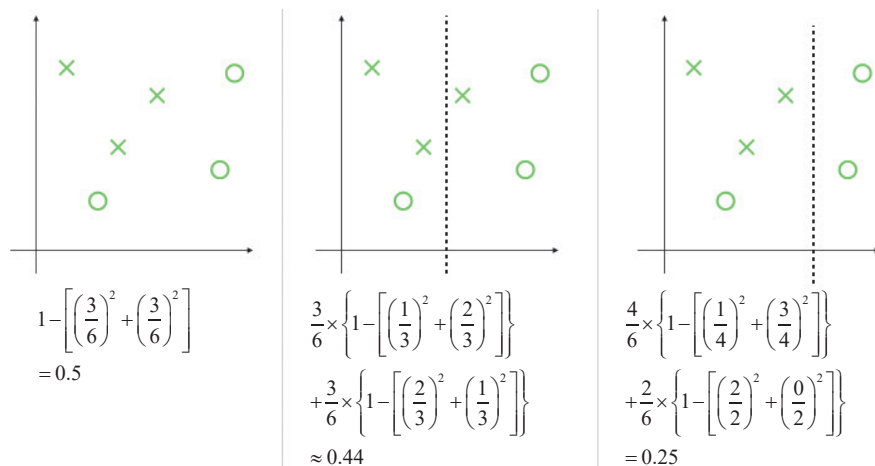
决策树是通过将训练数据按条件分支进行划分来解决分类问题的方法，在分割时利用了表示数据杂乱程度（或者不均衡程度）的不纯度的数值。决策树为了使表示数据杂乱程度的不纯度变小，对数据进行分割。当分割出来的组中存在很多相同的标签时，不纯度会变小；反之，当分割出来的组中存在很多不同的标签时，不纯度会变大。

表示不纯度的具体指标有很多种，本节利用基尼系数。基尼系数的计算式如下：

$$\text{基尼系数} = 1 - \sum_{i=1}^c p_i^2$$

c 是标签数， p_i 是某个标签的数据数量占数据总数的比例。

图 2-31 所示为使用几种不同的分割方法计算得出的加权平均基尼系数。左侧是分割前的状态，这时的基尼系数为 0.5。中间是分割后求出的基尼系数的平均值。使用分割出的每个部分的基尼系数乘以该部分所含数据数量所占的比例，得到加权平均值。右侧是使基尼系数的平均值最小的分割方法的例子。

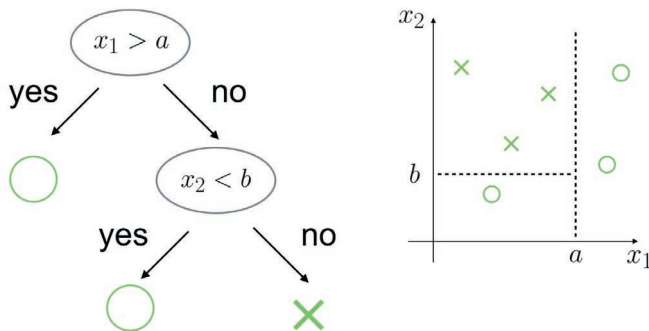


▲ 图 2-31 不同分割位置导致不纯度不同

决策树的学习是通过反复分割像图 2-31 那样的空间来进行的。具体来说，就是重复以下流程。

1. 计算某个区域的所有特征值和候选分割的不纯度。
2. 以分割时不纯度减小最多的分割方式分割区域。
3. 对于分割后的区域，重复步骤 1 和步骤 2。

决策树学习的过程如图 2-32 所示。



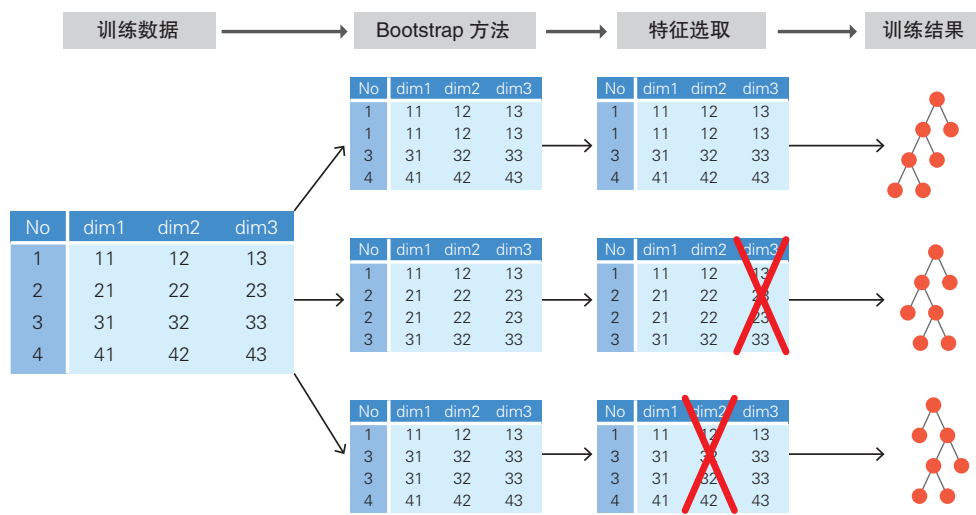
▲图 2-32 决策树学习的示意图

■ 随机森林

随机森林是如何使用多棵决策树来提高精度的呢？假设有 3 棵独立的决策树，每棵的正确率为 0.6。此时，对每棵决策树的结果进行多数表决，并将表决结果作为预测结果，即可提高正确率。在这种情况下，预测不正确有两种情况：一种是所有的决策树都预测不正确（概率为 $(1-0.6)^3 = 0.064$ ）；另一种是 3 棵决策树中有 2 棵预测不正确（概率为 $3 \times (1-0.6)^2 \times 0.6 = 0.288$ ），所以正确的概率是 $1 - 0.064 - 0.288 = 0.648$ 。可以看出，正确率的确变高了。

接下来，让我们思考如何使用相同的数据训练多棵独立的决策树。这其实并不简单。机器学习从相同的数据上学习的结果基本上是一样的，即使有 100 棵决策树，如果它们的学习方法都一样，那么多数表决的结果也还是一样的。

随机森林尝试对每棵决策树的数据应用下述方法来进行训练，以使分类结果尽可能地不同。首先采用 Bootstrap 方法，根据训练数据生成多个不同内容的训练数据。所谓 Bootstrap 方法，即通过对单个训练数据进行多次随机的抽样放回，“虚增”训练数据，这样就可以为每棵决策树输入不同的训练数据。然后在根据使用 Bootstrap 方法创建的训练数据训练决策树时，只随机选取部分特征值来训练决策树。通过“Bootstrap 方法”和“随机选取特征值”这两种方法，就可以训练出具有多样性的决策树（图 2-33）。



▲图 2-33 Bootstrap 方法

随机森林利用这种方式创建多棵数据集、训练多棵决策树、对预测结果进行多数表决，返回最终的分类结果。

▀ 示例代码

让我们用随机森林基于 3 种葡萄酒的各种测量值数据，对葡萄酒进行分类。

```
▼ 示例代码

from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
# 读取数据
data = load_wine()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.3)
model = RandomForestClassifier()
model.fit(X_train, y_train) # 训练
y_pred = model.predict(X_test)
accuracy_score(y_pred, y_test) # 评估
```

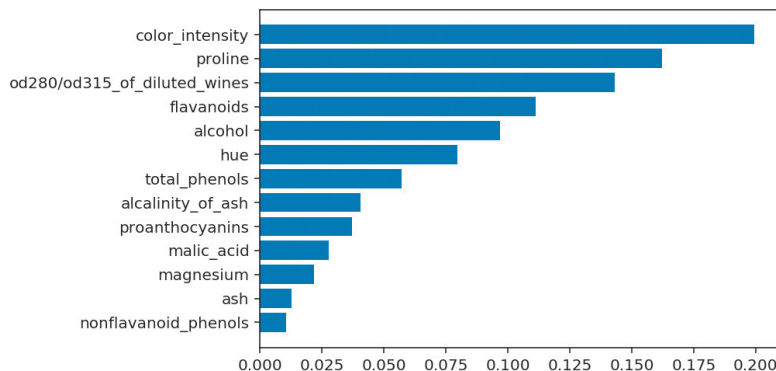
0.9444444444444442

详细说明

特征的重要度

随机森林可以让我们知道每个特征对预测结果的重要度。下面我们先了解一下特征重要度的计算方法，然后以葡萄酒分类为例，看一下各特征的重要度。首先介绍一下使用随机森林计算重要度的方法。前面讲过，每棵决策树的学习方法都是通过沿着某个值分割特征，使不纯度尽可能地小。通过对随机森林的所有决策树求在以某个特征分割时的不纯度并取平均值，可以得到特征的重要度。将重要度高的特征用于分割，有望大幅度减小不纯度。反之，重要度低的特征即使被用于分割，也无法减小不纯度，所以可以说这样的特征是非必要的。基于特征的重要度，我们可以去除非必要的特征。

接下来，我们以葡萄酒分类为例，看一下特征的重要度。图 2-34 所示为使用随机森林算出的特征重要度。图中重要度高的特征 `color_intensity` 表示色泽，这说明它对葡萄酒的分类非常重要，这个结论直观，让人信服。



▲图 2-34 重要度的可视化

2.8 算法 8：神经网络

一直以来，人们都认为**神经网络**（Neural Network，NN）是模仿生物体的神经网络设计而成的。

神经网络既可以用于回归，也可以用于分类，但在实际应用中常用于分类。基于神经网络的深度学习因在图像识别和语音识别等领域表现优异而广为人知。

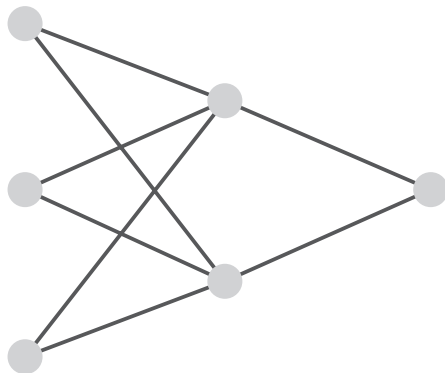
概述

神经网络在输入数据和输出结果之间插入了叫作中间层的层，能够学习复杂的决策边界。它既可以用于回归，也可以用于分类，但主要还是应用于分类问题。本节也以分类问题为例进行说明。

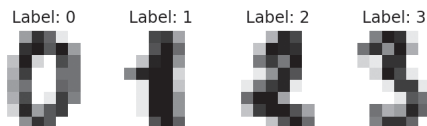
图 2-35 是一个典型的神经网络的网络结构。

在图 2-35 中，输入数据为三维数据，中间层为二维数据，输出为一维数据（关于从输入层到中间层、从中间层到输出层之间的计算，后面的“算法说明”部分将详细介绍）。左端的层叫作输入层，表示输入数据本身。右端的层叫作输出层，取输入数据分类结果的概率。在二元分类的情况下，输出层输出的概率只有一个。在多元分类的情况下，输出层同时输出属于每个分类对象的标签的概率。通过在输入层和输出层中间叠加中间层，神经网络得以学习复杂的决策边界。

下面我们将神经网络应用于具体任务，查看它的结果。本节将对一个叫作 MNIST 的手写数字数据集进行分类。MNIST 包含从 0 到 9 的 10 个手写数字的图片，我们对图片进行分类。MNIST 的图片是 8×8 的灰度图像，如图 2-36 所示。

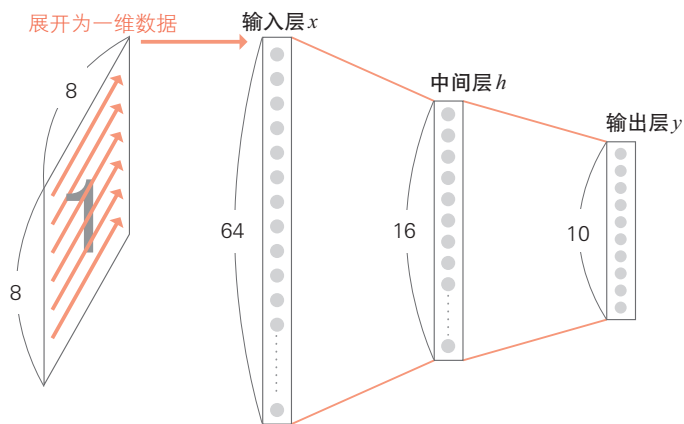


▲ 图 2-35 神经网络的典型网络结构



▲ 图 2-36 MNIST 手写数字数据示例

本次创建的神经网络如图 2-37 所示，图中省略了各个节点之间的连接线。



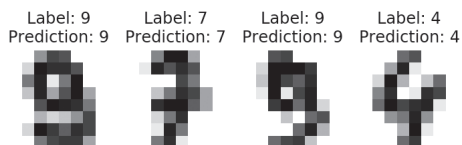
▲图 2-37 MNIST 的神经网络示意图

输入层表示输入图像（64 维向量）本身。如果将各个点的像素值存储在长度为 64 的一维数组的元素中，则可以将其视为 64 维数据来处理。下面通过神经网络来学习使用上述方法得到的 64 维数据。

中间层使用 Sigmoid 等非线性函数计算输入层传来的数据。中间层的维度是超参数。使维度变大可以学习更加复杂的边界，但是容易发生过拟合。本次设置中间层为 16 维。下面的“算法说明”部分将再次介绍中间层的计算方法、中间层的维度和学习结果的关系。

输出层也同样使用非线性函数计算中间层传来的数据。本次任务是对 0 ~ 9 这 10 个数字进行分类。因此，输出层输出的是输入手写图像分别为 0 ~ 9 这 10 个数字的概率。

下面使用这个神经网络进行学习，并进行分类（图 2-38）。



▲图 2-38 使用神经网络进行分类

图中的 Label 代表作为正确答案的数字，Prediction 代表作为神经网络的分类结果的数字，下方的图像是输入数据。可以看出，神经网络可以正确地识别出这些手写数字。

算法说明

通过叠加中间层，神经网络可以学习复杂的决策边界。下面首先介绍一下只由输入层和输出层构成的简单感知机，然后再探讨如何通过叠加中间层来学习更加复杂的决策边界。

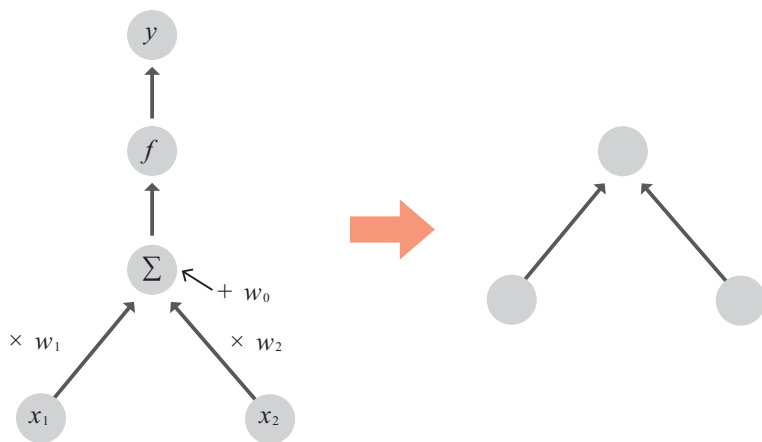
简单感知机

简单感知机是将非线性函数应用于对特征值加权后的结果并进行识别的模型。下面看一个例子：某特征的维度为 2，输入的特征值为 (x_1, x_2) ，使用下面的非线性函数 f 计算概率 y ：

$$y = f(w_0 + w_1x_1 + w_2x_2)$$

我们将其中的特征值的系数 w_1 和 w_2 称为权重，将常数项 w_0 称为偏置。权重和偏置都是学习参数。非线性函数 f 叫作激活函数，它以加权后的特征值之和作为输入，计算概率 y 。常用的激活函数有 Sigmoid 函数等。

图 2-39 的左边是简单感知机的示意图，展示了对加权后的输入和偏置之和应用激活函数并计算输出。我们有时也可以像图 2-39 右边那样省略求和的部分和激活函数，只显示输入和输出部分。在这种简化图中常常省略节点的变量名。

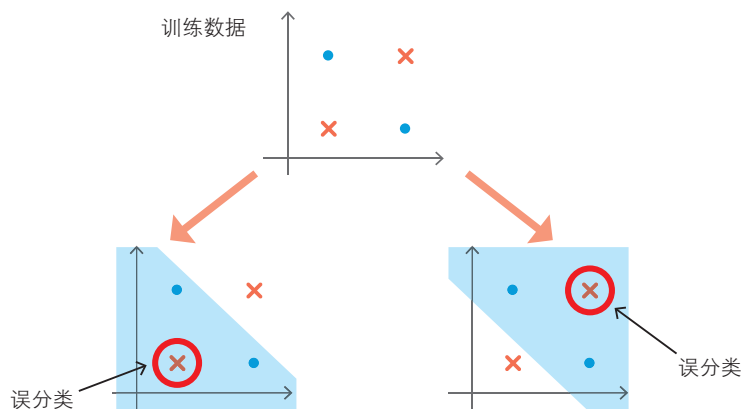


▲图 2-39 权重和激活函数的关系

简单感知机与逻辑回归具有类似的特性。其实在用 Sigmoid 函数作为激活函数 f 时，简单感知机和逻辑回归是等价的。

神经网络

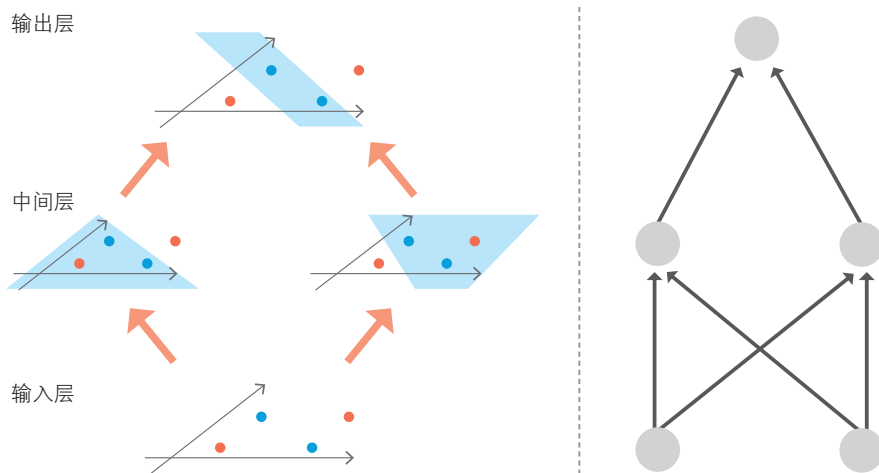
通过叠加简单感知机，神经网络得以表示复杂的决策边界。简单感知机不能很好地学习某些数据的决策边界。逻辑回归无法成功地对非线性决策边界进行分类，简单感知机也是如此。比如，对于如图 2-40 所示的线性不可分的数据，简单感知机无法进行正确分类。



▲ 图 2-40 简单感知机无法正确分类的例子

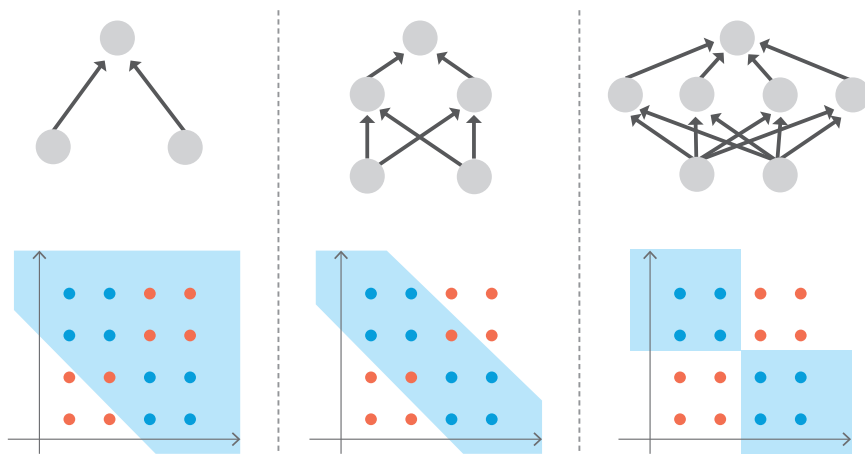
神经网络通过下面介绍的方式解决这个问题。首先，设置区分右上角的点和和其他点的层、区分左下角的点和和其他点的层。这些夹在输入和输出之间的层叫作中间层或隐藏层。然后，设置综合这两个输出结果并进行最终决定的层。通过这种做法，我们就可以根据数据是否进入被两条直线夹住的地方来分类了。

图 2-41 的左侧表示各个层的学习情况，右侧是模型的示意图。右图表示从二维输入到两个中间层的输出，再到得到最终的输出的过程。图中的节点表示特征及其输出等变量，边表示用于进行下一个变量的计算的输入。



▲图 2-41 神经网络的结构

神经网络通过这种设置中间层的方式，可以利用单一算法学习各种决策边界。通过调节中间层的数量及层的深度，神经网络可以学习更复杂的边界，如图 2-42 所示。



▲图 2-42 神经网络的示意图

■ 示例代码

下面读取“概述”部分提到的 MNIST 数据集，将其分割成训练数据和验证数据，使用训练数据训练模型，使用验证数据评估正确率。每次运行结果都有所不同，不过结果的正确率都在 95% 左右。

▼ 示例代码

```
from sklearn.datasets import load_digits
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
# 读取数据
data = load_digits()
X = data.images.reshape(len(data.images), -1)
y = data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
model = MLPClassifier(hidden_layer_sizes=(16, ))
model.fit(X_train, y_train) # 训练
y_pred = model.predict(X_test)
accuracy_score(y_pred, y_test) # 评估
```

```
0.95185185185185184
```

▶ 详细说明

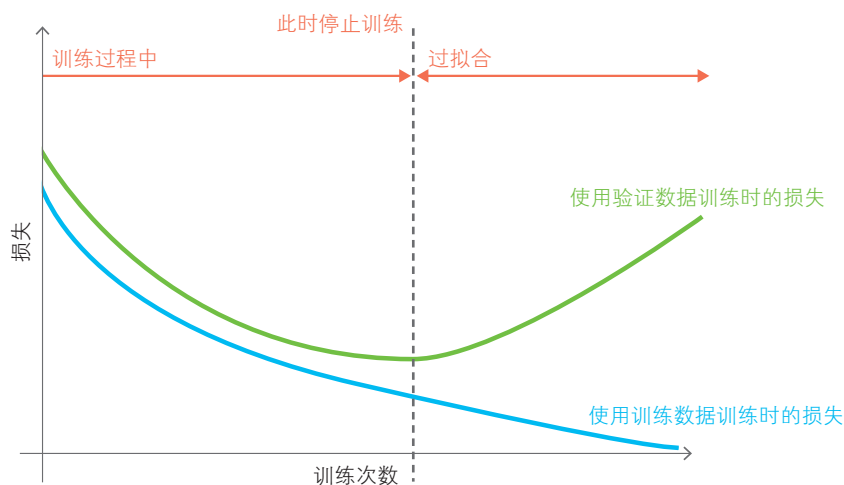
通过增加中间层的数量和增加中间层的节点数量等方法，神经网络可以学习复杂的数据。但是，模型变得复杂之后容易过拟合。下面介绍一种名为 **Early Stopping**（早停法）的防止过拟合的方法。

■ Early Stopping

Early Stopping 是指在进入过拟合状态前停止训练的防止过拟合的方法。与正则化的方法相比，它让人觉得有些不一样。

Early Stopping 进一步划分训练数据，将其中一部分作为训练中的评估数据。在训练过程中使

用评估数据，依次记录损失等评估指标，以了解训练的进度。在训练过程中，如果评估数据的损失开始恶化，开始出现过拟合的趋势，就停止训练，结束学习。这种在过拟合之前停止训练的做法就叫作 Early Stopping（图 2-43）。



▲ 图 2-43 Early Stopping

为了高效地进行神经网络的学习，还有各种技术可供使用。本书不再详细介绍，请读者参考 `scikit-learn` 的文档和本书推荐的图书。

2.9 算法 9: KNN

KNN (K-Nearest Neighbor, K 近邻) 算法是一种与众不同的机器学习算法, 它只是机械地记住所有的数据。

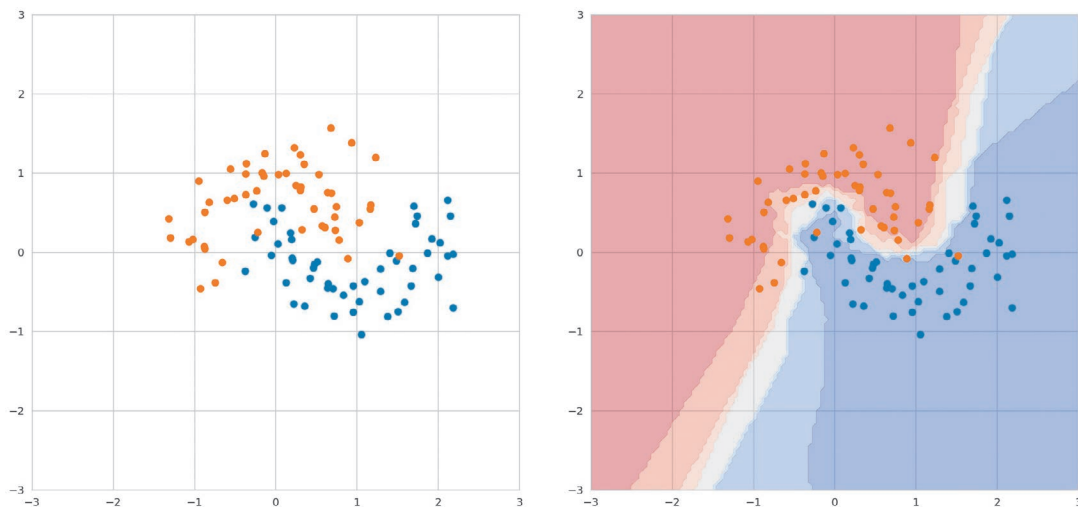
KNN 历史悠久, 早已为人所知。不过, 该算法虽然简单, 却可以学习复杂的边界。

概述

KNN 既可用于分类, 也可用于回归。本节以二元分类为例进行说明。KNN 算法在训练时机械地记住所有的训练数据。相较于其他算法要经历“根据训练数据计算最佳参数”的训练阶段和“使用计算出的学习参数进行预测”的预测阶段, KNN 在训练阶段不进行任何计算, 直到进入预测阶段之后才进行具体的计算。

在对未知数据进行分类时, KNN 将计算未知数据与训练数据的距离, 通过多数表决找到最邻近的 k 个点, 然后进行分类。

KNN 虽然是一种简单的算法, 但也适用于具有复杂边界的数据。图 2-44 是将 KNN 应用于具有复杂边界的数据而得到的结果。



▲ 图 2-44 使用 KNN 学习的例子

在图 2-44 中有橙色和蓝色两个标签的训练数据。在右图中，散点图中的所有点被分为两个标签，分类结果显示在了热图中。最近邻点的数量 k 设置为 5。热图中每个坐标的颜色表示 k 个最近邻标签的占比。暗红色区域表示 k 个点的标签都是橙色的，随着比例接近 1:1，颜色会变浅，接近于蓝色。

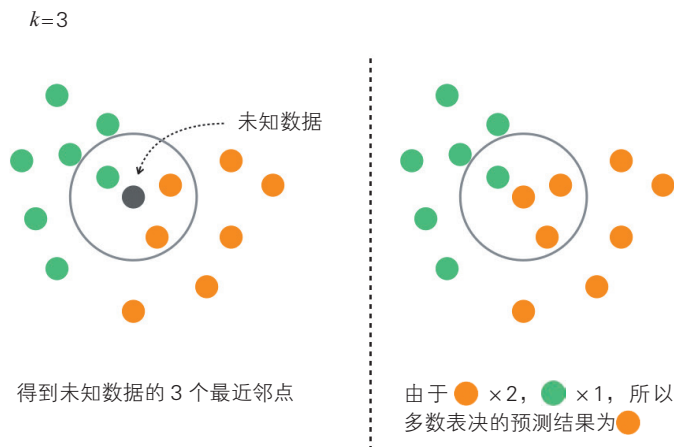
结果表明，KNN 算法能够对复杂数据进行学习。

算法说明

KNN 是一种在训练时机械地记住所有数据的简单算法。该算法使用训练数据对未知输入数据进行分类时的步骤如下。

1. 计算输入数据与训练数据之间的距离。
2. 得到距离输入数据最近的 k 个训练数据。
3. 对训练数据的标签进行多数表决，将结果作为分类结果。

多数表决的过程如图 2-45 所示，图中设置的最近邻点 k 的数量为 3。



▲图 2-45 通过多数表决进行分类

最近邻点 k 的数量是一个超参数。在二元分类时，通常取 k 为奇数，这样多数表决才能决定结果是其中的哪一个。

■ 示例代码

下面查看示例代码。我们使用呈曲线分布的样本数据进行学习，从而解决分类问题。最近邻点 k 的数量采用默认值 5。

▼ 示例代码

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 生成数据
X, y = make_moons(noise=0.3)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
model = KNeighborsClassifier()
model.fit(X_train, y_train) # 训练
y_pred = model.predict(X_test)
accuracy_score(y_pred, y_test) # 评估
```

```
0.9333333333333335
```

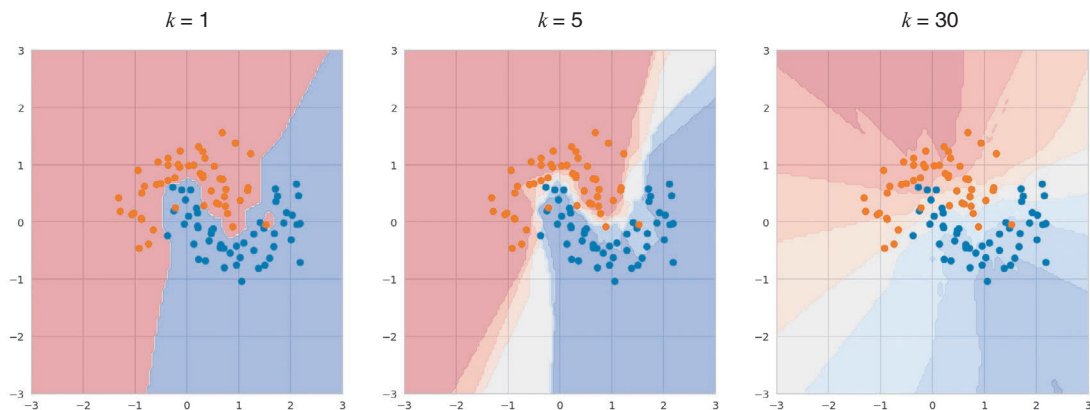
► 详细说明

■ 决策边界因 k 值而异

在 KNN 中， k 值是一个超参数。让我们改变 k 值，来看一下识别出的决策边界如何变化（图 2-46）。

在图 2-46 中，从左到右分别为 $k=1$ 、 $k=5$ 、 $k=30$ 时的决策边界。当 $k=1$ 时，图中出现了一些像飞地一样的决策边界，这说明发生了过拟合。中间的图的决策边界变得平滑，我们看不到在 $k=1$ 时出现的飞地，所以 $k=5$ 似乎比 $k=1$ 更好。而当 $k=30$ 时，背景为橙色的区域内夹杂着好多蓝色的点，这是边界过于宽松导致的错误判断。

总而言之， k 值的不同会导致学到的决策边界的外观发生变化。我们需要像其他算法一样对 k 值进行调优，得到最佳的 k 值。



▲图 2-46 学习结果因 k 值而异

注意

当数据量较小或维度较小时，KNN 的效果很好，但是当数据量较大或维度较大时，我们就需要考虑其他方法。

首先看一下数据量较大的情况。由于要处理大量的训练数据，所以分类将变慢。这是由于在未知数据进行分类时，KNN 需要在大量的训练数据上进行近邻搜索以找到最近的点。这就意味着 KNN 需要同时存储大量的训练数据，也就意味着需要存储容量。为了高效地进行近邻搜索，人们经常借助于利用树结构存储训练数据的技术，但一般来说，KNN 不适合处理大规模的训练数据。

KNN 也无法很好地学习高维数据。KNN 起作用的前提是“只要拥有的训练数据多，就能在未知数据的附近发现训练数据”这一假设。这个假设叫作**渐近假设**，但对于高维数据来说，这个假设不一定成立。对于高维的音频和图像数据，我们需要考虑其他方法。

虽然我们也可以使用前面提到的树结构的方式提高近邻搜索的速度，但对于大规模的数据，建议还是使用其他算法。



第 3 章

无监督学习

序 号	算 法
10	PCA
11	LSA
12	NMF
13	LDA
14	k -means 算法
15	混合高斯分布
16	LLE
17	t -SNE

3.1 算法 10: PCA

在众多降维算法中，PCA（Principal Component Analysis，主成分分析）历史悠久，被广泛应用于各个领域。

使用 PCA 可以将相关的多变量数据以主成分简洁地表现出来。

概述

PCA 是一种用于减少数据中的变量的算法。它对变量之间存在相关性的数据很有效，是一种具有代表性的降维算法。降维是指在保留数据特征的前提下，以少量的变量表示有许多变量的数据，这有助于降低多变量数据分析的复杂度。比如在分析有 100 个变量的数据时，与其直接分析数据，不如使用 5 个变量表示数据，这样可以使后续分析比较容易。

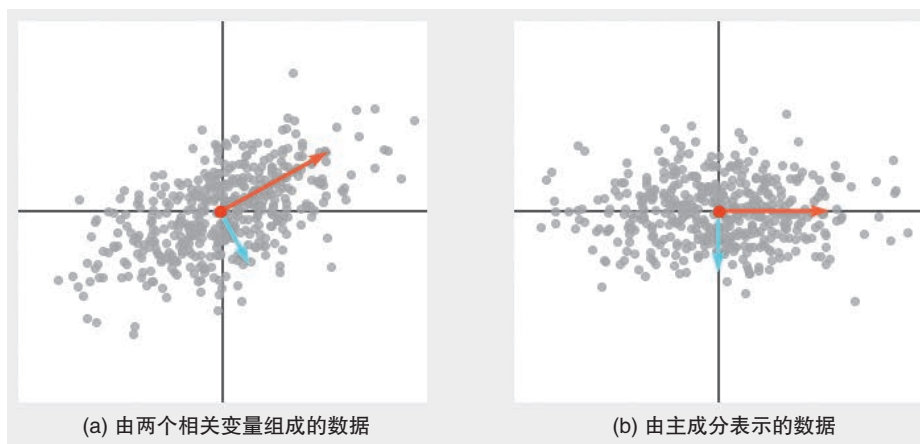
减少数据变量的方法有两种：一种是只选择重要的变量，不使用其余变量；另一种是基于原来的变量构造新的变量。PCA 使用的是后一种方法。

也就是说，PCA 可以使用低维变量表示高维空间中的数据。这个低维的轴叫作**主成分**，以原来的变量的线性组合的形式组成。

下面的“算法说明”部分将介绍寻找主成分的具体算法。下面结合示意图来介绍 PCA。

PCA 可发现对象数据的**方向**和**重要度**。对图 3-1a 的散点图应用 PCA，可以画出正交的两条线，线的方向表示数据的方向，长度表示重要度。方向由构成新变量时对象数据变量的权重决定，而重要度与变量的偏差有关。在每个数据点取相同值的变量并不重要，但是在每个数据点取不同值的变量能很好地体现数据整体的特点。

图 3-1b 是以这两条线为新轴对原始数据进行变换后得到的图形，变换后的数据称为**主成分得分**。按主成分轴的重要度的值从高到低排序，依次称它们为第一主成分、第二主成分。从转换后的图来看，第一主成分方向的方差大，第二主成分方向的方差不大。通过 PCA 计算得到的第一主成分在方差最大的轴上，所以这个新的轴包含了更多的原始数据的特点。

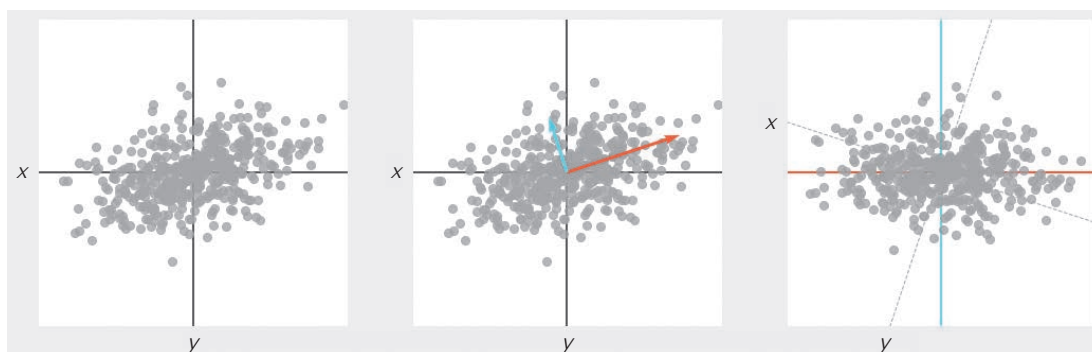


▲图 3-1 PCA 的示意图

▶ 算法说明

PCA 采用以下步骤来寻找主成分（图 3-2）。

1. 计算协方差矩阵。
2. 对协方差矩阵求解特征值问题，求出特征向量和特征值。
3. 以数据表示各主成分方向。



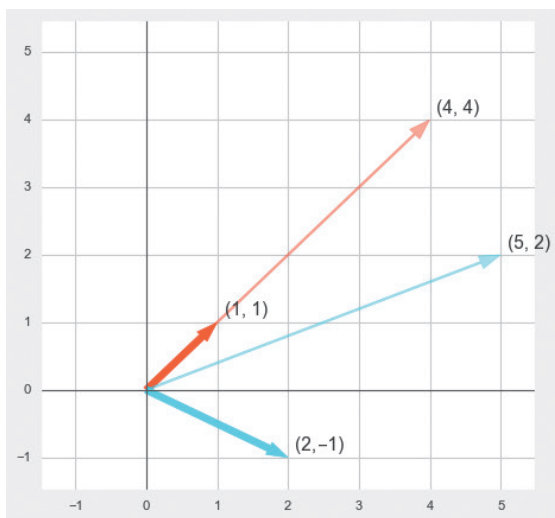
▲图 3-2 PCA 的算法

特征值问题是指对 n 维方形矩阵 A ，寻找使得 $A\mathbf{x} = \lambda\mathbf{x}$ 的 λ 和 $\mathbf{x}$ 的问题。求解协方差矩阵的特征值问题在数学上等同于寻找使方差最大化的正交轴的问题。

让我们观察一下式 $A\mathbf{x} = \lambda\mathbf{x}$ 。左边是一个矩阵和向量的乘积。举个具体的例子，将下面的向量 $\mathbf{x}_1$ 、 $\mathbf{x}_2$ 与矩阵 A 相乘。

$$\begin{aligned}\mathbf{x}_1 &= \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \mathbf{x}_2 = \begin{pmatrix} 2 \\ -1 \end{pmatrix} \\ A &= \begin{pmatrix} 3 & 1 \\ 2 & 2 \end{pmatrix} \\ A\mathbf{x}_1 &= \begin{pmatrix} 3 & 1 \\ 2 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 4 \\ 4 \end{pmatrix} \\ A\mathbf{x}_2 &= \begin{pmatrix} 3 & 1 \\ 2 & 2 \end{pmatrix} \begin{pmatrix} 2 \\ -1 \end{pmatrix} = \begin{pmatrix} 5 \\ 2 \end{pmatrix}\end{aligned}$$

与矩阵相乘后， $\mathbf{x}_1$ 、 $\mathbf{x}_2$ 分别变换为 $(4, 4)$ 和 $(5, 2)$ ，如图 3-3 所示。像这个例子一样，矩阵可以变换向量的大小和方向，但有的向量并不能通过这种变换改变方向，这样的向量叫作 A 的特征向量。这时向量的大小在该变换下缩放的比例叫作特征值。



▲ 图 3-3 通过矩阵变换向量

从图 3-3 来看，以红线表示的变换后的向量 $(4, 4)$ 与原向量在一条直线上。这就意味着乘以矩

阵 A 的变换与乘以常数 4 的变换是一样的。

在 PCA 中，矩阵 A 就是协方差矩阵。PCA 对协方差矩阵求解特征值问题，并计算出特征值和特征向量的几种组合。这时按特征值大小排序的特征向量分别对应于第一主成分、第二主成分……照此类推。

“概述”部分介绍的方向和重要度与特征值问题的特征向量和特征值相关。

如果我们使用针对每个主成分计算出的特征值除以特征值的总和，就能够以百分比来表示主成分的重要度，这个比例叫作贡献率，它表示每个主成分对数据的解释能力。从第一主成分开始依次相加，得到的贡献率叫作累计贡献率。

■ 示例代码

下面是对 scikit-learn 内置的鸢尾花数据使用 PCA 的示例代码，用于将 4 个特征变量变换为 2 个主成分。

▼ 示例代码

```
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
data = load_iris()
n_components = 2 # 将减少后的维度设置为2
model = PCA(n_components=n_components)
model = model.fit(data.data)
print(model.transform(data.data)) # 变换后的数据
```

```
[[ -2.68420713  0.32660731]
 [ -2.71539062 -0.16955685]
 [ -2.88981954 -0.13734561]
 ~ 略 ~
 [ 1.76404594  0.07851919]
 [ 1.90162908  0.11587675]
 [ 1.38966613 -0.28288671]]
```

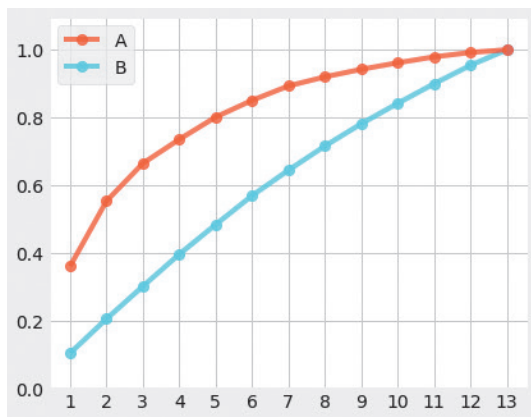
详细说明

主成分的选择方法

PCA 可以将原始数据中的变量表示为新的轴的主成分。这些主成分按照贡献率大小排序，分别为第一主成分、第二主成分……照此类推。这时通过计算累计贡献率，我们可以知道使用到第几个主成分为止可以包含原始数据多少比例的信息。

图 3-4 中的横轴为主成分，纵轴为累计贡献率，其中 A 为变量之间存在相关性的数据的 PCA 结果。这时的累计贡献率分别为 0.36、0.55、0.67、0.74、0.80……当根据累计贡献率选取主成分时，我们可以像“累计贡献率在 0.7 以上→4 个主成分”“累计贡献率在 0.8 以上→5 个主成分”这样，根据基准值决定主成分的数量。

而 B 是对变量之间没有相关性的数据进行 PCA 的结果，从图中可以看出，各个主成分的贡献率几乎相同。可以说这样的数据不适合用 PCA 进行降维，需要考虑其他方法。



▲ 图 3-4 主成分在各个数据集上的累计贡献率

参考文献

Smith, L.I. A tutorial on principal components analysis [EB/OL]. Cornell Univ, 2002.
株式会社日经调查. 主成分分析 [EB/OL]. [2019-2-6].

3.2 算法 11: LSA

LSA (Latent Semantic Analysis, 潜在语义分析) 是一种自然语言处理技术。作为一种降维算法, 它常被用于信息搜索领域。

使用 LSA 能够从大量的文本数据中找出单词之间的潜在关联性。

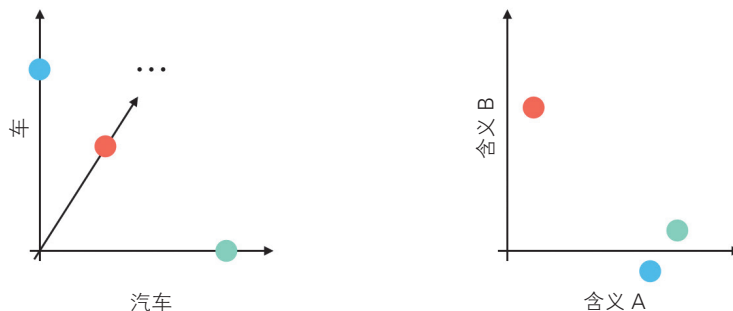
概述

LSA 是在 1988 年被提出的算法, 用于信息搜索领域。当时信息搜索的做法是, 事先对作为搜索对象的文本中包含的单词建立索引 (index), 如果索引与搜索关键字相同, 就将该文本加入到搜索结果中。

但是这种方法有一个缺点: 如果对搜索对象建立的索引与搜索关键字有一点点不同, 那就无法很好地找到信息。假设有一篇索引中包含“车”的文章, 在使用“汽车”这个搜索关键字搜索时, 就会出现搜索结果中没有这篇文章的问题 (同义性的问题)。虽然我们人类都知道“车”和“汽车”是含义基本上相同的单词, 但是一个词一个词地教给计算机“这个单词和这个单词含义相似”的做法是不现实的。

而使用 LSA 就可以根据大量文本自动计算单词和单词的相似度, 以及单词和文本的相似度。

通过 LSA 对文本和单词的矩阵进行降维, 将其变换为潜在语义空间 (图 3-5)。这种变换使用矩阵分解进行。矩阵分解是指将某个矩阵表示为多个矩阵的乘积的形式。矩阵分解是如何用于无监督学习的降维算法的呢? 下面的“算法说明”部分将进行介绍。



▲ 图 3-5 潜在语义空间的示意图

在单词空间中，车和汽车都被当作正交维度处理，而在语义空间中，二者被表示为相似的单词。

算法说明

下面结合具体的例子来讲解矩阵分解和降维。

首先将以下文本变换为矩阵 X 。矩阵 X 的各元素是文本中出现的单词的个数（表 3-1）。

- 坐汽车去公司
- 坐车去的
- 在餐厅吃汉堡牛肉饼
- 在餐厅吃意大利面

▼表 3-1 文本中出现的单词的个数

	文 本 1	文 本 2	文 本 3	文 本 4
汽车	1	0	0	0
公司	1	0	0	0
去	1	1	0	0
车	0	1	0	0
餐厅	0	0	1	1
汉堡牛肉饼	0	0	1	0
吃	0	0	1	1
意大利面	0	0	0	1

我们先未卜先知地看一下对矩阵 X 进行矩阵分解后的结果。8 行 4 列的 X 被表示为 8 行 4 列的 U 、4 行 4 列的矩阵 D 和 4 行 4 列的矩阵 V^T 的乘积的形式。

$$X = UDV^T$$

$$= \begin{bmatrix} 0.00 & -0.45 & -0.45 & 0.00 \\ 0.00 & -0.45 & -0.45 & 0.00 \\ \vdots & \vdots & \vdots & \vdots \\ -0.32 & 0.00 & 0.00 & -0.71 \end{bmatrix} \times \begin{bmatrix} 2.24 & 0 & 0 & 0 \\ 0 & 1.90 & 0 & 0 \\ 0 & 0 & 1.18 & 0 \\ 0 & 0 & 0 & 1.00 \end{bmatrix} \times \begin{bmatrix} 0.00 & 0.00 & -0.71 & -0.71 \\ -0.85 & -0.53 & 0.00 & 0.00 \\ -0.53 & 0.85 & 0.00 & 0.00 \\ 0.00 & 0.00 & 0.71 & -0.71 \end{bmatrix}$$

其中， U 是包含单词和归纳的特征的变换信息的矩阵， D 是包含信息的重要度的矩阵， V 是包含归

纳的特征和文本的变换信息的矩阵。

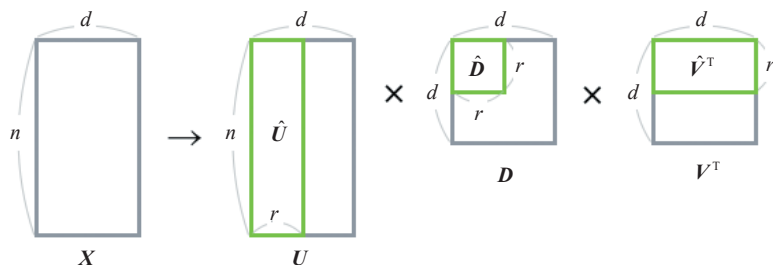
另外, $\mathbf{D}$ 是一个对角矩阵, (1, 1)、(2, 2) 等对角线上的元素之外的元素都是 0, 对角元素按信息的重要度从大到小排列。在使用 3 个矩阵降维时, 我们重点关注 $\mathbf{D}$ 。下面思考这样一个需求: 原始数据有 4 个特征, 但我们希望将其降维到 2 个特征。

从 $\mathbf{D}$ 的 4 个值中选出最重要的 2 个, 建立一个 2 行 2 列的对角矩阵。为了匹配这个 $\mathbf{D}$, 我们相应地删去 $\mathbf{U}$ 的第 3 列和第 4 列, 以及 $\mathbf{V}^T$ 的第 3 列和第 4 列, 将它们分别变形为 8 行 2 列和 2 行 4 列的矩阵。

$$\begin{aligned}\hat{\mathbf{X}} &= \hat{\mathbf{U}}\hat{\mathbf{D}}\hat{\mathbf{V}}^T \\ &= \begin{bmatrix} 0.00 & -0.45 \\ 0.00 & -0.45 \\ \vdots & \vdots \\ -0.32 & 0.00 \end{bmatrix} \times \begin{bmatrix} 2.25 & 0 \\ 0 & 1.90 \end{bmatrix} \times \begin{bmatrix} 0.00 & 0.00 & -0.71 & -0.71 \\ -0.85 & -0.53 & 0.00 & 0.00 \end{bmatrix}\end{aligned}$$

这些矩阵的乘积 $\hat{\mathbf{X}}$ 是原来的矩阵 $\mathbf{X}$ 的近似。即使我们只用了 $\mathbf{D}$ 的值中的一半 (2 个值), $\hat{\mathbf{X}}$ 还是在一定程度上保留了原来的信息。

当作为降维算法使用时, 我们要用到的是在变换为原始特征的形式之前 (在乘以 $\hat{\mathbf{V}}^T$ 之前) 的 $\hat{\mathbf{U}}\hat{\mathbf{D}}$ 。 $\hat{\mathbf{U}}\hat{\mathbf{D}}$ 是一个 8 行 2 列的矩阵, 我们可以将其解释为从归纳的特征中选择的 2 个重要度高的特征 (图 3-6)。



▲图 3-6 对 n 行 d 列的 $\mathbf{X}$ 进行矩阵分解的示意图 (在将维度减少到 r 个时, 使用 $\hat{\mathbf{U}}\hat{\mathbf{D}}$)

我们来看一下 $\hat{\mathbf{U}}\hat{\mathbf{D}}$ 的具体数值。设这里归纳的 2 个特征为 A 和 B 。 A 和 B 没有明确的意义, 但它们是基于一词的关联性而创建的具有潜在意义的特征 (表 3-2)。

▼表 3-2 两个特征

使用潜在语义空间表示的单词

	A	B
汽车	0.00	0.85
公司	0.00	0.85
去	0.00	1.38
车	0.00	0.53
餐厅	1.41	0.00
汉堡牛肉饼	0.71	0.00
吃	1.41	0.00
意大利面	0.71	0.00

“汽车”和“车”拥有变量 B 的值，“汉堡牛肉饼”和“意大利面”拥有变量 A 的值。 A 和 B 的特征值显示了各个单词之间的关联性（图 3-7）。



▲图 3-7 以潜在变量表示的单词

■ 示例代码

下面使用 Python 代码解决前面探讨的问题。假设有一个使用 8 个变量（= 单词的个数）表示的数据集，现在用 2 个潜在变量来表示它。

▼ 示例代码

```

from sklearn.decomposition import TruncatedSVD
data = [[1, 0, 0, 0],
        [1, 0, 0, 0],
        [1, 1, 0, 0],
        [0, 1, 0, 0],
        [0, 0, 1, 1],
        [0, 0, 1, 0],
        [0, 0, 1, 1],
        [0, 0, 0, 1]]
n_components = 2 # 潜在变量的个数
model = TruncatedSVD(n_components=n_components)
model.fit(data)
print(model.transform(data)) # 变换后的数据
print(model.explained_variance_ratio_) # 贡献率
print(sum(model.explained_variance_ratio_)) # 累计贡献率

```

```

[[ 0.00000000e+00  8.50650808e-01]
 [ 0.00000000e+00  8.50650808e-01]
 [-1.08779196e-15  1.37638192e+00]
 [-1.08779196e-15  5.25731112e-01]
 [ 1.41421356e+00  8.08769049e-16]
 [ 7.07106781e-01  2.02192262e-16]
 [ 1.41421356e+00  8.08769049e-16]
 [ 7.07106781e-01  6.06576787e-16]]

```

```
[0.38596491 0.27999429]
```

```
0.6659592065833297
```

另外，与 PCA 一样，我们也可以检查 LSA 变换后的矩阵中包含多少原始信息。使用了 `scikit-`

learn 的上述代码输出的累计贡献率约为 0.67，表明这 2 个变量包含了约 67% 的原始数据的信息。

详细说明

■ 使用 LSA 时的注意事项

“算法说明”部分提到的矩阵分解是一种叫作**奇异值分解**的技术。使用了奇异值分解的 LSA 在信息检索中的应用备受关注，它具有以新的空间表示文本等优点。但在实际使用中，需要注意一些问题。

一是变换后的矩阵有时难以解释。在通过奇异值分解降维时，各个维度可能是正交的，矩阵中的元素也可能是负值。因此，与之相比，还是后面即将介绍的 NMF 和 LDA 等算法的结果更容易解释。

二是 LSA 的计算成本有时很高。特别是在用于文本时，由于原始矩阵的维度就是单词的个数，所以 LSA 必须在非常大的矩阵上进行奇异值分解。

还有一个与计算成本有关的问题：随着新词的加入，原有的矩阵必须重新创建，我们必须在此基础上重新计算，所以模型的更新难度很大。

参考文献

Deerwester, S., Dumais, S.T., Furnas, G.W., et al Indexing by Latent Semantic Analysis [J].
Journal of the American Society for Information Science 41, 1990, 391-407.

3.3 算法 12: NMF

NMF (Non-negative Matrix Factorization, 非负矩阵分解) 是一种降维算法, 它的特点是输入数据和输出数据的值都是非负的。

这个特点在处理图像等数据时有一个优点, 那就是模型的可解释性强。

概述

NMF 是一种矩阵分解方法, 在计算机视觉、文本挖掘、推荐等各个领域都有应用。与 LSA 一样, 它也可以找到矩阵的潜在变量, 但 NMF 只适用于原始矩阵的所有元素都是非负数 (即大于等于 0) 的情况。NMF 具有以下特点。

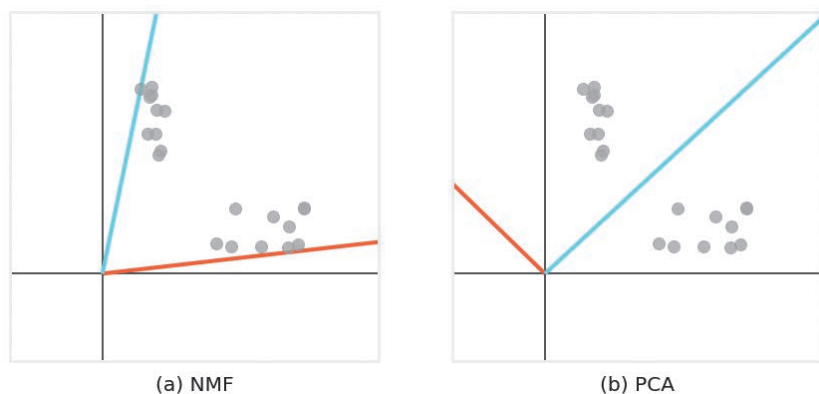
- 原始矩阵的元素是非负数。
- 分解后矩阵的元素是非负数。
- 没有“潜在语义空间的每一个维度都是正交的”这一约束条件。

在将 NMF 应用于真实数据时, 这些特点会带来几个优点。一个优点是分析结果的可解释性强。例如, 在将 NMF 应用于文本数据时, 由于文本能够以潜在变量的加法来表示, 所以可以使用 NMF 对文本进行降维, 然后将潜在变量视为主题, 这样就能使用“某个文本的主题 A 为 0.5、主题 B 为 0.3……”的方式描述文本的信息。由于实际的文本 (如新闻文章和论文) 也可能包含多个主题, 这就使得它们的文本建模结果的可解释性更强。但如果潜在变量为负值, 比如主题 A 为 -0.3、主题 B 为 0.6……就比较难解释。

另外, NMF 没有“潜在变量必须正交”的约束条件, 所以各个潜在变量可能有一定程度的信息重复。

拿前面的文本主题的例子来说, 这就意味着各个主题有一定程度的信息重复。这也是符合真实数据情况的建模。

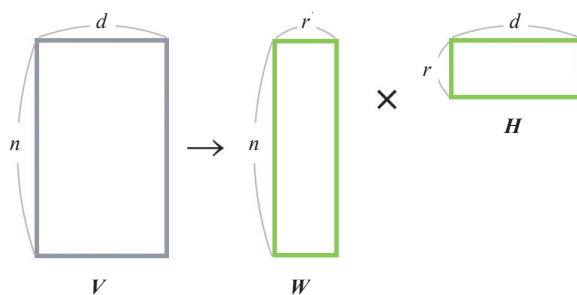
图 3-8 所示为对二维数据分别应用 NMF 和 PCA 的结果。从图中可以看出, NMF 的潜在空间的每个轴都有重复信息。这一特性使得我们可以捕捉到多个数据块的特征。而 PCA 等算法则由于其潜在空间的维度是正交的, 所以无法找到所有数据块的特征。



▲图 3-8 对二维数据分别应用 NMF 和 PCA 的结果

算法说明

NMF 作为一种矩阵分解方法，通过将原始数据分解为两个矩阵来降维（图 3-9）。设原始数据为 n 行 d 列的矩阵 V ，将其表示为两个矩阵 W 和 H 的乘积。 W 是 n 行 r 列的矩阵， H 是 r 行 d 列的矩阵， WH 是原始矩阵 V 的近似，选择比 d 小的 r 就可以进行降维。这时， W 的每一行都是对 V 的每一行降维后的结果。



▲图 3-9 对 n 行 d 列的 V 进行矩阵分解的示意图（在降维到 r 个特征时，使用 W ）

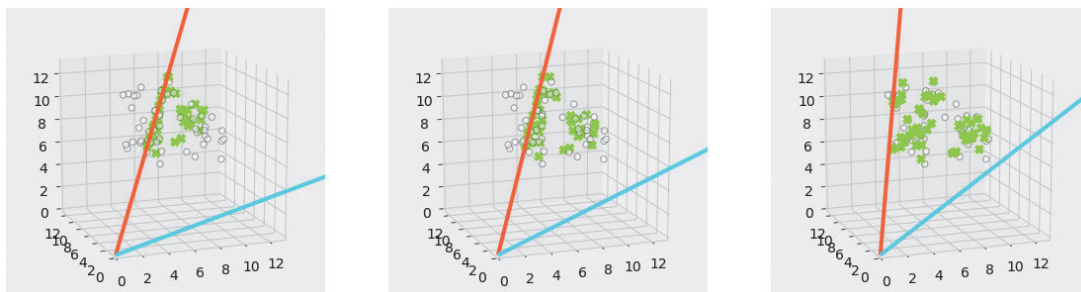
在求 W 和 H 的过程中，NMF 在 $W \geq 0$ 、 $H \geq 0$ 的条件下，使 WH 接近 V 。

NMF 采取“将 H 视为常数，更新 W ”“将 W 视为常数，更新 H ”的方式交替更新 W 和 H 。

下面对 NMF 计算过程中的每一步进行可视化。

灰色的点为原始矩阵 V ，绿色的点为近似矩阵 WH 。随着计算的进行，我们可以看到近似矩阵越来越接近原始矩阵。此外，红线和蓝线是潜在空间的轴，所有近似矩阵的图形都能在潜在空间（二维空间）的轴上表示出来（图 3-10）。

- 将 W 和 H 初始化为正值。
- 将 H 视为常数，更新 W 。
- 将 W 视为常数，更新 H 。
- 当 W 和 H 收敛时，停止计算。



▲ 图 3-10 对三维数据应用 NMF 时参数更新的示意图

■ 示例代码

下面是使用 scikit-learn 运行 NMF 算法的示例代码。

▼ 示例代码

```
from sklearn.decomposition import NMF
from sklearn.datasets.samples_generator import make_blobs
centers = [[5, 10, 5], [10, 4, 10], [6, 8, 8]]
V, _ = make_blobs(centers=centers) # 以centers为中心生成数据
n_components = 2 # 潜在变量的个数
model = NMF(n_components=n_components)
model.fit(V)
W = model.transform(V) # 分解后的矩阵
```

```
H = model.components_
print(W)
print(H)
```

```
[[0.93222442 0.51572953]
 [0.47720456 0.97821177]
 [0.95984912 0.63949264]
 ~略~
 [0.59450123 0.85559602]
 [1.31398151 0.11257937]
 [0.55687813 1.11451525]]
```

```
[[7.29377403 1.2260137 7.80020415]
 [0.          9.42217044 0.17445289]]
```

详细说明

NMF 和 PCA 的比较

现在我们已经了解了 NMF 算法，下面尝试将 NMF 应用到一个具体的数据集上。使用的数据是人脸图像数据（19 像素 × 19 像素，2429 张），图 3-11 是其中一张图像。为了进行比较，我们同时使用 PCA 进行降维。不管使用哪种算法，都假定降维后由 49 个变量表示数据。由于原始图像数据有 361 个特征（=19 像素 × 19 像素），所以我们的任务是将 361 个特征变换为 49 个潜在变量。

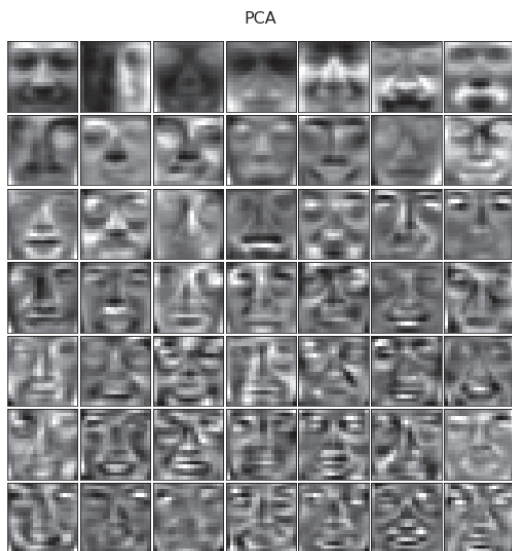
不管是 PCA 还是 NMF，潜在变量都是根据原始特征计算出来的，所以每个潜在变量与 361 个原始特征有关（该特征在 NMF 算法中是矩阵 H 的某一行）。将 49 个潜在变量的这种关系可视化，得到图 3-12 和图 3-13。在可视化时，为了便于查看，我们使用每个变量的最小值和最大值进行了缩放。在恢复原始图像时，我们使用变换后的数



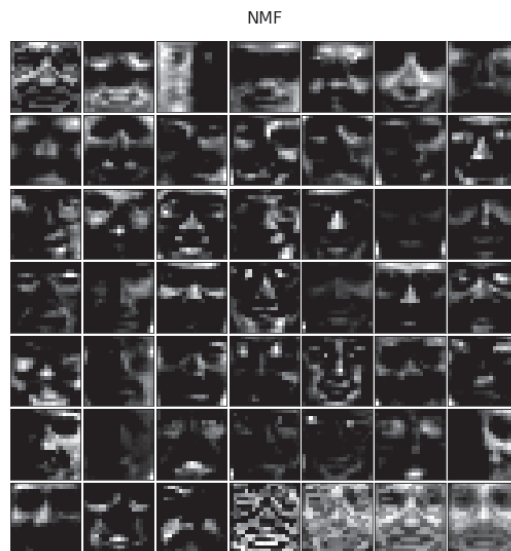
▲图 3-11 人脸图像

据点（NMF 中矩阵 W 的某一行）和可视化数据的乘积（图 3-14）。这相当于计算 1 行 49 列的矩阵和 49 行 361 列矩阵的乘积，得到 1 行 361 列的矩阵。

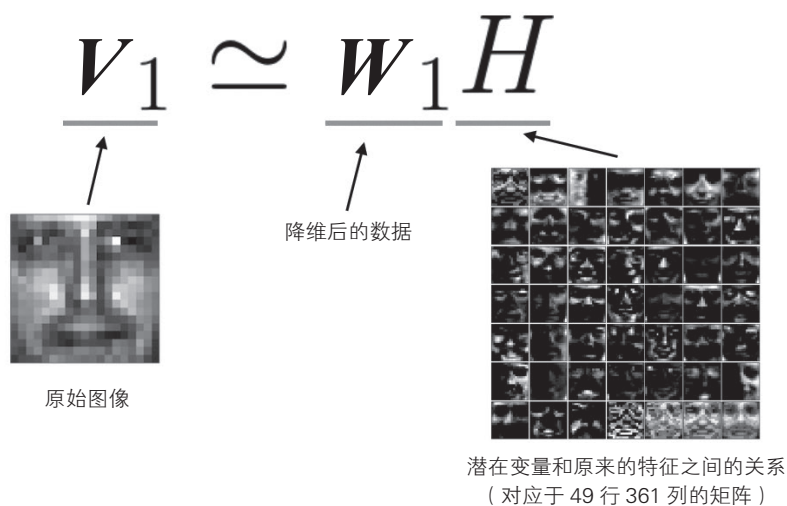
图 3-12 是 PCA 的结果，图像中负值看起来暗，正值看起来亮。图像从左上角开始按特征贡献度从高到低的顺序排列。可以看出，每张图像都表示人的整个面部。图 3-13 是 NMF 的结果，图像中暗的区域较多，但这些区域的值为 0。我们还可以看出，每个潜在变量都代表了人脸的一部分特征。通过还原原始图像，可以明确两种方法的不同特点。PCA 通过将不同的脸（负的脸和正的脸。听上去是不是有些奇怪？）加在一起恢复原始图像。NMF 则通过组合具有人脸部分特征的图像来恢复原始图像。NMF 的潜在变量的含义（在本例中就是人脸的部分特征）的可解释性更强。



▲ 图 3-12 对人脸图像应用 PCA 的结果



▲ 图 3-13 对人脸图像应用 NMF 的结果



▲图 3-14 根据降维数据恢复图像的结果

参考文献

D. Lee, H. Seung. Algorithms for non-negative matrix factorization, Advances in neural information processing [J]. systems 13, 2001, 556-562.

3.4 算法 13: LDA

LDA (Latent Dirichlet Allocation, 隐含狄利克雷分布) 是一种降维算法, 适用于文本建模。

一篇新闻文本可能会有一个诸如“体育”或“教育”等的主题, 也可能有多个主题。使用 LDA 可以将文本中的单词作为输入, 为其分配多个主题。

概述

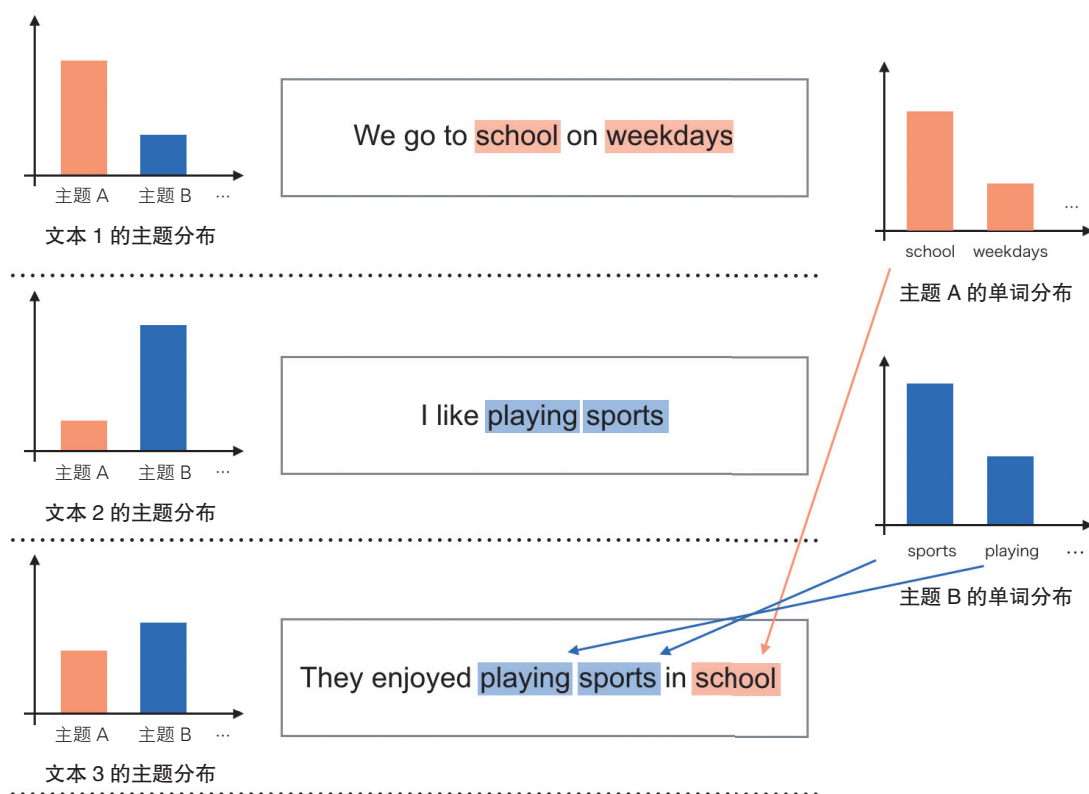
LDA 是一种用于自然语言处理等的算法。该算法可以根据文本中的单词找出潜在的主题, 并描述每个文本是由什么主题组成的, 还可以用于说明一个文本不只有一个主题, 而是有多个主题。例如, 一篇真实的新闻文本可能包含多个主题, 如“体育”和“教育”等, 使用 LDA 就可以很好地描述这种新闻文本。

为了便于理解, 我们来思考几个具体的例子。对以下 5 个例句应用 LDA, 结果会是什么样呢? 假设这些例句中的主题数为 2。

1. We go to school on weekdays.
2. I like playing sports.
3. They enjoyed playing sports in school.
4. Did she go there after school?
5. He read the sports columns yesterday.

我们可以将主题看作单词的概率分布。推测出的主题 A 和主题 B 的单词的概率分布如图 3-15 所示, school 是主题 A 的代表性单词, sports 是主题 B 的代表性单词。此外, 也可以推测文本中包含的主题的比例, 从而以主题的概率分布 (主题分布) 描述各文本。

如图 3-15 所示, LDA 可以利用主题分布和单词分布创建文本数据。



▲图 3-15 基于每个文本的主题分布和每个主题的单词分布创建文本

具体做法是基于文本的主题分布选择主题，之后基于主题的单词分布选择文本中的单词。重复这一操作，就能得到生成文本的模型。下面的“算法说明”部分将介绍如何根据输入数据计算主题分布和单词分布。

1. 基于文本的主题分布为单词分配主题。
2. 基于分配的主题的单词分布确定单词。
3. 对所有文本中包含的单词执行步骤 1 和步骤 2 的操作。

算法说明

LDA 通过以下步骤计算主题分布和单词分布。

1. 为各文本的单词随机分配主题。
2. 基于为单词分配的主题，计算每个文本的主题概率。
3. 基于为单词分配的主题，计算每个主题的单词概率。
4. 计算步骤 2 和步骤 3 中的概率的乘积，基于得到的概率，再次为各文本的单词分配主题。
5. 重复步骤 2 ~ 步骤 4 的计算，直到收敛。

根据步骤 4 中计算得到的概率，为各文本的单词分配主题。由于步骤 2 中确定了文本的主题概率，所以在同一个文本内，某些主题被选中的可能性较大。另外，同一个文本中的单词往往被选为同一主题。通过重复这样的计算，文本分配到特定主题的概率就会增加。同时，由于与每个主题相关联的单词更容易被选中，所以单词作为代表主题的词的的概率也会增加。

■ 示例代码

下面使用 scikit-learn 实现基于 LDA 的主题模型的创建。我们使用一个名为 20 Newsgroups 的数据集，这个数据集是 20 个主题的新闻组文本的集合，每个文本属于一个主题。

▼ 示例代码

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation
# 使用remove去除正文以外的信息
data = fetch_20newsgroups(remove=('headers', 'footers', 'quotes'))
max_features = 1000
# 将文本数据变换为向量
tf_vectorizer = CountVectorizer(max_features=max_features,
                                stop_words='english')
tf = tf_vectorizer.fit_transform(data.data)
n_topics = 20
model = LatentDirichletAllocation(n_components=n_topics)
model.fit(tf)
print(model.components_) # 各主题包含的单词的分布
print(model.transform(tf)) # 使用主题描述的文本
```

```

[[5.00661573e-02 5.00001371e-02 1.15548091e+01 ... 5.00000866e-02
 5.00000382e-02 5.00001268e-02]
[5.00000042e-02 5.00000039e-02 5.00000035e-02 ... 5.00491344e-02
 5.00000045e-02 5.00000054e-02]
[5.00000041e-02 5.00034336e-02 5.00000066e-02 ... 5.32886389e+02
 5.00000282e-02 1.07251934e+02]
~略~
[5.00000064e-02 5.00000043e-02 5.00000074e-02 ... 5.00970504e-02
 5.00000045e-02 1.98535651e+00]
[5.00000046e-02 5.00003898e-02 5.00000046e-02 ... 5.00116838e-02
 5.00000693e-02 1.12692943e+02]
[5.00000048e-02 5.00000068e-02 5.00000047e-02 ... 6.19074192e+01
 5.00000045e-02 9.91448535e-02]]

[[0.00208333 0.00208333 0.28397031 ... 0.09848581 0.00208333 0.00208333]
[0.0025      0.10318462 0.0025      ... 0.07045392 0.0025      0.0025      ]
[0.00060241 0.00060241 0.43801521 ... 0.13849737 0.04999365 0.00060241]
~略~
[0.00454545 0.09681648 0.36397178 ... 0.00454545 0.00454545 0.00454545]
[0.00294118 0.00294118 0.16126703 ... 0.00294118 0.39963043 0.11214416]
[0.00357143 0.13604762 0.36396555 ... 0.00357143 0.00357143 0.00357143]]

```

详细说明

使用主题描述文本

下面我们仔细看一下刚刚在“算法说明”部分训练的 LDA 的结果。

下图按照概率从高到低的顺序列出了每个主题分布中的单词，由此我们可以知道哪些单词代表主题。

例如，主题 16 是关于体育的，主题 18 是关于计算机的……我们可以根据主题中包含的单词来解释主题。

```
- 主题16:
game team year games season play hockey players league win teams ...

- 主题18:
windows drive card scsi disk use problem bit memory dos pc using ...

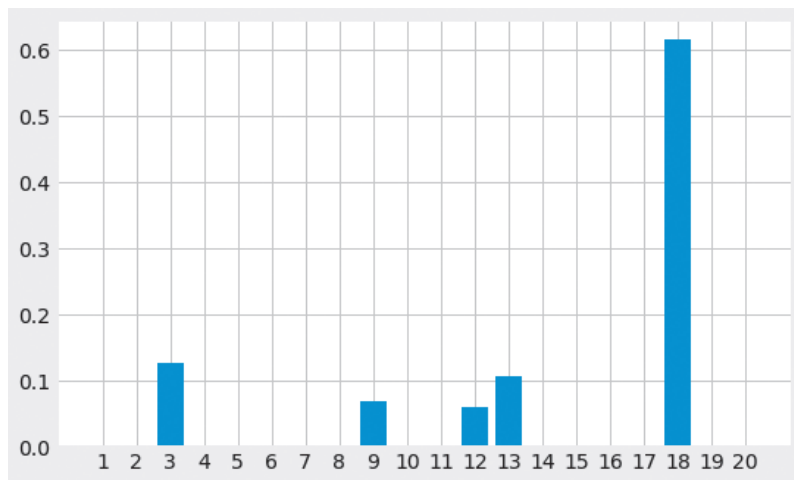
...
```

另一些主题要么只包含数值，要么包含的都是一些不能表达主题的不特指的单词，如下图所示。这些结果乍一看很难解释，下面通过停用词（为了提高精度而排除在外的单词）来对其进行改进。

```
- 主题4:
00 10 25 15 12 11 16 20 14 13 18 30 50 17 55 40 21 ...

- 主题6:
people said know did don didn just time went like say think told ...
```

接下来，我们来看一下文本的主题分布。图 3-16 所示为某文本的主题分布图。由于它包含了较多主题 18 的成分，所以可以断定这是关于计算机的文本。这个文本的实际主题是 `comp.sys.mac.hardware`，是关于苹果公司的 Mac 的文本。



▲图 3-16 某个文本的主题分布（主题数：20 个）

如果只用单词来描述文本的特征，就很难直观地理解，但通过 LDA，我们就能够用主题来描述文本的特征。

3.5 算法 14: k -means 算法

把相似的数据汇总为簇的方法叫作聚类。

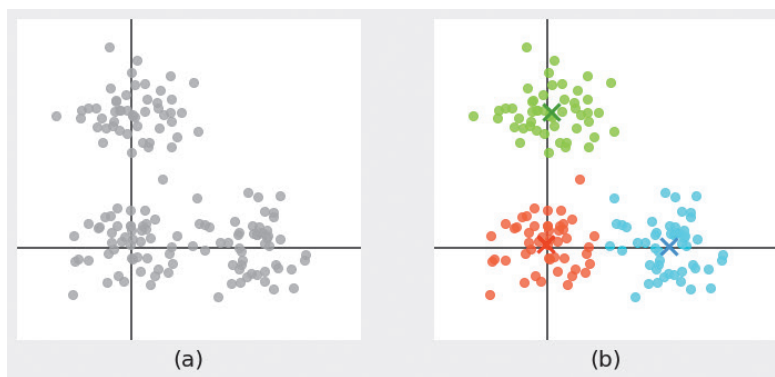
k -means 算法是一种聚类算法，该算法非常简单，所以被广泛应用于数据分析。

概述

k -means 算法是一种有代表性的聚类算法。由于该算法简单易懂，又可以用于比较大的数据集，所以在市场分析和计算机视觉等领域得到了广泛的应用。

我们来看一下 k -means 算法是如何进行聚类的。例如，将 k -means 算法应用于人工数据集，并把各个数据点分为 3 个簇，如图 3-17 所示。图 3-17b 中的 $\times$ 表示的点是各个簇的重心，是 k -means 算法中簇的代表点。

通过计算数据点与各重心的距离，找出离得最近的簇的重心，可以确定数据点所属的簇。求簇的重心是 k -means 算法中重要的计算。

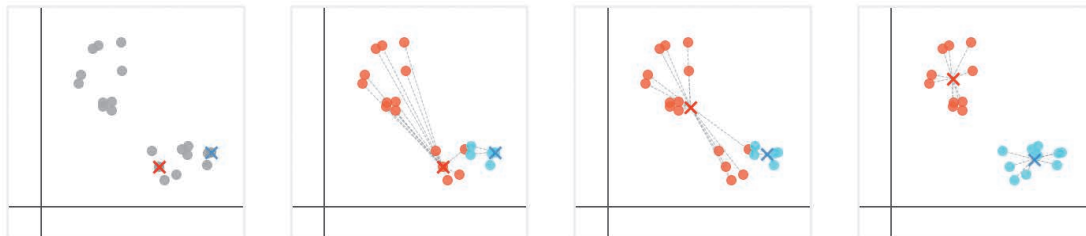


▲ 图 3-17 对二维数据 (a) 应用 k -means 算法的结果 (b)

算法说明

k -means 算法的典型计算步骤如下（图 3-18）。

1. 从数据点中随机选择数量与簇的数量相同的数据点，作为这些簇的重心。
2. 计算数据点与各重心之间的距离，并将最近的重心所在的簇作为该数据点所属的簇。
3. 计算每个簇的数据点的平均值，并将其作为新的重心。
4. 重复步骤 2 和步骤 3，继续计算，直到所有数据点不改变所属的簇，或者达到最大计算步数。



▲图 3-18 k -means 算法的计算步骤

步骤 1 中的簇的数量是一个超参数，需要在训练时设置。对于有些数据集，我们可能很难决定要设置的簇的数量，在这种情况下，可以使用“详细说明”部分介绍的 **Elbow 方法**（肘方法）等技巧。

另外，有时选择的重心不好可能会导致步骤 2 和步骤 3 的训练无法顺利进行。比如在随机选择的重心之间太近等的情况下，就会出现这种问题。利用 k -means++ 等方法，选择位置尽可能远离的数据点作为重心的初始值，就可以解决这个问题。

■ 示例代码

下面是对鸢尾花数据集应用 k -means 算法的代码，簇的数量设置为 3。

▼ 示例代码

```
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
data = load_iris()
n_clusters = 3 # 将簇的数量设置为3
model = KMeans(n_clusters=n_clusters)
model.fit(data.data)
print(model.labels_) # 各数据点所属的簇
print(model.cluster_centers_) # 通过fit()计算得到的重心
```


图 3-19a 给人的印象是数据集划分得比较好，每个簇的区域清楚地分开了。不过从图形给人的印象来判断分析结果是一种定性的判断。另外，对于高维空间的数据，有可能难以进行可视化判断。因此，我们需要某种用于评估聚类结果是好是坏的定量的标准。

聚类结果的好坏可以通过计算簇内平方和（Within-Cluster Sum of Squares, WCSS）来定量评估（这个 WCSS 随着簇的数量的增加而变小，所以可以用于相同数量的簇的情况下的比较）。

WCSS 指的是对所有簇计算其所属的数据点与簇的重心之间距离的平方和，并将它们相加得到的值。这个值越小，说明聚类结果越好。

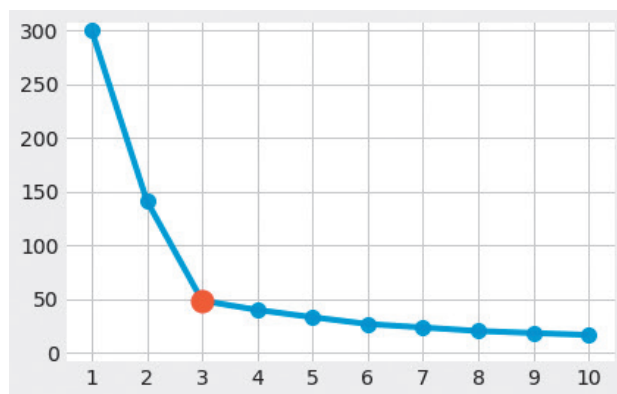
簇的重心与簇所属的数据点之间的距离越小，即数据点越在靠近簇的重心的地方集聚，WCSS 越小。

图 3-19a 的 WCSS 为 50.1，图 3-19b 的 WCSS 为 124.5，因此可以得出左图的聚类结果更好的结论。

■ 使用 Elbow 方法确定簇的数量

在使用 k -means 算法时，首先要确定的就是簇的数量这个超参数，但有时我们并不知道应该将簇的值设置为多少合适。确定合理的簇的数量的一个方法是 Elbow 方法。我们已经知道，随着簇的数量增加，WCSS 会变小，但有时 WCSS 的变大幅度会从簇的数量为某个值时开始放缓。

在图 3-20 中，在簇的数量增加到 3 的过程中，WCSS 明显变小，但当簇的数量逐渐增加为 4、5……时，可以看到 WCSS 变小的幅度变缓了。Elbow 方法将图形中看起来像手臂弯曲时的肘部的点作为簇的数量。在这个例子中，我们将簇的数量设置为 3 似乎很合适。



▲ 图 3-20 基于 Elbow 方法确定簇的数量

当没有很明确的理由来确定簇的数量或对数据知之甚少时，Elbow 方法很有用。不过在实际的分析中，常常不会出现图中那样明显的“肘部”，所以 Elbow 方法的结果只不过是一种参考而已。

参考文献

Kodinariya, T. M., Makwana P. R. Review on determining number of cluster in k -means clustering [J]. Int. J. 1(6), 1990, 90-95.

3.6 算法 15：混合高斯分布

在机器学习和统计学中常用的高斯分布，可以表示一组数据。

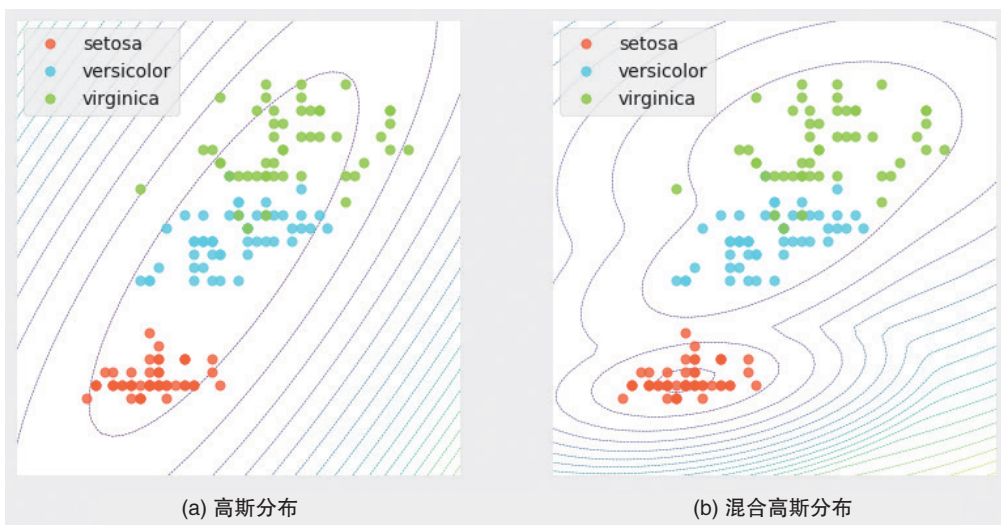
如果数据集中有多组数据，可以使用混合高斯分布（即多个高斯分布的线性组合）来实现聚类。

概述

与其说混合高斯分布是一种算法的名称，不如说它是一种概率分布的名称。本节我们来介绍对于被看作由这种概率分布产生的数据，估计其分布的参数的方法。混合高斯分布可以用于聚类。

高斯分布是混合高斯分布的基础，是统计学和机器学习中经常使用的一种概率分布。数据的分布可以用均值和方差表示，均值描述数据的中心位置，方差描述数据的离散程度。混合高斯分布是以多个高斯分布的线性叠加来表示数据的模型。

下面来看一下高斯分布和混合高斯分布的区别。图 3-21 所示为对鸢尾花数据分别应用高斯分布和混合高斯分布后的结果，每种分布情况在图中以等高线表示。



▲图 3-21 对鸢尾花数据分别应用高斯分布 (a) 和混合高斯分布 (b) 的结果

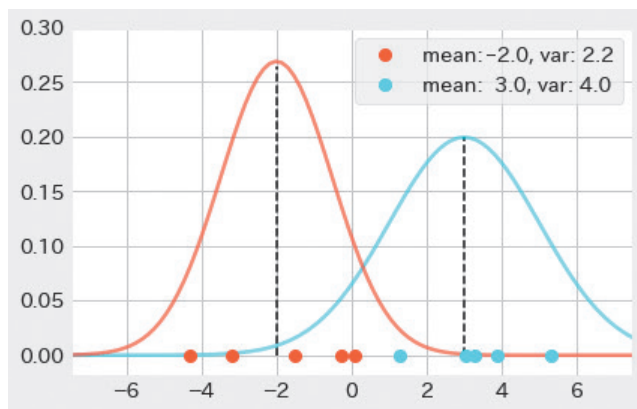
观察图 3-21a，可以知道整个鸢尾花数据每个轴的均值和方差。不过鸢尾花数据包括 setosa（山

鸢尾)、versicolor (杂色鸢尾) 和 virginica (维吉尼亚鸢尾) 3 个品种, 而高斯分布的每个轴都只有一个均值和一个方差, 所以使用一个分布无法体现不同品种之间的差异。

图 3-21b 是应用了混合高斯分布的结果, 它是由 3 个高斯分布叠加而成的。混合高斯分布能够表示由多个类别 (在本例中为品种) 组成的复杂数据。

▶ 算法说明

混合高斯分布的学习过程是从给定的数据点中找到每个高斯分布的均值和方差的过程。简单起见, 这里以在一维数据上求出这些参数的方法为例进行说明。图 3-22 是两个高斯分布及从这两个高斯分布采样得到的一维数据的示意图。红色的点是从均值为 -2.0 、方差为 2.2 的高斯分布中采样得到的数据; 蓝色的点是从均值为 3.0 、方差为 4.0 的高斯分布中采样得到的数据。

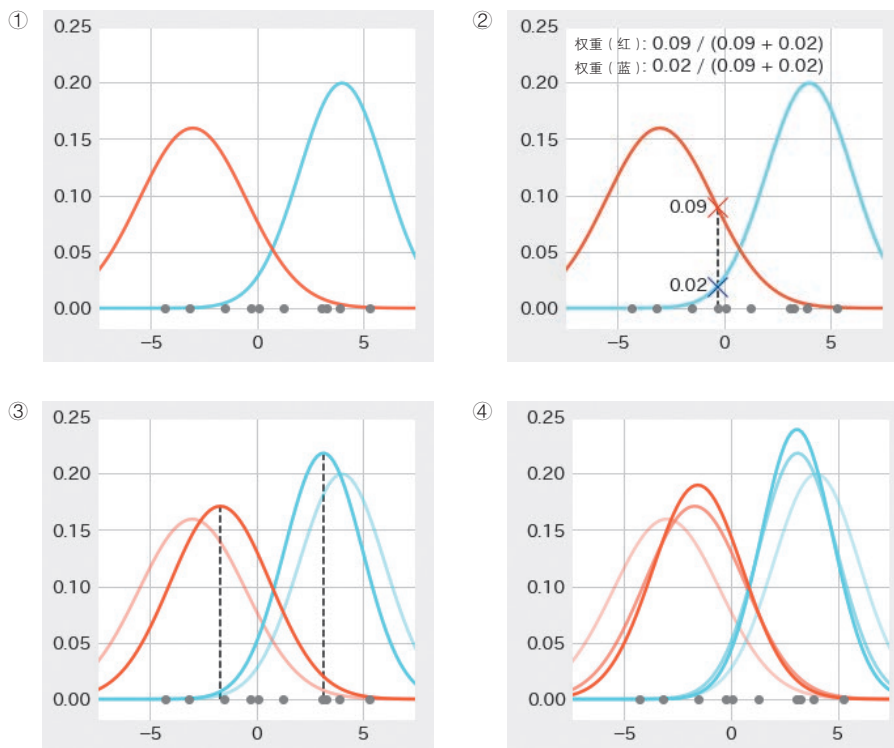


▲ 图 3-22 两个高斯分布及从这两个高斯分布中采样得到的一维数据

现在要解决的混合高斯分布的问题是从数据点中找到高斯分布的参数。如果是如图 3-22 所示的“每个数据点所属的类可以通过颜色来区别”的情况, 问题就简单了, 因为只需计算每种颜色的数据点的均值和方差即可。然而, 混合高斯分布必须在不知道每个数据点的类别的情况下求出参数。因此, 我们需要在推测“每个数据点属于某个类别”的权重的基础上, 计算出数据点的各个类别的高斯分布的参数 (均值和方差)。具体来说, 要通过如图 3-23 所示的步骤求得参数。

1. 初始化参数 (各高斯分布的均值和方差)。
2. 对每个类别计算数据点的权重。

3. 根据步骤 2 中计算出的权重重新计算参数。
4. 重复步骤 2 和步骤 3，直到通过步骤 3 更新前后的每个均值的变化足够小。



▲图 3-23 对一维数据应用混合高斯分布

在步骤 1 中初始化参数，确定每个高斯分布的均值和方差。在步骤 2 中计算数据点拥有的权重，权重计算方法为“每个高斯分布的值 / 所有高斯分布的值之和”。当计算出所有数据点的权重后，求出每个类别的权重的均值，然后计算方差。方差是均值与数据点之间的差的平方的均值，不过这里计算的是均值与数据点之间的差的平方的加权平均值。根据得到的均值和方差，在步骤 3 中计算一个新的高斯分布。然后，依次重复步骤 2 和步骤 3 来更新参数。这种方法的特点是，它并不明确数据点所属的具体是哪一个类别，而是用权重来表示所属类别，并一点一点地更新均值和方差。

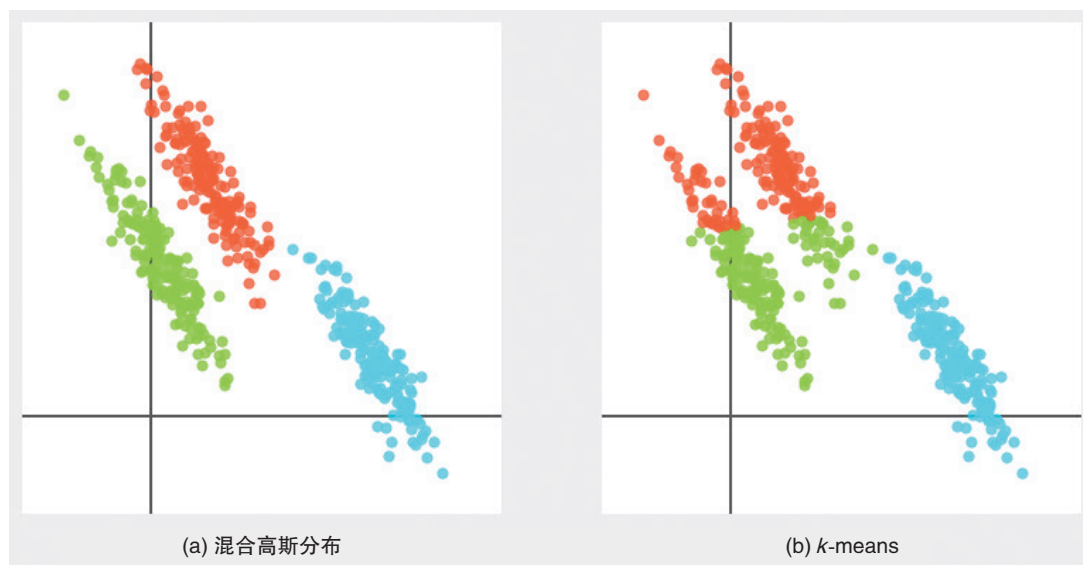
■ 示例代码

下面是对包含 3 个品种的鸢尾花数据应用混合高斯分布的示例代码。

详细说明

使用混合高斯分布实现聚类

这里尝试使用混合高斯分布对以下数据进行聚类。图 3-24 所示为对同一数据集分别应用混合高斯分布和 k -means 算法的聚类结果。



▲ 图 3-24 分别应用混合高斯分布 (a) 和 k -means 算法 (b) 后的聚类结果

由于混合高斯分布中的每个高斯分布对呈椭圆形分布的数据有效，所以混合高斯分布对这个数据集的聚类效果很好。而 k -means 算法对从重心开始呈圆形分布的数据有效，所以它对这个数据集的部分数据的聚类效果不好。

参考文献

scikit-learn. sklearn.mixture.GaussianMixture [EB/OL]. [2019-2-15].

3.7 算法 16: LLE

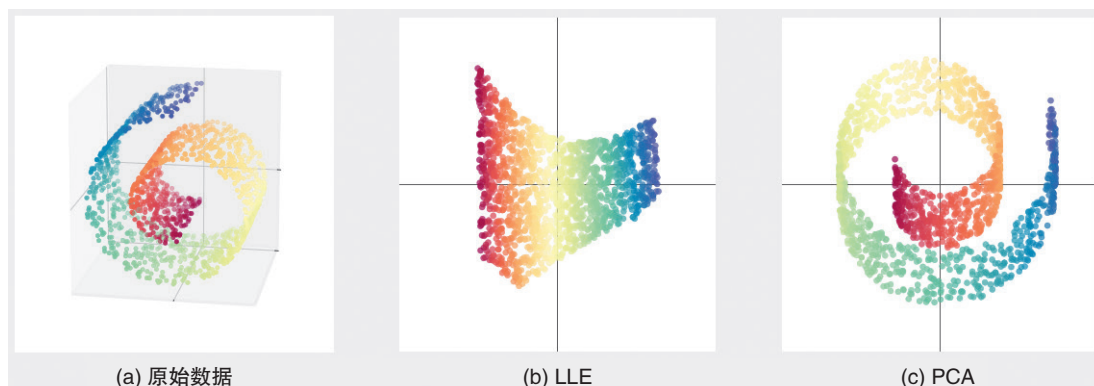
无监督学习中的一项重要任务是将结构复杂的数据转化为更简单的形式。

LLE (Locally Linear Embedding, 局部线性嵌入) 可以将以弯曲或扭曲的状态埋藏在高维空间中的结构简单地在低维空间中。

概述

LLE 是一种被称为**流形学习** (manifold learning) 的算法, 它的目标是对具有非线性结构的数据进行降维。下面来看一下具体的例子。

图 3-25a 是经常作为非线性数据的例子的瑞士卷 (Swiss rolls) 数据集的图形。瑞士卷数据集就像一个二维结构 (长方形) 被卷曲后埋藏在三维空间中的数据集。我们将 LLE 和 PCA 应用于该数据集, 降维至二维后的结果分别如图 3-25b 和图 3-25c 所示。LLE 可以把埋藏在三维空间中的二维结构取出, 将其以二维数据表示。PCA 则采取像是将原始数据压扁的方式进行降维。由于 PCA 适用于变量之间存在相关性的数据, 所以在对瑞士卷数据集这种非线性数据降维时, 更适合采用 LLE 这样的方法。

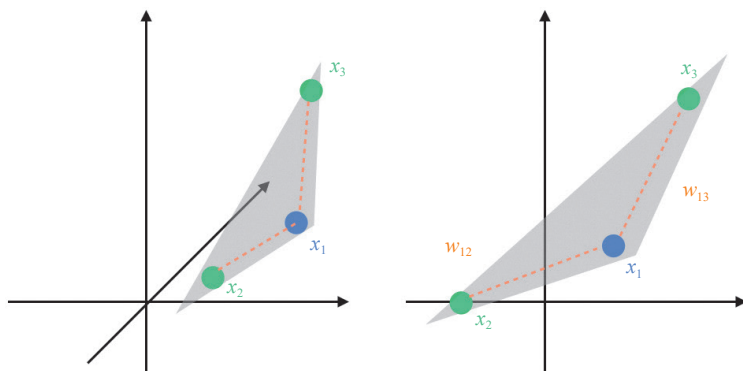


▲ 图 3-25 瑞士卷数据 (a)、应用 LLE 的结果 (近邻点的数量: 12) (b) 和应用 PCA 的结果 (c)

▶ 算法说明

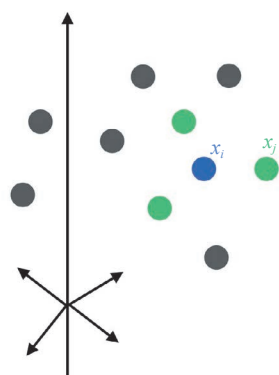
LLE 算法要求数据点由其近邻点的线性组合来表示。我们看一个例子：对于数据点 x_1 ，以最接近 x_1 的两个点 x_2 和 x_3 的线性组合来表示它。图 3-26 中的左图是当 $x_1=(1, 1, 1)$, $x_2=(-1, 0, -1)$, $x_3=(2, 3, 2)$ 时的情况，如果 x_2 和 x_3 的权重分别为 $w_{12}=-1/3$, $w_{13}=1/3$ ，那么 x_1 可以表示为 $x_1=w_{12}x_2 + w_{13}x_3$ 。如果在不扭转三维空间的 3 个点之间的关系的前提下把它们移动到二维空间，我们就可以继续使用这些权重来以近邻点表示 x_1 。

实际的数据虽然在三维空间上呈现瑞士卷那样的卷曲结构，但 LLE 将其视为局部的点与点的关系，即近邻点之间是不弯曲的空间，所以仍使用近邻点来表示数据点。

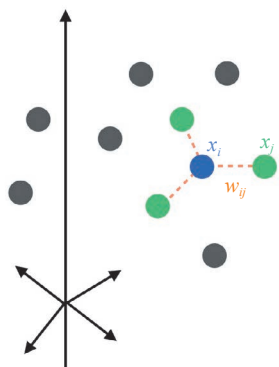


▲ 图 3-26 近邻点的示意图

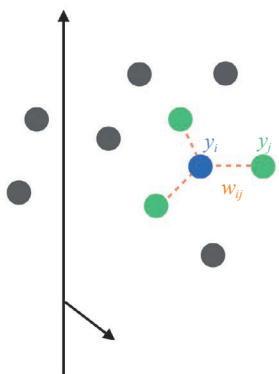
下面对 LLE 算法进行说明。假设在 D 维空间内存在一个 d 维空间的结构 ($d < D$)，使用图 3-27 所示的步骤进行降维。



1. 找到数据点 x_i 的近邻点 (k 个)。



2. 求出能用 k 个近邻点的线性组合表示 x_i 的权重 w_{ij} 。



3. 使用权重 w_{ij} 计算低维 (d 维) 的 y_i 。

▲图 3-27 LLE 算法示意图

考虑数据点 x_i 的近邻点是 LLE 的特点。

在确定近邻点数量后，首先，为了求出权重 w_{ij} ，我们将 x_i 和“ x_i 的近邻点的线性组合”的误差表示为 $x_i - \sum_j w_{ij} x_j$ 。

随着 w_{ij} 值的变化，这个误差会增大或减小。

通过计算所有 x_i 和线性组合的差的平方和，我们将权重 w_{ij} 与误差之间的关系表示为以下误差函数：

$$\varepsilon(W) = \sum_i |x_i - \sum_j w_{ij} x_j|^2$$

除了 w_{ij} 近邻点之外，表达式的值都是 0。另外还有一个约束条件：对于某个 i ，有 $\sum_j w_{ij} = 1$ 。我们可以认为权重 w_{ij} 表示数据点 x_i 及其近邻点之间的关系，而且这种关系在低维空间中也得以保持。在计算完权重后，我们计算在低维空间中表示数据点的 y_i ：

$$\Phi(y) = \sum_i |y_i - \sum_j w_{ij} y_j|^2$$

前面求出了使误差最小的权重 w_{ij} ，现在要做的是利用刚刚求出的 w_{ij} ，求使误差最小的 y 。

■ 示例代码

下面是对瑞士卷数据集应用 LLE 的代码，其中设置的近邻点的数量为 12。

▼ 示例代码

```
from sklearn.datasets import samples_generator
from sklearn.manifold import LocallyLinearEmbedding
data, color = samples_generator.make_swiss_roll(n_samples=1500)
n_neighbors = 12 # 近邻点的数量
n_components = 2 # 降维后的维度
model = LocallyLinearEmbedding(n_neighbors=n_neighbors,
                               n_components=n_components)
model.fit(data)
print(model.transform(data)) # 变换后的数据
```

```
[[ 0.03417034  0.01889039]
 [ 0.03160555  0.05350996]
 [ 0.03590683  0.04619116]
```

```
...
```

```
[ 0.01255618 -0.02078975]
```

```
[ 0.02722956  0.00697508]
```

```
[ 0.02950613 -0.0050912 ]]
```

详细说明

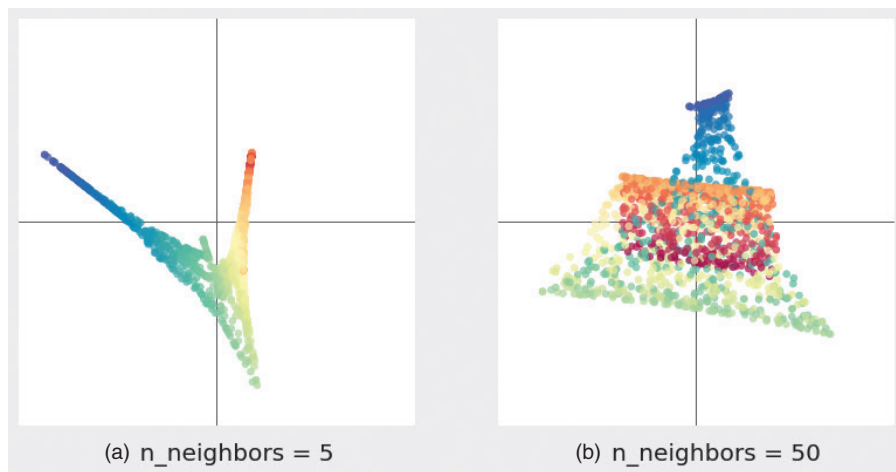
关于流形学习

LLE 和下一节将介绍的 t -SNE 等针对非线性数据的降维算法被统称为流形学习。流形指的是可以将局部看作没有弯曲的空间。类似地，地球虽然是球体，但是我们可以对地球的局部绘制平面地图。关于流体本身的详细定义已超出本书的范围，所以这里不再详细展开。在阅读本书的过程中，只要理解“所谓的流形就是从局部来看是低维空间的结构被埋藏在高维空间里”就足够了。通过以 LLE 为首的各种各样的流形学习算法，可以找到（在弯曲或扭曲的状态下）埋藏在高维空间中的低维的数据结构。

近邻点的数量

LLE 的近邻点数量需要被定义为超参数。

在对图 3-25a 所示的瑞士卷数据应用 LLE 时，将近邻点的数量分别设置为 5 和 50，结果如图 3-28 所示。当近邻点的数量被设置为 5（图 3-28a）时，LLE 没能取出连贯的结构，所以降维后的点的分布显得不是很宽，看上去聚集在一个狭窄的区域，这并不能反映太多的信息。当近邻点的数量被设置为 50（图 3-28b）时，不同颜色的点离得很近。这是由于近邻点的数量太多，无法把握局部的结构。因为近邻点的数量对 LLE 结果影响较大，所以必须慎重设置。



▲图 3-28 不同的近邻点数量导致 LLE 结果不同 ((a) 的近邻点数量为 5, (b) 的近邻点数量为 50)

参考文献

J.A. Lee, M. Verleysen. Nonlinear dimensionality reduction of data manifolds with essential loops [J]. Neurocomputing, 2005, 67:29-53.

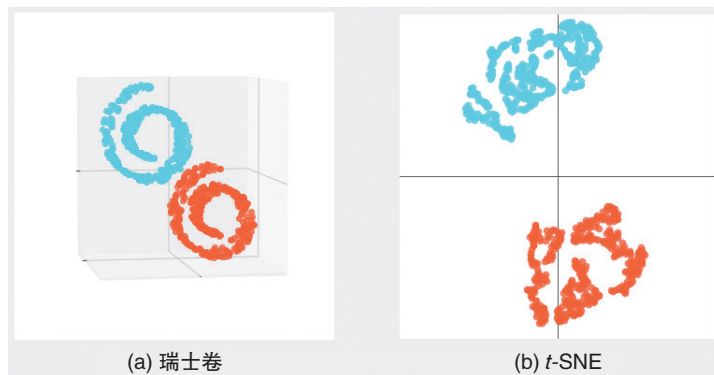
3.8 算法 17: t -SNE

t -SNE (t -Distributed Stochastic Neighbor Embedding, t 分布随机邻域嵌入) 是一种将高维的复杂数据降维为二维 (或三维) 的算法, 用于低维空间的可视化。

在降维时, t -SNE 会将类似结构的数据聚集在一起, 这有助于我们理解数据的结构。

概述

t -SNE 是一种流形学习算法, 用于实现复杂数据的可视化。我们可以通过将高维数据降维为二维或三维来实现可视化。图 3-29 所示为利用 t -SNE 将数据从三维空间降维到二维空间的示例。原始数据是三维空间上的两个瑞士卷数据 (图 3-29a), 二者的区别可以从经过降维后显示在二维空间中的图形看出 (图 3-29b)。



▲图 3-29 使用 t -SNE 降维

如上所示, t -SNE 具有在低维空间巧妙表现多个结构的机制。

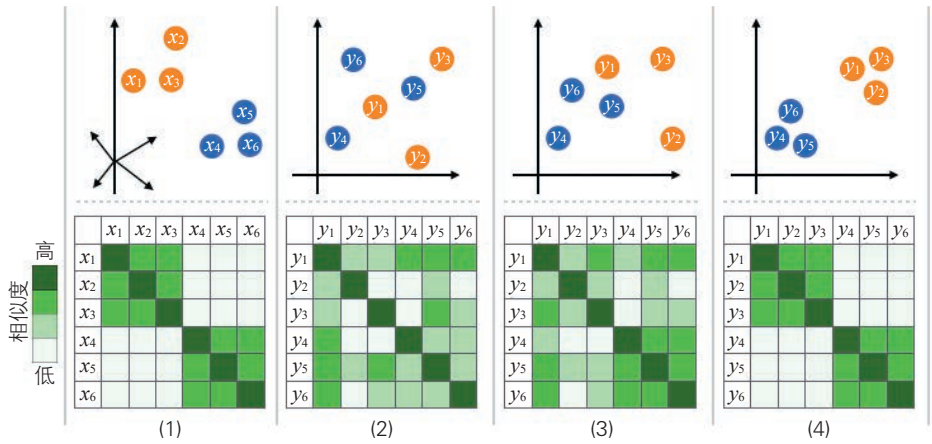
t -SNE 的特点是在降维时使用了自由度为 1 的 t 分布。通过 t 分布, 可以使在高维空间中原本很近的结构在低维空间中变得更近, 原本较远的结构变得更远。下面的“算法说明”及其后的内容将对此进行详细介绍。

算法说明

t -SNE 算法的步骤如下。

1. 对于所有的组 $i、j$ ，使用高斯分布来表示 x_i 和 x_j 的相似度。
2. 在低维空间中随机配置与 x_i 相同数量的点 y_i ，对于所有的组 $i、j$ ，使用 t 分布表示 y_i 和 y_j 的相似度。
3. 更新数据点 y_i ，使得步骤 1 和步骤 2 中定义的相似度分布尽可能相似。
4. 重复步骤 3，直到达到收敛条件。

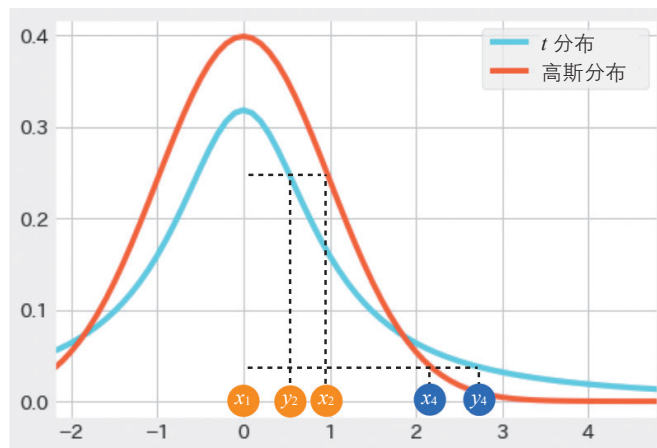
下面介绍步骤 1 和步骤 2 中出现的相似度。相似度是衡量数据点之间的相似程度的概念。它不是简单地使用数据之间的距离，而是使用如图 3-30 所示的概率分布来衡量的。



▲ 图 3-30 t -SNE 算法示意图

在图 3-31 中，横轴是距离，纵轴是相似度。从图中可以看出，数据之间的距离越近，相似度越高；距离越远，相似度越低。我们首先在原来的高维空间中用高斯分布计算相似度，以 p_{ij} 这个分布表示。这个 p_{ij} 表示数据点 x_i 和 x_j 之间的相似度。

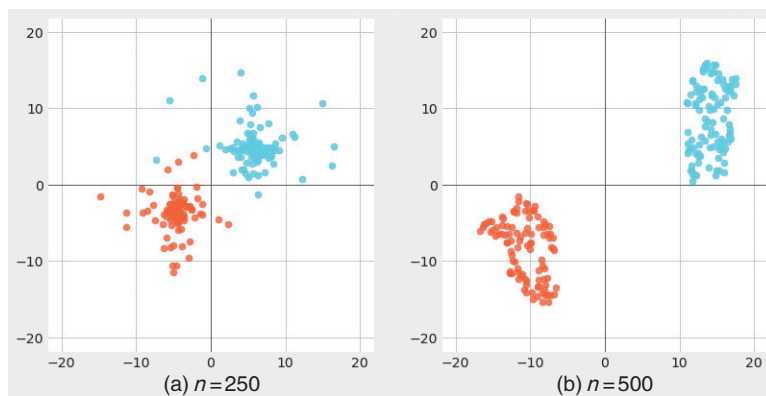
接下来，在低维空间中随机配置与 x_i 对应的数据点 y_i 。我们也对这个数据点计算表示相似度的 q_{ij} ，不过这时使用的是 t 分布。



▲ 图 3-31 t 分布和高斯分布

在计算出 p_{ij} 和 q_{ij} 之后，我们来更新数据点 y_i ，使 q_{ij} 具有与 p_{ij} 相同的分布，这样就能够以低维空间的 y_i 再现高维空间中各 x_i 的相似度的关系。由于这时在低维空间使用的是 t 分布，所以可以看到，当再现大的相似度时，数据点在低维空间中配置的距离更近；反之，当再现小的相似度时，数据点在低维空间中配置的距离更远。

在将 t -SNE 应用于前面提到的瑞士卷数据时，数据点 y_i 的更新情况如图 3-32 所示。从图中可以看出，随着更新次数的增加，数据点的差异逐渐展现了出来。



▲ 图 3-32 数据点的更新情况 (a) 中的更新次数为 250 次, (b) 中为 500 次

“概述”部分提到 t -SNE 基本上被用于将数据降维到三维或二维的场景。由于 t 分布是重尾分布，所以在高维空间中，远离中心的区域占主导地位，局部信息将无法保留。因此，有时无法降维到四维或更高维度的空间。

示例代码

下面是将 t -SNE 应用于手写数字数据集的代码，其中设置的降维后的维度是 2。

▼ 示例代码

```
from sklearn.manifold import TSNE
from sklearn.datasets import load_digits
data = load_digits()
n_components = 2 # 设置降维后的维度为2
model = TSNE(n_components=n_components)
print(model.fit_transform(data.data))
```

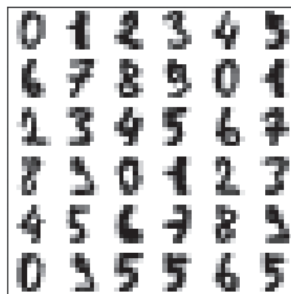
```
[[ 12.328249   64.76087 ]
 [-16.284025  -24.134592 ]
 [-14.58529   -0.6316269]
 ~略~
 [ -4.541453   -6.5503755]
 [ 26.231192   12.501517 ]
 [ -0.6332957  -1.1858237]]
```

详细说明

与其他降维算法的比较

下面将 t -SNE 与其他降维算法进行比较，来研究它的特点。我们使用的数据是如图 3-33 所示的手写数字。

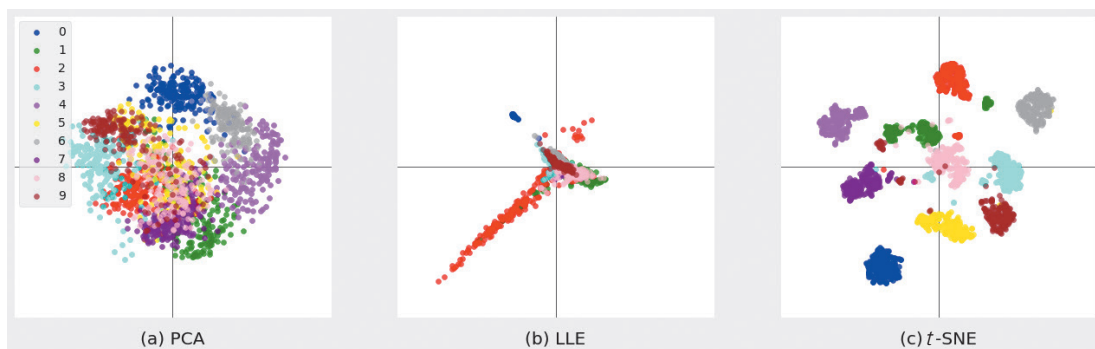
手写数字是 8 像素 × 8 像素的图像数据，每个图像中的数字是



▲ 图 3-33 手写数字

0, 1, 2, ..., 9 其中之一。换言之，我们可以认为它是一个包含了 10 种不同结构的 $8 \times 8 (= 64)$ 维空间。

下面使用 PCA、LLE 和 t -SNE 这 3 种算法对手写数字数据进行降维（图 3-34）。



▲图 3-34 PCA、LLE 和 t -SNE 的比较

从图中可以看出，图 3-34a 中的 PCA 虽然在一定程度上按数值进行了归类，但每部分依然混杂着不同的数值；图 3-34b 中的 LLE 虽然适用于非线性数据，但如果数据集不像瑞士卷数据集那样数据点聚在一起，我们就不能很好地把握其结构；而图 3-34c 中的 t -SNE 在二维空间中将数据按值归类，很好地对结构完成了分类。

参考文献

van der Maaten, L. & Hinton, G. E.. Visualizing data using t -SNE [J]. J. Mach. Learn. Research 9, 2008, 2579-2605.

scikit-learn. 2.2. Manifold learning [EB/OL]. [2019-2-7].

scikit-learn. Sklearn.manifold.TSNE [EB/OL]. [2019-2-7].

第 4 章

评估方法和各种数据的处理

到这里为止，我们已经介绍了各种算法。接下来，让我们再来了解一些实践性的内容。本章将介绍有监督学习的评估方法、提高机器学习性能的方法，以及解决过拟合的对策。本章的后半部分将介绍如何处理文本数据和图像数据。

4.1 评估方法

► 有监督学习的评估

如 1.2 节所述，有监督学习有多种评估模型的指标。第 2 章和第 3 章介绍了许多机器学习算法，我们现在应该已经知道了每种算法的特点，以及如何为数据集选择合适的算法。本节将梳理一下有监督学习的常见的评估方法、提高机器学习性能的方法，以及提高性能时的障碍。

在评估有监督学习的分类问题和回归问题时，所需的指标不同（表 4-1）。

▼表 4-1 本章介绍的指标

分类问题	回归问题
混淆矩阵	均方误差
正确率	决定系数
精确率	
召回率	
F 值	
AUC	

下面就让我们看一下各个指标的含义，学会为要解决的问题选择合适的指标。

在介绍如何评估有监督学习之后，本节还将介绍防止过拟合的方法，以及选择超参数的方法。过拟合是使用机器学习时绕不开的问题。即使选择了正确的评估指标，但是如果在训练数据上发生了过拟合，也无法创建一个好的模型。有几种方法可以防止过拟合，本节将重点介绍一种叫作交叉验证的技术。

► 分类问题的评估方法

下面我们来看一下分类问题的评估方法。

1.2 节中有一段使用逻辑回归模型对美国威斯康星州乳腺癌数据集进行机器学习的代码。

大家可以回到 1.2 节看一下数据的详细信息。

需要注意的是，这里把目标变量（类别标签数据）的 0 和 1 反转了。这是为了将恶性视为 1（阳性），将良性视为 0（阴性）。在 scikit-learn 中，标签往往没有意义。这个数据本来是将恶性视为 0 的，但是在实际的诊疗中，一般将发现恶性肿瘤作为检查目标，所以就这个问题来说，把恶性当作阳性来处理更加自然。

▼ 示例代码

```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
X = data.data
y = 1 - data.target
# 反转标签的0和1
X = X[:, :10]
from sklearn.linear_model import LogisticRegression
model_lor = LogisticRegression()
model_lor.fit(X, y)
y_pred = model_lor.predict(X)
```

■ 混淆矩阵

首先要介绍的分类问题评估指标是**混淆矩阵**（confusion matrix）。混淆矩阵可以将分类结果以表格的形式汇总，这样就可以检查哪些标签分类正确，哪些标签分类错误。我们可以通过 scikit-learn 的 confusion_matrix 函数创建混淆矩阵。

▼ 示例代码

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y, y_pred)
print(cm)
```

```
[[341  16]
 [ 36 176]]
```

将这个二元分类的结果作为混淆矩阵输出，我们会得到一个 2 行 2 列的矩阵。如表 4-2 所示，

它是一个真实数据（正确答案）和预测数据的矩阵。

▼表 4-2 混淆矩阵

		预测	
		0	1
真实数据	0	<i>TN</i>	<i>FP</i>
	1	<i>FN</i>	<i>TP</i>

- 左上角的 *TN* (True Negative) 指的是阴性数据实际被正确预测为阴性的情况（正确判断为良性）。
- 右上角的 *FP* (False Positive) 指的是阴性数据实际被错误地预测为阳性的情况（良性被判断为恶性）。
- 左下角的 *FN* (False Negative) 指的是阳性数据实际被错误地预测为阴性的情况（恶性被判断为良性）。
- 右下角的 *TP* (True Positive) 指的是阳性数据实际被正确预测为阳性的情况（正确判断为恶性）。

结合输出的数据可知，*TN* 有 341 条，*TP* 有 176 条，*FP* 有 16 条，*FN* 有 36 条。正确预测的 *TN*、*TP* 的值较大，总体良好。但是，有 36 条 *FN* 出现，说明有 36 个恶性患者被漏诊。再看一下相反的情况，*FP* 有 16 条，说明模型将 16 条良性患者预测为恶性了。

这一点可以通过观察后面即将介绍的召回率（recall）再次得到证实。如果想避免恶性患者被漏诊，可以通过后面即将介绍的预测概率来调整预测。

混淆矩阵由 4 个数值组成，作为评估指标使用可能让人比较难以理解。因此，有时我们会使用混淆矩阵中的元素计算其他数值，并将其作为评估指标。经常使用的指标有以下几个。

- 正确率：
$$\frac{TP + TN}{TP + TN + FP + FN}$$
- 精确率：
$$\frac{TP}{TP + FP}$$
- 召回率：
$$\frac{TP}{TP + FN}$$
- *F* 值： $2 \times (\text{精确率} \times \text{召回率}) / (\text{精确率} + \text{召回率})$

具体使用哪些指标取决于使用目的。为了正确判断对给定的问题使用哪个指标，我们需要了解每个指标的含义。下面来详细看一下各个指标。

正确率

正确率（accuracy）指的是预测正确的结果占总预测结果的比例。accuracy_score 函数用于计算正确率。

▼ 示例代码

```
from sklearn.metrics import accuracy_score
accuracy_score(y, y_pred)
```

```
0.9086115992970123
```

这里输出的是基于“作为实际答案的目标变量 y”和“使用学习后的模型预测的 y_pred”计算出来的正确率。

正确率超过了 90%，看上去模型正确地学习了数据。

精确率

精确率（precision）指的是在所有被预测为阳性的数据中，被正确预测为阳性的数据所占的比例。precision_score 函数用于计算精确率。

▼ 示例代码

```
from sklearn.metrics import precision_score
precision_score(y, y_pred)
```

```
0.9166666666666666
```

与正确率的计算一样，精确率是基于 y 和 y_pred 计算出来的。在这个问题中，它表示预测为恶性的病例中实际确实为恶性的病例所占的比例。精确率低，意味着预测为恶性的病例之中包含了其实是良性的病例。在这种情况下，可以通过再次检查等方式解决精确率低的问题。

召回率

召回率指的是在实际为阳性的数据中，被正确预测为阳性的数据所占的比例。recall_score 函数用于计算召回率。

▼ 示例代码

```
from sklearn.metrics import recall_score  
recall_score(y, y_pred)
```

0.8301886792452831

与正确率的计算一样，召回率是基于 y 和 y_pred 计算出来的。在这个问题中，它表示实际为恶性的病例中被正确地预测为恶性的病例所占的比例。召回率低，意味着实际为恶性的病例大部分被预测为良性。与精确率低的情况相比，召回率低的问题更严重。在使用这样的模型解决实际问题之前，需要想办法提高召回率。后面的“预测概率”部分将介绍如何提高召回率。

■ F 值

F 值是综合反映精确率和召回率两个趋势的指标。 $f1_score$ 函数用于计算 F 值。

▼ 示例代码

```
from sklearn.metrics import f1_score  
f1_score(y, y_pred)
```

0.8712871287128713

精确率和召回率之间是此消彼长的关系，如果试图让其中一个的值更高，则会导致另一个的值更低。如果这两个指标同等重要，可以观察 F 值。

■ 预测概率

到目前为止，我们都是使用 `predict` 进行标签预测的。接下来我们看一下预测概率。在进行二元分类时，预测结果必须为 0 或者 1 这两个值中的一个。然而，实际情况是大多数模型预测的是数据被分类为某一个标签的概率。也就是说，我们可以分别求出数据被分类为 0 或 1 的概率。

我们可以使用学习后的模型的 `predict_proba` 方法查看预测概率。

▼ 示例代码

```
model_lor.predict_proba(X)
```

```
array([[4.41813058e-03, 9.95581869e-01],
       [4.87318118e-04, 9.99512682e-01],
       [3.31064277e-04, 9.99668936e-01],
       ...,
       [2.62819353e-02, 9.73718065e-01],
       [5.09374706e-06, 9.99994906e-01],
       [9.74068776e-01, 2.59312242e-02]])
```

上面的代码输出的是特征 x 中每个数据被预测为 0 或 1 的概率。以第一行为例，数据被分类为 0 的概率约为 0.44%，所以预测其为恶性（1）。相反，最后一行的数据被分类为 0 的概率约为 97.4%，所以预测其为良性（0）。

另外，scikit-learn 的 `predict` 方法使用的是基于阈值 0.5 输出的判断结果。

为了尽量避免漏掉恶性病例，我们考虑预测概率的第 2 个元素（即恶性的概率）为 10%（0.1）或更大的情况。

▼ 示例代码

```
import numpy as np
y_pred2 = (model_lor.predict_proba(X)[:, 1]>0.1).astype(np.int)
print(confusion_matrix(y, y_pred2))
```

```
[[259  98]
 [  2 210]]
```

在输出混淆矩阵后，和我们设想的一样，左下角的 FN （实际为 1 却被错误地预测为 0 的数据）减少了。

这里再看一下其他指标。

▼ 示例代码

```
print(accuracy_score(y, y_pred2))
print(recall_score(y, y_pred2))
```

```
0.8242530755711776
```

```
0.9905660377358491
```

正确率下降到 0.82 左右，但是恶性（1）病例的召回率非常高，达到了 0.99 左右。

ROC 曲线与 AUC

现在我们已经了解了正确率和 F 值等指标，但实际上在有些情况下，这些指标并不能很好地发挥作用。这些情况发生在阳性数据的数量和阴性数据的数量有很大偏差的时候。假设有一个总是预测输出结果为阳性的模型。由于对任何输入都预测数据是阳性的，所以我们可能会以为这个模型的正确率非常低。但在输入数据有偏差的情况下，比如阳性数据有 95 个，阴性数据有 5 个的情况下，正确率会高达 95%。

虽然这个模型是一个极端的例子，但是在实际工作中，这种没有经过很好训练的模型因数据不均衡而计算出高正确率结果的情况经常出现。

应对数据不均衡问题的指标有 AUC （Area Under the Curve，曲线下面积）。 AUC 指的是 ROC（Receiver Operating Characteristic，接收器操作特性）曲线下的面积。这里的 ROC 曲线（图 4-1）指的是横轴为假阳性率（即 FP 的占比），纵轴为真阳性率（即 TP 的占比）的图形。图中显示了当预测数据为阳性的预测概率的阈值从 1 开始逐渐下降时， FP 和 TP 之间关系的变化。

假阳性率、真阳性率是可视化 ROC 曲线的必要条件，可以使用 `roc_curve` 函数计算。

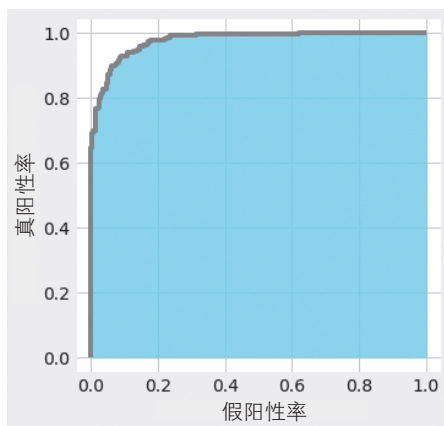
▼ 示例代码

```
from sklearn.metrics import roc_curve
probas = model_lor.predict_proba(X)
fpr, tpr, thresholds = roc_curve(y, probas[:, 1])
```

`roc_curve` 函数的输入是目标变量（类别标签数据）和预测概率。这里使用 `predict_proba` 方法计算了预测概率。下面在 Matplotlib 中对 `roc_curve` 函数输出的 `fpr` 和 `tpr` 进行可视化。

▼ 示例代码

```
%matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('fivethirtyeight')
fig, ax = plt.subplots()
fig.set_size_inches(4.8, 5)
ax.step(fpr, tpr, 'gray')
ax.fill_between(fpr, tpr, 0, color='skyblue', alpha=0.8)
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_facecolor('xkcd:white')
plt.show()
```



▲ 图 4-1 ROC 曲线

ROC 曲线下方的面积是 AUC 。面积的最大值是 1，最小值是 0。 AUC 越接近于 1（面积越大），说明精度越高；如果其值在 0.5 左右，则说明预测不够理想。换言之，如果值在 0.5 左右，则得到的分类模型和抛硬币随机决定良性恶性没多大区别。

`roc_auc_score` 函数用于计算 AUC 值。

▼ 示例代码

```
from sklearn.metrics import roc_auc_score
roc_auc_score(y, probas[:, 1])
```

0.9767322023148881

AUC 的结果约为 0.977，接近于 1。对于一个分类模型来说，这样的精度是很高的。

由于乳腺癌的数据偏差不大，所以用正确率来考查模型的性能也没有太大问题。不过如果输入数据为关于“观看网站广告的用户中有多少会购买商品”的预测数据，那么正例和反例的数量可能有相当大的偏差，常常出现正确率为 0.99，但 *AUC* 只有 0.6 的情况。在处理不平衡数据时，我们使用 *AUC* 作为指标。

 回归问题的评估方法

接下来介绍回归问题的评估方法。由于回归问题的目标是预测大小关系有意义的值，所以有些评估方法和分类问题的评估方法不同。这里使用的数据是美国波士顿的房价数据集。这个数据集有 13 个特征变量，目标变量是 5.0 ~ 50.0 的数值。简单起见，这里介绍一元回归的评估方法，所以我们只使用 13 个特征变量中的“住宅平均房间数”（列名为 RM）。

▼ 示例代码

```
from sklearn.datasets import load_boston
data = load_boston()
X = data.data[:, [5,]]
y = data.target
```

这里设从数据集中的特征变量选择的平均房间数的列为 x ，目标变量为 y 。虽然特征变量 x 是由一列组成的数据，但这里也按照惯例使用大写的 x 。目标变量 y 是数值数据。 x 是 506 行 1 列的数据， y 也是 506 行 1 列的数据。

▼ 示例代码

```
from sklearn.linear_model import LinearRegression
model_lir = LinearRegression()
model_lir.fit(X, y)
y_pred = model_lir.predict(X)
```

上面的代码导入并使用了 `LinearRegression` 类，然后初始化 `model_lir`，使用 `fit` 方法进行训练。接着，代码使用训练好的 `model_lir` 的 `predict` 方法进行预测，并将预测结果赋给变量 `y_pred`。

本次使用的 `LinearRegression` 是一种线性回归算法。另外，由于特征变量是 1 列数据，所以模型可以使用线性方程 $y = ax + b$ 表示。现在看一下斜率 a 和截距 b 的值。

▼ 示例代码

```
print(model_lir.coef_)
print(model_lir.intercept_)
```

```
array([9.10210898])
```

```
-34.67062077643857
```

斜率 a 约为 9.10，截距 b 约为 -34.67，所以本次训练的模型可以表示为直线 $y = 9.10x - 34.67$ 。当房间数为 5 时，将其代入该式，有 $9.10 \times 5 - 34.67$ ，可以得到预测租金为 10.83。线性回归利用训练数据确定学习参数 a 和 b 。随着训练数据的增减，这些学习参数会变为不同的值。

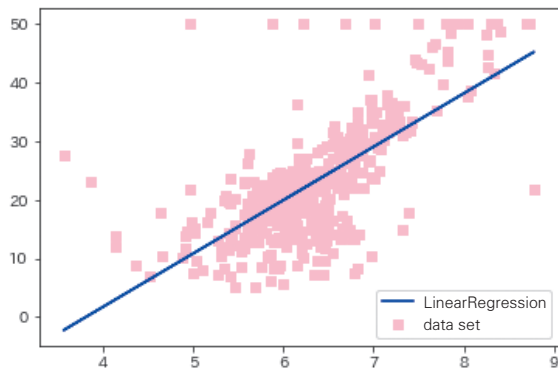
接下来绘制预测结果的图形，来看一下训练好的模型是如何预测的。由于这次是基于一个特征变量（平均房间数）对目标变量（租金）进行预测的，所以我们可以绘制一个横轴为平均房间数，纵轴为租金的图形。

下面使用 Matplotlib 查看数据。

▼ 示例代码

```
%matplotlib inline
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.scatter(X, y, color='pink', marker='s', label='data set')
ax.plot(X, y_pred, color='blue', label='LinearRegression')
ax.legend()
plt.show()
```

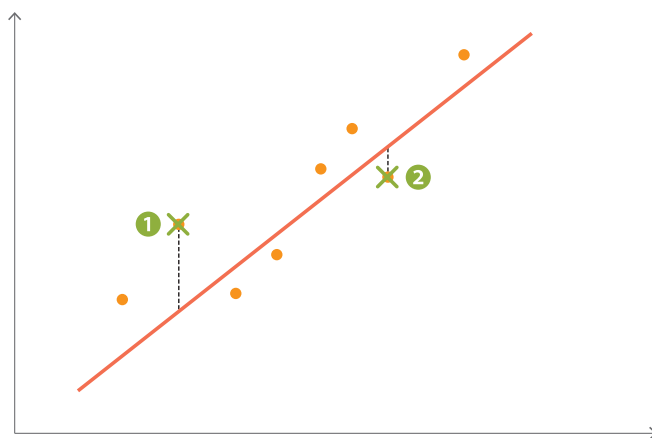
图 4-2 中数据集的颜色是粉红色。从粉红色的数据来看，图形的右上角包含了离群值（在图形顶部绘制的纵轴值为 50 的数据）。蓝线代表预测结果，与数据基本一致。这是一个预测房价会随着房间数量的增加而上涨的一次函数。通过可视化，我们能够大致了解学习结果。接下来使用均方误差和决定系数这两个指标来量化学习结果。



▲ 图 4-2 基于平均房间数预测租金

■ 均方误差

平方误差是一个表示实际值和预测值之间存在多大差异的数值。下面结合例子来说明，请看图 4-3。



▲图 4-3 平方误差的示意图

图中的直线表示预测值，直线上有以 × 标记的点①和②。从每个点出发，分别朝着表示预测值的直线画一条线段作为辅助线，辅助线的长度就是误差。

从图中可以看出，①比②离预测值更远。也就是说，②更接近预测值，误差更小。对于要评估的数据，计算所有预测值与数据之间的误差的平方，并取平方的平均值，得到的就是均方误差 (Mean-Square Error, MSE)。换言之，均方误差越小，预测效果越好。我们可以使用 scikit-learn 提供的 `mean_squared_error` 函数计算均方误差。

▼ 示例代码

```
from sklearn.metrics import mean_squared_error
mean_squared_error(y, y_pred)
```

43.60055177116956

均方误差约为 43.6。

■ 决定系数

决定系数 (coefficient of determination) 是使用均方误差来表示训练好的模型的预测效果的数值，也就是被称为 R^2 的系数。

当该系数取最大值 1.0 时，说明没有误差。它的值通常在 0.0 和 1.0 之间，但如果预测的误差过

大，也可能为负值。换言之，该系数的值越接近 1.0，说明模型对数据点的解释能力越强。我们可以使用 `r2_score` 函数计算决定系数。

▼ 示例代码

```
from sklearn.metrics import r2_score
r2_score(y, y_pred)
```

0.48352545599133423

决定系数约为 0.484。

均方误差和决定系数指标的不同

前面解释了如何使用均方误差和决定系数两个指标评估回归问题。光看均方误差的数值不能判断精度是好还是坏。如果目标变量的方差较大，均方误差也会变大。而决定系数可以使用不依赖于目标变量方差的取值范围在 0.0 和 1.0 之间的值表示，所以即使目标变量的数量级不同，也可以将决定系数作为一般的指标使用。

与其他算法进行比较

前面使用 `LinearRegression` 介绍了均方误差和决定系数，我们再来看一下使用其他算法的情况。下面使用 `SVR` 进行回归，并与使用 `LinearRegression` 时的情况进行比较。

`SVR` 是将第 2 章介绍的支持向量机（核方法）应用于回归而得到的算法。

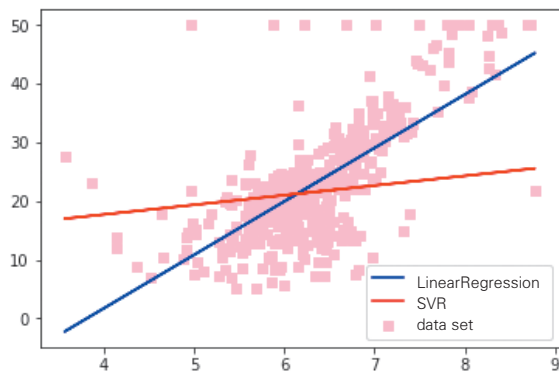
▼ 示例代码

```
from sklearn.svm import SVR
model_svr_linear = SVR(C=0.01, kernel='linear')
model_svr_linear.fit(X, y)
y_svr_pred = model_svr_linear.predict(X)
```

上面的代码导入了 `SVR` 类，用于训练和预测。下面在图 4-4 中查看使用 `LinearRegression` 和 `SVR` 时的学习结果。

▼ 示例代码

```
%matplotlib inline
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.scatter(X, y, color='pink', marker='s', label='data set')
ax.plot(X, y_pred, color='blue', label='LinearRegression')
ax.plot(X, y_svr_pred, color='red', label='SVR')
ax.legend()
plt.show()
```



▲ 图 4-4 基于平均房间数预测的租金以及回归直线

蓝线表示 LinearRegression，红线表示 SVR。从图中可以看出，SVR 的线和数据不太一致。下面看一下均方误差和决定系数的值。

▼ 示例代码

```
print(mean_squared_error(y, y_svr_pred)) # 均方误差
print(r2_score(y, y_svr_pred)) # 决定系数
print(model_svr_linear.coef_) # 斜率
print(model_svr_linear.intercept_) # 截距
```

```
72.14197118147209
0.14543531775956597
[[1.64398]]
11.13520958
```

结果依次是均方误差、决定系数、斜率和截距。LinearRegression 的均方误差和决定系数分别约为 43.6 和 0.484，与之相比，SVR 的均方误差和决定系数都变差了，分别约为 72.1 和 0.145。经过比较，我们发现 LinearRegression 的各个指标的值都更好。

通过均方误差和决定系数，我们能够对模型进行定量评估。这个结果可能会给人以 SVR 不是一个好算法的印象。其实通过改变 SVR 的 C 和 kernel 参数，就可以改善 SVR 的均方误差和决定系数。

超参数的设置

下面将初始化 SVR 时的参数变更为 C=1.0，kernel='rbf'。

▼ 示例代码

```
model_svr_rbf = SVR(C=1.0, kernel='rbf')
model_svr_rbf.fit(X, y)
y_svr_pred = model_svr_rbf.predict(X)
print(mean_squared_error(y, y_svr_pred)) # 均方误差
print(r2_score(y, y_svr_pred)) # 决定系数
```

```
36.42126375260171
0.5685684051071418
```

可以看出，均方误差和决定系数都得到了改善。C 和 kernel 是 SVR 的超参数。model_svr_rbf.coef_ 和 model_svr_rbf.intercept_ 等学习参数是由机器学习算法更新的，而超参数需要在训练开始前由用户给出。因此，如果超参数设置得不好，模型的性能就可能会很差。

模型的过拟合

下面来看一下模型的过拟合。

使用以下代码将数据集分为训练数据和用来确认性能的验证数据，然后进行 SVR 的训练和预测。

▼ 示例代码

```
train_X, test_X = X[:400], X[400:]
train_y, test_y = y[:400], y[400:]
model_svr_rbf_1 = SVR(C=1.0, kernel='rbf')
model_svr_rbf_1.fit(train_X, train_y)
test_y_pred = model_svr_rbf_1.predict(test_X)
print(mean_squared_error(test_y, test_y_pred)) # 均方误差
print(r2_score(test_y, test_y_pred)) # 决定系数
```

69.16928620453004

-1.4478345530124388

虽然超参数没有变过，但模型在验证数据上的表现远差于在训练数据上的表现。这种对训练数据的预测效果很好，但对验证数据（没有用于训练的数据）的预测效果不好的现象叫作**过拟合**。

在有监督学习中，防止过拟合是一个重要课题。仅仅考查之前介绍的各指标，并不足以判断模型的好坏。重要的是在解决实际问题时，模型对未知数据的预测精度。模型对这种未知数据的预测能力叫作**泛化能力**。即使模型对训练数据的均方误差很小，如果发生过拟合，泛化能力也会很低。

过拟合和超参数的设置是分类问题和回归问题的共同挑战。接下来本书将讨论如何应对这些挑战。

防止过拟合的方法

有监督学习的特征值和目标变量是作为训练数据预先给出的。前面介绍了对训练数据的性能评估方法。但在使用有监督学习解决实际问题时，除了评估模型对训练数据的性能之外，评估模型对不包括在训练数据中的数据（未知数据）的性能也是非常重要的。以乳腺癌数据集为例，“患者的

身体数据”（特征）和“恶性/良性”（目标变量）是训练数据。

在实际应用中，对于“恶性/良性”不明的患者，重要的是能否通过患者的体检数据预测出“恶性/良性”。一个模型如果对训练数据的预测精度很高，但对未知数据不能进行很好的预测，那就不能说它是一个好模型。防止过拟合的方法有几种，以下是一些有代表性的方法。

将数据分为训练数据和验证数据

防止过拟合的一个代表性的方法是将数据分为训练数据和验证数据。换言之，这种方法不使用事先给定的所有数据进行训练，而是留出一部分数据用于验证，不用于训练。

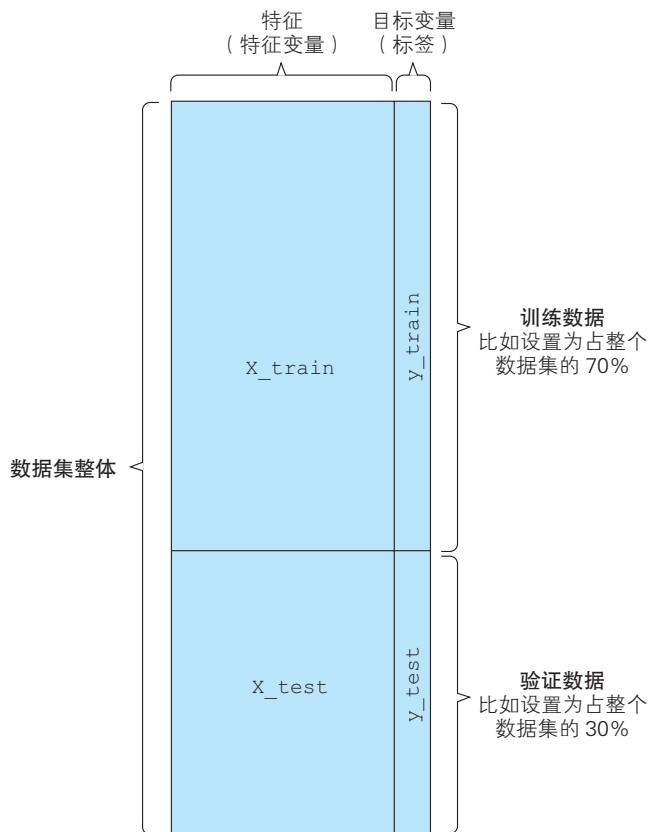
使用 `scikit-learn` 的 `train_test_split` 函数，我们可以很容易地分割数据。

▼ 示例代码

```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
X = data.data
y = data.target
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

- 训练用的特征值: `X_train`
- 验证用的特征值: `X_test`
- 训练用的目标变量: `y_train`
- 验证用的目标变量: `y_test`

我们将数据分为训练数据和验证数据，其中 70% 用于训练，30% 用于验证（图 4-5）。这个分割比例设为多少是没有明确规定的。如果数据集很大，有足够的用于训练，将分割比例设置为 6:4 也是可行的；反之，如果数据集太小，不能很好地进行训练，可以将分割比例设置为 8:2 等。另外要注意的是，每次运行时 `train_test_split` 的结果都是不同的，如果想保持结果固定，需要设置 `random_state` 参数。



▲图 4-5 按 7 : 3 将数据分割为训练数据和验证数据的例子

下面使用训练数据和验证数据来进行算法的学习，并创建模型。

▼示例代码

```
from sklearn.svm import SVC
model_svc = SVC()
model_svc.fit(X_train, y_train)
y_train_pred = model_svc.predict(X_train)
y_test_pred = model_svc.predict(X_test)
from sklearn.metrics import accuracy_score
print(accuracy_score(y_train, y_train_pred))
print(accuracy_score(y_test, y_test_pred))
```

```
1.0
```

```
0.6023391812865497
```

如果与对训练数据的正确率相比，对验证数据的正确率要低很多，就说明数据发生了过拟合。上面代码中的模型对未知数据的正确率约为 60%。下面使用另一个模型 `RandomForestClassifier` 来试一下。

▼ 示例代码

```
from sklearn.ensemble import RandomForestClassifier
model_rfc = RandomForestClassifier()
model_rfc.fit(X_train, y_train)
y_train_pred = model_rfc.predict(X_train)
y_test_pred = model_rfc.predict(X_test)
from sklearn.metrics import accuracy_score
print(accuracy_score(y_train, y_train_pred))
print(accuracy_score(y_test, y_test_pred))
```

```
0.9974874371859297
```

```
0.9590643274853801
```

这次虽然对验证数据的正确率依然比对训练数据的低，却是约为 96% 的高正确率。由于模型对验证数据的正确率也很高，所以可以说防止了过拟合。

从这些模型的结果来看，也许我们应该使用 `RandomForestClassifier`。在选择模型时，如果没有分割数据，只看对训练数据的正确率，我们可能会选择 `SVC`。通过观察对验证数据的正确率，我们能够避免使用出现了过拟合的模型。

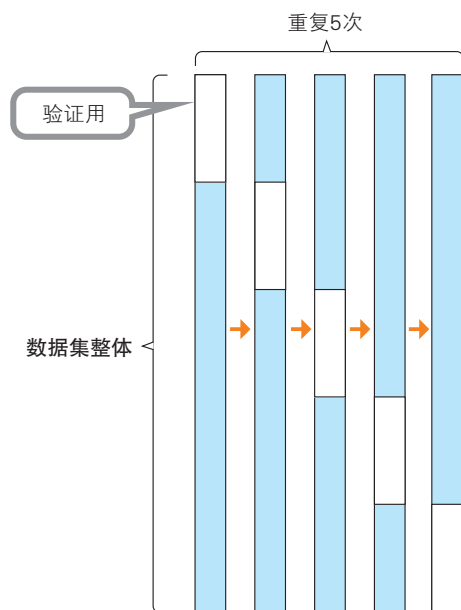
交叉验证

即使在将数据分为训练数据和验证数据后进行评估，也依然可能发生过拟合。可以想到的原因

是使用的训练数据和验证数据碰巧非常相似。反过来也有可能出现训练数据和验证数据非常不相似的情况。为了避免这种数据分割的误差，可以使用不同的分割方案进行多次验证，这就是所谓的交叉验证（cross validation）。

本节以将数据分割 5 次，其中 80% 的数据用于训练，20% 的数据用于验证的情况为例进行说明。

如图 4-6 所示，每次获取不同的 20% 的数据作为验证数据，重复 5 次。在这个例子中，20% 的数据是按分组顺序分别分割的，但在实际应用中，作为验证数据的 20% 的数据是随机抽取的。



▲图 4-6 交叉验证

以下代码非常轻松地将数据分成了 5 块，即运行 5 次，每次留下 20% 的数据用于训练后的验证。

▼示例代码

```
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import KFold
cv = KFold(5, shuffle=True)
model_rfc_1 = RandomForestClassifier()
cross_val_score(model_rfc_1, X, y, cv=cv, scoring='accuracy')
```

```
array([0.99122807, 0.92982456, 0.94736842, 0.96491228, 0.92920354])
```

这时正确率会被输出 5 次。我们可以看到正确率有时很高，有时很低。在选择模型时，需要考虑所有正确率的均值和方差。

另外，我们也可以输出 F 值的评估结果。

通过将 `cross_val_score` 函数的 `scoring` 参数定义为 `f1`，就可以输出 F 值，代码如下所示。

▼ 示例代码

```
cross_val_score(model_rfc_1, X, y, cv=cv, scoring="f1")
```

```
array([0.99280576, 0.93846154, 0.97902098, 0.97297297, 0.97435897])
```

► 搜索超参数

前面介绍了如何使用分割得到的数据来选择不会出现过拟合的模型。如果在此基础上仔细地选择超参数，就可以进一步提高模型的性能。就像前面在“超参数的设置”部分介绍的那样，通过反复设置一个超参数并检查其性能，最终可以得到更好的超参数。但是多个超参数的组合数量非常多，逐一设置每个超参数的过程非常耗时。

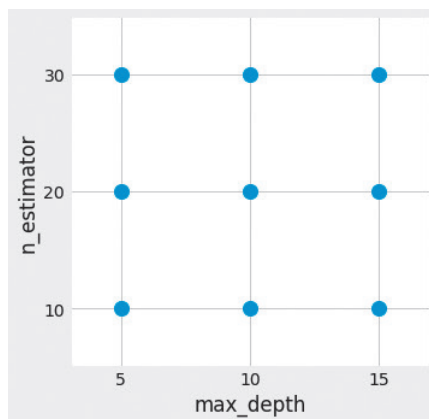
本节将介绍搜索超参数的方法。

► 使用网格搜索选择超参数

网格搜索是一种自动搜索超参数的方法。如图 4-7 所示，这是一种对各个超参数组合进行穷尽搜索的方法。需要注意的是，要搜索的超参数必须事先确定。

下面是使用 `scikit-learn` 的 `GridSearchCV` 进行 `RandomForestClassifier` 超参数搜索的示例代码。`GridSearchCV` 一边关注对验证数据的性能，一边执行超参数的搜索。

首先加载数据。这里要做的是对分类任务进行网格



▲ 图 4-7 网格搜索的示意图

搜索，所以要重新加载美国威斯康星州乳腺癌数据集。

▼ 示例代码

```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
X = data.data
y = 1 - data.target
# 反转标签的0和1
X = X[:, :10]
```

接下来进行网格搜索。

▼ 示例代码

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import KFold
cv = KFold(5, shuffle=True)
param_grid = {'max_depth': [5, 10, 15], 'n_estimators': [10, 20, 30]}
model_rfc_2 = RandomForestClassifier()
grid_search = GridSearchCV(model_rfc_2, param_grid, cv=cv, scoring='accuracy')
grid_search.fit(X, y)
```

上面的代码为 `max_depth` 准备了 3 个值，为 `n_estimators` 也准备了 3 个值，对二者所有的组合，即 $3 \times 3 = 9$ 种情况进行了评估。下面输出所得到的最好的得分及相应的超参数值。

▼ 示例代码

```
print(grid_search.best_score_)
print(grid_search.best_params_)
```

```
0.9490333919156415
```

```
{'max_depth': 10, 'n_estimators': 10}
```

除了交叉验证外，GridSearchCV 还支持使用 F 值进行评估。修改 GridSearchCV 的评估方法很简单，将 scoring 参数指定为 f1 即可。

▼ 示例代码

```
grid_search = GridSearchCV(model_rfc_2, param_grid, cv=cv, scoring='f1')
```

专栏 防止过拟合的各种方法

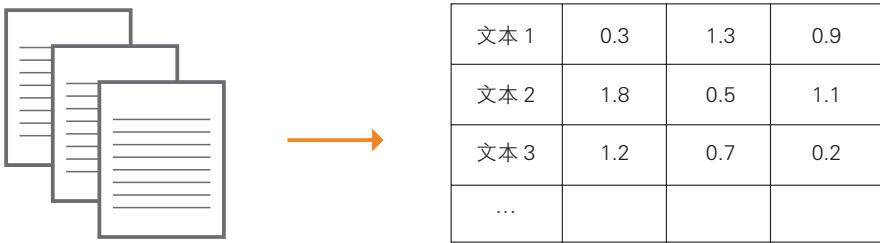
前面介绍了通过调整超参数等来防止过拟合的方法。除此之外，还有其他一些方法可以防止过拟合。下面列出防止过拟合的主要方法的名称。在应对过拟合时，也可以考虑这些方法。

- 增加训练数据
- 减少特征值
- 正则化
- Early Stopping
- 集成学习

4.2 文本数据的转换处理

本书之前涉及的机器学习模型的输入都是元素为数值的表格形式的数据。然而，在自然语言处理领域，我们需要处理不能直接作为输入的文本数据。

本节将介绍两种将文本数据转换为表格数据的方法：一种是基于单词出现次数的转换；另一种是基于 tf-idf 的转换（图 4-8）。然后，我们将对转换后的表格形式的数据应用机器学习模型，并比较它们的结果。



▲图 4-8 文本数据的转换

基于单词出现次数的转换

下面介绍基于单词出现次数的转换方法。这是一种通过统计文本中的单词出现次数将文本数据转化为表格形式的数据的方法。

我们看一下对下列文本数据进行转换的例子。

“This is my car”

“This is my friend”

“This is my English book”

表 4-3 统计了每个文本中各个单词出现的次数。表中的行是文本，列是每个文本中各个单词出现的次数。如果单词在文本中没有出现，则相应单元格的值为 0。通过这个例子，我们就能理解文本数据是如何转换为表格形式的数据的。

▼表 4-3 文本数据中单词出现的次数

	This	is	my	car	friend	English	book
文本 1	1	1	1	1	0	0	0
文本 2	1	1	1	0	1	0	0
文本 3	1	1	1	0	0	1	1

通过分析单词出现的次数，我们可以描述出文本的特点。仔细观察各个单词，可以发现每个文本中都会出现一些常见的单词，也有一些文本特有的单词。比如 This 这个单词是每个文本中都会出现的常见词，但 car 只在文本 1 中出现，所以可以说它是描述文本的一个重要的单词。

只考虑单词出现次数的做法无须考虑这些单词的重要度，对所有单词都平等地进行计数。下面将介绍 tf-idf 方法是如何考虑并表示单词的重要度的。

基于 tf-idf 的转换

tf-idf 是一种基于 tf (term frequency, 词频) 和 idf (inverse document frequency, 逆文本频率指数) 这两个指标来表示单词在文本中的重要度的方法。

tf 是单词在文本中出现的频率，idf 是一个包含该单词的文本越多，值就越小的值。换言之，像 This 这种出现在许多文本中的单词的 idf 就很小。这两个指标相乘得到的结果叫作 tf-idf。表 4-4 是对刚才的文本数据计算的 tf-idf 的结果。

▼表 4-4 文本数据的 tf-idf 表示

	This	is	my	car	friend	English	book
文本 1	0.41	0.41	0.41	0.70	0.00	0.00	0.00
文本 2	0.41	0.41	0.41	0.00	0.70	0.00	0.00
文本 3	0.34	0.34	0.34	0.00	0.00	0.57	0.57

This 和 my 等单词在每个文本中都出现了，所以它们的 tf-idf 值较小，而 car 和 friend 等只在特定文本中出现的单词的 tf-idf 值则较大。行业术语和专有名词等只在特定文本中出现的词，往往具有较大的 tf-idf 值，有时能够很好地表示包含这些单词的文本。

应用于机器学习模型

下面基于单词出现次数和 tf-idf 将文本数据转换为表格形式的数据，并应用于机器学习模型。scikit-learn 的 CountVectorizer 可以计算单词出现次数，TfidfVectorizer 可以进行 tf-idf 的转换。另外，通过 fetch_20newsgroups 获取使用的文本数据，机器学习模型则采用 LinearSVC。

▼ 示例代码

```
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.datasets import fetch_20newsgroups
categories = ['misc.forsale', 'rec.autos', 'comp.graphics', 'sci.med']
remove = ('headers', 'footers', 'quotes')
twenty_train = fetch_20newsgroups(subset='train',
                                   remove=remove,
                                   categories=categories) # 训练数据
twenty_test = fetch_20newsgroups(subset='test',
                                   remove=remove,
                                   categories=categories) # 验证数据
```

这里使用的是前面出现过的 20 Newsgroups 数据集，代码通过 categories 变量明确指定了 4 个主题的数据。

首先将文本数据转换为单词出现次数，然后使用 LinearSVC 学习和预测。得到的模型对验证数据正确率约为 0.794。也就是说，我们成功地将文本数据转换为了表格形式的数据，并进行了有监督学习。

▼ 示例代码

```
count_vect = CountVectorizer() # 单词出现次数
X_train_counts = count_vect.fit_transform(twenty_train.data)
X_test_count = count_vect.transform(twenty_test.data)
model = LinearSVC()
model.fit(X_train_counts, twenty_train.target)
predicted = model.predict(X_test_count)
np.mean(predicted == twenty_test.target)
```

```
0.7937619350732018
```

接下来使用 tf-idf 进行转换，然后以同样的方式进行训练和预测。此时得到的模型的正确率约为 0.87，比基于单词出现次数进行转换的方法的正确率高。通过 tf-idf，看起来我们很好地抓住了文本数据的特点。

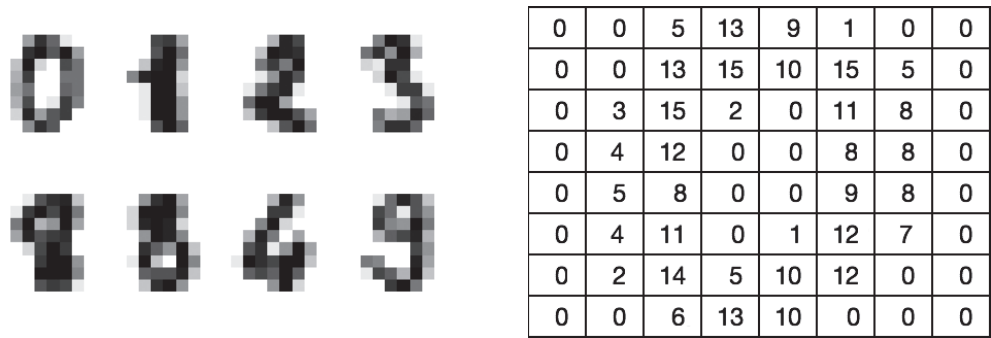
▼ 示例代码

```
tf_vec = TfidfVectorizer() # tf-idf
X_train_tfidf = tf_vec.fit_transform(twenty_train.data)
X_test_tfidf = tf_vec.transform(twenty_test.data)
model = LinearSVC()
model.fit(X_train_tfidf, twenty_train.target)
predicted = model.predict(X_test_tfidf)
np.mean(predicted == twenty_test.target)
```

```
0.8701464035646085
```

4.3 图像数据的转换处理

本节将介绍如何处理才能将图像数据作为机器学习的输入数据。这里以灰度图像数据为例进行说明（图 4-9）。灰度图像指的是图像中的每个像素只表示亮度的图像。大家可以思考一下，要想把这样的图像数据作为输入数据，需要进行什么处理呢？



▲图 4-9 图像数据和表格形式的数据

▶ 直接将像素信息作为数值使用

一种方法是直接将各像素的信息作为数值使用。表 4-5 是将图像数据转换为向量数据的例子。如果是简单的图像识别问题，就可以使用这种简单的转换建立一个具有一定精度的模型。

▼表 4-5 将图像数据转换为向量数据

图像数据	向量数据
	[8, 0, 12, ..., 19]
	[8, 0, 12, ..., 19]
	[8, 0, 12, ..., 19]

虽然通过这种转换，可以创建机器学习模型的输入数据，但由于具有二维关系的像素数据被转换成了一维向量，所以可以说重要的信息被丢弃了。有些模型会在保留图像的二维关系的前提下直接将其作为输入数据进行处理，比如在图像识别中常用的深度学习使用的就是像素的近邻像素的信息。

下面是将图像数据转换为向量数据的示例代码。代码中使用第三方 Python 包 `Pillow` 将图像（png）数据转换为了向量数据。

▼ 示例代码

```
from PIL import Image
import numpy as np
img = Image.open('mlzukan-img.png').convert('L')
width, height = img.size
img_pixels = []
for y in range(height):
    for x in range(width):
        # 通过getpixel获取指定位置的像素值
        img_pixels.append(img.getpixel((x,y)))
print(img_pixels)
```

```
[250, 255, 216, ..., 89]
```

将转换后的向量数据作为输入来应用机器学习模型

前面介绍了如何将图像数据转换为向量数据，接下来我们尝试实际使用转换后的数据来创建机器学习模型。这里我们使用灰度手写数字数据建立一个模型，用于预测从 0 到 9 的 10 个数字。

由于 `scikit-learn` 的 `datasets` 模块能够获取转换后的向量数据，所以我们使用它来获取输入数据。

从预测结果来看，使用了向量数据的模型成功地进行了高精度的预测。

▼ 示例代码

```

from sklearn import datasets
from sklearn import metrics
from sklearn.ensemble import RandomForestClassifier
digits = datasets.load_digits()
n_samples = len(digits.images)
data = digits.images.reshape((n_samples, -1))
model = RandomForestClassifier()
model.fit(data[:n_samples // 2], digits.target[:n_samples // 2])
expected = digits.target[n_samples // 2:]
predicted = model.predict(data[n_samples // 2:])
print(metrics.classification_report(expected, predicted))

```

	precision	recall	f1-score	support
0	0.94	0.99	0.96	88
1	0.90	0.89	0.90	91
2	0.95	0.88	0.92	86
3	0.80	0.88	0.84	91
4	0.95	0.87	0.91	92
5	0.81	0.82	0.82	91
6	0.98	0.98	0.98	91
7	0.95	0.97	0.96	89
8	0.88	0.73	0.80	88
9	0.76	0.86	0.81	92
avg / total	0.890	0.890	0.89	899



第 5 章

环境搭建

本章将介绍搭建 Python 机器学习环境的方法。首先介绍在各种操作系统中安装 Python 的方法，然后介绍创建虚拟环境和安装包的方法。

5.1 Python 3 的安装

Python 有两个主要的版本，它们之间是不兼容的。以往使用的 Python 2 已在 2020 年 1 月停止维护。另外，Python 2 还存在对中文、日文等语言支持得不好和标准库体系不清晰等问题。不过，这些问题在 Python 3 中已经得以解决。

因此，本书使用的是 Python 3。在进行机器学习时，Python 3 基本上不会给我们带来麻烦。但要注意的是，有些操作系统默认安装的是 Python 2，或者老的代码是用 Python 2 编写的。

这里将安装 Python 3.7.2，也就是本书撰写时（2019 年 3 月）的最新版本。下面将分别介绍在 3 种操作系统 Windows、macOS 和 Linux 上的安装方法。本节前半部分介绍的是 PSF（Python Software Foundation，Python 软件基金会）官方发布的标准安装方法，后半部分介绍的是在 Windows 上使用第三方安装程序 Anaconda 进行安装的方法。

Windows

这里使用官方安装程序在 Windows 10 上安装 Python 3.7。Windows 版安装程序可从 Python 官方网站下载。

在官方网站上的 Python 版本中，点击 Windows x86-64 web-based installer，下载 exe 文件。

在打开下载的文件后，会弹出安装对话框。按照安装向导，勾选 Add Python 3.7 to Path 复选框，然后点击 Install Now，即可完成 Python 3.7 的安装。

在命令提示符或 PowerShell 中运行命令 `py`，看到显示了 `>>>`，就说明进入到了 Python 交互模式，这样就可以了。如果要退出交互模式，则键入 `quit()` 或使用快捷键 `Ctrl + Z + Enter`，这样就能回到操作系统的命令输入模式。

macOS

macOS 中默认安装了 Python 2.7（在不久的将来，它可能会被 Python 3.x 所取代）。这是因为 macOS 内置了使用 Python 的工具。我们在与操作系统默认安装的 Python 2.7 共存的前提下安装 Python 3.7。这里使用官方安装程序在 macOS 10.9 及以后的版本上安装 Python 3.7。

macOS 版安装程序同样需要从 Python 官方网站下载。

在官方网站上的 Python 版本中，点击 macOS 64-bit installer，下载 pkg 文件。打开下载的文件

后，会弹出安装对话框。根据界面的说明点击，即可完成 Python 3.7 的安装。

然后输入以下命令，允许 Python 使用 macOS 的 SSL 根证书。

```
$ /Applications/Python\ 3.7/Install\ Certificates.command
```

在终端运行 `python3` 命令，界面显示 `>>>`，进入 Python 交互模式。如果要退出交互模式，则键入 `quit()` 或使用快捷键 `Control + D`，这样就能回到操作系统的命令输入模式。

Linux

许多基于 Linux 的操作系统默认安装了 Python。有的 Linux 发行版安装了 Python 3。下面我们在与操作系统默认安装的 Python 2.7 共存的前提下安装 Python 3.7。

除了构建源代码进行安装的方式之外，还可以使用操作系统的包管理器，比如 `apt` 等，来安装最新版本的 Python。

Linux 版安装程序也从 Python 官方网站下载。

在官方网站上的 Python 版本中，点击 Gzipped source tarball，下载 `tgz` 文件。为了构建 Python，需预先安装以下软件库。

- OpenSSL header 库（Ubuntu 软件包名：`libssl-dev`）
- sqlite header 库（Ubuntu 软件包名：`libsqlite3-dev`）

如果在 Ubuntu 18.4 上安装，需要事先做好以下准备工作，其中包括安装机器学习所需的操作系统库。

```
$ sudo apt install build-essential
$ sudo apt install libssl-dev libsqlite3-dev libbz2-dev
$ sudo apt install libffi-dev zlib1g-dev libreadline-gplv2-dev
$ sudo apt install libxml2-dev libxslt1-dev libjpeg62-dev
$ sudo apt install libblas-dev liblapack-dev gfortran libfreetype6-dev
```

解压已下载的文件，在终端切换到解压后的目录。然后运行以下命令，将 Python 安装到 `/opt/python37` 目录下。

```
$ sudo apt install build-essential
$ ./configure --prefix=/opt/python37
$ make
$ sudo make install
```

在终端运行 `/opt/python37/bin/python3` 命令，看到显示了 `>>>`，就说明进入到了 Python 交互模式，这样就可以了。如果要退出交互模式，则键入 `quit()` 或使用快捷键 `Control + D`，这样就能回到操作系统的命令输入模式。

使用 Anaconda 在 Windows 上安装

Anaconda 是一个一体化的第三方安装程序。一方面，它降低了需要编译的包的导入难度，用它来进行机器学习的开发非常方便。另一方面，它具有很强的独特性，用它进行 Web 应用的开发很麻烦。conda cloud 中缺少许多用于机器学习以外场景的库。另外，还要注意的是，conda install 和 pip install 是相互冲突的，所以混在一起使用会破坏开发环境。

Anaconda 可以从 Anaconda 公司的官方网站下载。点击网站上的 Download 按钮，下载安装程序。下载后按照对话框中的指示安装软件。安装完毕后，在开始菜单选择 Anaconda Prompt，这时系统会打开一个命令提示符，看到显示了 `>>>`，就说明进入到了 Python 交互模式，这样就可以执行 python 命令了。在交互模式键入 `quit()` 或使用快捷键 `Ctrl + Z + Enter`，就能回到操作系统的命令输入模式。

5.2 虚拟环境

一台计算机中可以设置多个 Python 运行环境，也可以允许多个 Python 版本运行。本节将准备一个虚拟环境来管理要安装的库的版本。通过虚拟环境，我们还能统一 Python 和 Jupyter Notebook 的启动命令。这里所说的虚拟环境并不是指 VirtualBox 或者 VMWare 等操作系统的虚拟环境，而是指 Python 运行环境的虚拟环境。假如我们要在同一台计算机上开发 X 和 Y 两个 Python 程序，虽然 X 和 Y 使用同一个库 A，但是 X 使用 A 的 1.0 版本，Y 使用 A 的 1.1 版本，在这种情况下，Python 虚拟环境就能派上用场了。此外，X 和 Y 程序使用的 Python 版本也有可能不同，对于这种情况，虚拟环境也是有用的。

通过官方安装程序安装 Python 的情况

Python 标配了一个叫 `venv` 的模块，推荐通过官方安装程序安装 Python 的用户使用这个模块创建虚拟环境。

Windows 的情况

从命令提示符或 PowerShell 切换到要创建虚拟环境的文件夹，运行下列命令后，Python 将在名为 `env` 的文件夹中创建虚拟环境。

```
> Set-ExecutionPolicy RemoteSigned -Scope CurrentUser  
> py -m venv env
```

运行以下命令进入虚拟环境。

```
> env\Scripts\Activate.ps1  
(env) >
```

完成后，提示符前将出现虚拟环境的文件夹名 `(env) >`。

■ macOS 的情况

从终端切换到要创建虚拟环境的文件夹，运行下列命令后，Python 将在名为 env 的文件夹中创建虚拟环境。

```
$ python3 -m venv env
```

运行以下命令进入虚拟环境。

```
$ source env/bin/activate  
(env) $
```

提示符前将出现虚拟环境的文件夹名 (env) \$。

■ Linux 的情况

从终端切换到要创建虚拟环境的文件夹，运行下列命令后，Python 将在名为 env 的文件夹中创建虚拟环境。

```
$ /opt/python37/bin/python3 -m venv env
```

运行以下命令进入虚拟环境。

```
$ source env/bin/activate  
(env) $
```

提示符前将出现虚拟环境的文件夹名 (env) \$。

■ Python 的启动方法和关闭虚拟环境

在启用虚拟环境后，在命令提示符等中键入 python，即可启动 Python。

```
(env) > python
```

运行 `deactivate` 命令退出虚拟环境。

```
(env) > deactivate  
>
```

当提示符恢复到标准形态，说明已经离开了虚拟环境。

最后介绍一下如何处理不再需要的虚拟环境。当不再需要虚拟环境时，直接删除 `env` 文件夹即可，此时在虚拟环境中另外安装的包等也会被删除。

通过 Anaconda 安装 Python 的情况

如果是通过 Anaconda 安装的 Python，那么许多用于机器学习的库就已经安装好了。在大多数情况下，我们不需要创建虚拟环境和安装新的库。

如果需要在 Anaconda 中创建虚拟环境，可以使用 `conda create` 命令，详细信息请参考 Anaconda 的官方文档。

5.3 第三方包的安装

什么是第三方包

对于 Python 非标配的功能，我们可以通过第三方开发的包扩展 Python。许多包是在 PyPI 网站上发布的。第三方包各式各样，许多是公开的，有的经过了充分的验证，有的需要试用。第三方包中也有用于机器学习和数据分析的经典的包，本书中使用的 scikit-learn 就是其中之一。

安装第三方包的方法

要安装第三方包，可以使用 pip 命令。

如果是通过 Anaconda 安装的 Python，由于 Anaconda 已经内置了许多第三方包，所以有时不需要另行安装。此外，我们也可以使用 Anaconda 的 conda 命令代替 pip 来安装第三方包。

通过官方安装程序安装 Python 的情况

下面安装本书中用到的包，如下所示。

- Jupyter Notebook
- NumPy
- SciPy
- pandas
- Matplotlib
- scikit-learn

需要添加的包的安装命令如下。

```
(env) $ pip install jupyter numpy scipy pandas matplotlib scikit-learn
```

■ 通过 Anaconda 安装 Python 的情况

运行本书范围内的代码是不需要另行安装任何第三方包的。如果需要安装其他包，可使用 `conda install` 命令，详细信息请参考 Anaconda 的官方文档。

图解机器学习算法

附录

数学式的说明

矩阵运算

对于如下所示的矩阵 A 和 B ，分别进行加法和乘法运算。

$$A = \begin{pmatrix} a & b & c \\ d & e & f \end{pmatrix}, B = \begin{pmatrix} g & h & i \\ j & k & l \end{pmatrix}$$

● 矩阵的加法

矩阵的加法运算如下所示。

$$A + B = \begin{pmatrix} a+g & b+h & c+i \\ d+j & e+k & f+l \end{pmatrix}$$

● 矩阵的乘法

矩阵 A 与 B^T 的乘法运算如下所示。 B^T 是 B 的转置。

$$AB^T = \begin{pmatrix} ag+bh+ci & aj+bk+cl \\ dg+eh+fi & dj+ek+fl \end{pmatrix}$$

均值、方差、标准差

● 均值

均值是数据点 x 的值的总和除以数据的数量后得到的值。均值用希腊字母 μ 表示。

$$\mu_x = \frac{1}{n} \sum_{i=1}^n x_i$$

在下面的图 A-1 中，数据点 x 的均值 μ_x 为 5.25；数据点 y 的均值 μ_y 为 4.50。

● 方差、标准差

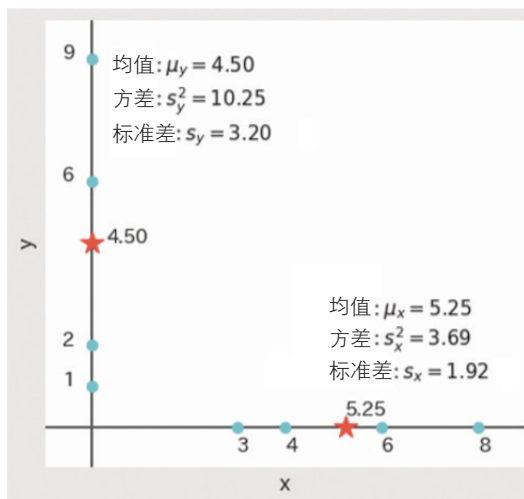
方差是计算数据点 x 的均值与每个数据点之间的差值的平方和的均值而得到的值。方差的平方根是标准差。方差用 s^2 表示，标准差用 s 表示。方差包括有偏样本方差和无偏样本方差，本书在提到方差一词时，指的是有偏样本方差。

方差和标准差表示数据整体与均值之间的偏差情况。

标准差与实际值的单位相同，在有些场景下可以让我们更直观地了解数据的偏差情况。

$$s_x^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu_x)^2$$

图 A-1 中的数据点 x 的方差 s_x^2 为 3.69，标准差 s_x 为 1.92；数据点 y 的方差 s_y^2 为 10.25，标准差 s_y 为 3.20。



▲ 图 A-1 x 和 y 的均值、方差和标准差

■ 协方差、相关系数

● 协方差

协方差是表示两组数据 x 、 y 之间的关系性的值。

首先计算 x 和 y 各自的均值和数据点之间的差的乘积，然后取这些乘积的总和的平均值，得到的就是协方差。

$$s_{xy} = \frac{1}{n} \sum_{i=1}^n (x_i - \mu_x)(y_i - \mu_y)$$

图 A-2 中的数据点 x 和 y 的协方差 s_{xy} 分别为 5.88、-39.50。

从图中可以看出，当数据有向右上延伸的趋势时，协方差为正。反之，当数据有向右下方延伸的趋势时，协方差为负。由于协方差的值取决于数据点的方差（或标准差）的大小，所以协方差

值大并不意味着向右上方延伸的趋势大。如果需要一个不依赖于方差大小的指标，可以使用下面介绍的相关系数。

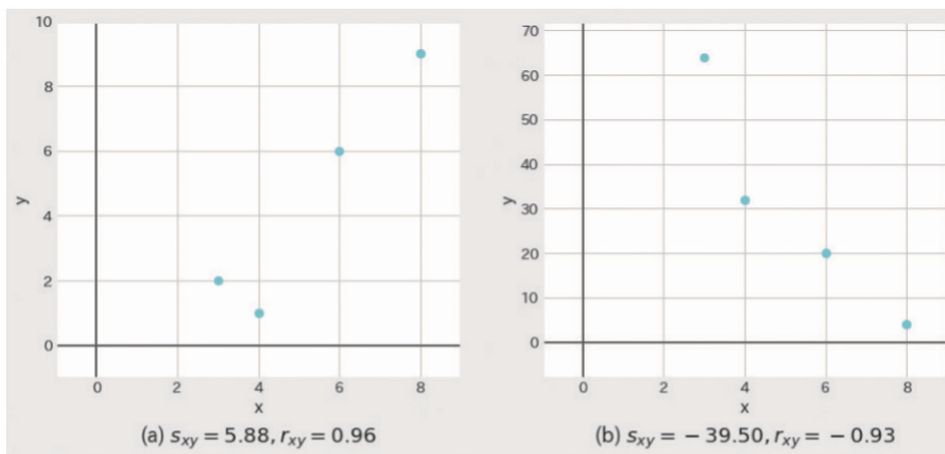
● 相关系数

相关系数的计算方法是将两组数据 x 和 y 的协方差 s_{xy} 分别除以各自的标准差 s_x 和 s_y 。

$$r_{xy} = \frac{s_{xy}}{s_x s_y}$$

相关系数和协方差一样，都可以表示数据之间的关系性，但由于它是协方差除以各自的标准差后的结果，所以值在 -1 到 1 之间。

因此，相关系数的值与数据点的数量级没有关系。图 A-2 中的相关系数 r_{xy} 分别为 0.96 、 -0.93 。



▲图 A-2 相关系数

■ 方差 - 协方差矩阵、相关矩阵

● 方差 - 协方差矩阵

方差 - 协方差矩阵是对两组以上的数据计算各自的协方差，然后汇总而成的矩阵。方差 - 协方差矩阵用 Σ 表示。 $S_{x_1 x_2}$ 是数据点 x_1 和 x_2 的协方差， $S_{x_1 x_1}$ 表示数据点 x_1 和 x_1 的协方差，即 x_1 的方差。

$$\Sigma = \begin{bmatrix} & S_{x_1x_n} & S_{x_1x_2} & \cdots \\ S_{x_1x_n} & S_{x_2x_n} & S_{x_2x_2} & \cdots \\ S_{x_2x_n} & \vdots & \vdots & \ddots \\ \vdots & S_{x_nx_1} & S_{x_nx_2} & \cdots \\ & S_{x_nx_n} & & \end{bmatrix}$$

● 相关矩阵

相关矩阵是对两组以上的数据计算各自的相关系数，然后汇总而成的矩阵。 $r_{x_1x_2}$ 是数据点 x_1 和 x_2 的相关系数。

$$\begin{bmatrix} & r_{x_1x_1} & r_{x_1x_2} & \cdots \\ r_{x_1x_n} & r_{x_2x_1} & r_{x_2x_2} & \cdots \\ r_{x_2x_n} & \vdots & \vdots & \ddots \\ \vdots & r_{x_nx_1} & r_{x_nx_2} & \cdots \\ & r_{x_nx_n} & & \end{bmatrix}$$

 名词解释

名词	首次出现章节	名词解释
AI	1.1	人工智能（Artificial Intelligence）的缩写。人工智能本身的含义很广，包括基于规则预测的方法，基于数理统计预测的方法，以及机器学习方法
Anaconda	5.1	由 Anaconda 公司发布的机器学习和数据分析专用的一体化安装程序
AUC	4.1	ROC 曲线下的面积，最大值为 1，最小值为 0，AUC 越接近 1（面积越大），预测越准确；而接近 0.5 则表示预测不够理想
DataFrame	1.2	Python 库 pandas 使用的数据形式。它可以方便地处理像 Excel 的工作表那样的二维数据
Elbow 方法	3.5	用在 k-means 算法中，通过计算簇内平方和来估计合适的簇的数量
False Negative	4.1	实际为阳性的数据被错误地预测为阴性的情况
False Positive	4.1	实际为阴性的数据被错误地预测为阳性的情况
F 值	4.1	综合反映精确率和召回率两个趋势的指标。精确率和召回率之间是相互制约的关系，如果试图让其中一个的值更高，另一个就会更低。F 值重视的是二者的平衡

(续)

名词	首次出现章节	名词解释
<i>k</i> -means++	3.5	<i>k</i> -means 算法的一种改进，它选择位置尽可能远离的数据点作为重心的初始值
Lasso 回归	2.2	使用学习参数的绝对值之和作为惩罚项来进行正则化的线性回归
RBF 核函数	2.5	一种核函数。RBF 是 Radial Basis Function 的缩写。又称径向基函数
岭回归	2.2	使用学习参数的平方和作为惩罚项来进行正则化的线性回归
roc_curve 函数	4.1	以目标变量和预测概率作为参数，返回 FP 的占比（FPR）、TP 的占比（TPR）和阈值的函数
ROC 曲线	4.1	以 FP 的占比为横轴，以 TP 的占比为纵轴的图形。图形中显示了当预测数据为阳性的预测概率的阈值从 1 逐渐下降时，FP 和 TP 之间关系的变化
True Negative	4.1	实际为阴性的数据被正确预测为阴性的情况
True Positive	4.1	实际为阳性的数据被正确预测为阳性的情况
<i>t</i> 分布	3.8	一个以自由度为参数的概率分布
鸢尾花数据集	1.2	以表格形式记录的鸢尾花的萼片长度、萼片宽度、花瓣长度、花瓣宽度，以及品种的数据集。它是一个人们可以用来测试各种算法的著名的测试集
权重	2.1	在计算输出时，用来乘以特征值或计算过程中的结果的学习参数。举例来说，有数据 <i>x</i> 和学习参数 <i>w</i> ₀ 、 <i>w</i> ₁ ，输出 <i>y</i> = <i>w</i> ₀ + <i>w</i> ₁ <i>x</i> ，那么其中的 <i>w</i> ₁ 就是权重
核函数	2.5	可用于核方法的函数的总称，包括 RBF 核函数和 Sigmoid 核函数等
高斯分布	3.6	用均值和方差表示的概率分布。它是以均值为峰值的左右对称的形状
过拟合	1.2	对训练数据的预测效果很好，但对没有用于训练的数据的预测效果不好的现象
训练误差	2.2	模型的输出相对于训练数据的误差
学习参数	2.1	由算法从数据中计算出来的机器学习模型的参数
加权平均	2.7	在求均值时，像计算期望值一样，通过个数和概率相乘而计算出来的均值
虚拟环境	5.2	在本书中虚拟环境指的是 Python 的虚拟环境，是在使用多个第三方 Python 包时会用到的一种技术
激活函数	2.8	用在感知器中的、根据加权特征值之和计算概率的非线性函数
机器学习	1.1	通过输入数据而不是编程来获得对数据进行分类或数值预测的计算方法的技术。它分为有监督学习、无监督学习和强化学习
机器学习模型	2.2	表示机器学习所使用的算法及假定的数据之间的关系。也简称为模型
树结构	2.9	一种具有层级结构的数据结构，它具有图论中的树的结构

(续)

名词	首次出现章节	名词解释
强化学习	1.1	在某个环境下，智能体以获得的报酬最大化为目标而采取行动的学习方法
有监督学习	1.1	通过向计算机提供问题答案来训练机器学习模型的方法。答案为类别的情况称为分类问题，答案为连续值的情况称为回归问题
无监督学习	1.1	在不提供问题答案的情况下训练机器学习模型的方法。无监督学习的目标是通过将数据变换为其他形式或者在数据中找到子集的方式来理解输入数据的结构
贡献率	3.1	表示某个元素在整个数据中所占信息量的比率
近似矩阵	3.3	通过矩阵分解方法得到的包含概要信息的矩阵
近邻搜索	2.9	找到空间中最近的点。 k NN 基于未知数据的近邻点的信息来进行分类
簇内平方和	3.5	用在 k -means 算法中，计算簇内所有数据点与簇的重心之间距离的平方和的总和
聚类	1.1	把目标数据分成若干个簇（相似数据的集合）
网格搜索	2.4	一种超参数搜索方法。预先将参数排列为格子状，对其进行穷尽搜索，找到最佳的参数
决策树	2.7	一种机器学习模型。它对数据空间进行分割以使不纯度变小。表示不纯度的指标有基尼系数等
决策边界	2.3	用在分类问题中，指的是训练数据的分类结果发生变化的边界
验证误差	2.2	模型的输出相对于验证数据的误差
交叉验证	4.1	英文为 cross validation。一种以多种分割方案分割有监督学习的数据，并进行多次验证的方法
特征值	3.1	在某个线性变换下，特征向量变化的倍率
特征值问题	3.1	在向量线性变换时，找出仅有向量大小发生变化的情况下的特征值和特征向量的问题
特征向量	3.1	在某个线性变换下，只有大小会发生变换的向量
混合高斯分布	3.6	由多个高斯分布组成的概率分布
混淆矩阵	4.1	将分类结果以表格的形式汇总，统一查看哪些标签被正确分类，哪些标签被错误分类
计算机视觉	3.3	进行图像的数据处理及必要信息提取的研究领域
第三方包	5.3	本书涉及的是 Python 的第三方包。包用来扩展标准 Python 的功能。用于机器学习的 scikit-learn 就是一个 Python 第三方包
召回率	4.1	实际为阳性的数据中，被正确预测为阳性的数据所占的比例

(续)

名词	首次出现章节	名词解释
渐近假设	2.9	一个“拥有的训练数据越多，就有可能找到接近未知数据的训练数据”的假设
采样	3.6	从某个概率分布中获取数据点（样本）
阈值	2.3	当根据计算的概率进行分类时，作为边界的值。例如，在进行 0 和 1 的二元分类时，设阈值为 0.5，那么如果概率大于 0.5，则分类为 1；如果小于 0.5，则分类为 0
Sigmoid 核函数	2.5	一种核函数。它将 Sigmoid 函数用作核函数
Sigmoid 函数	2.3	函数表达式为 $\sigma(z)=1/(1+\exp(-z))$ 。函数的输出值在 0 到 1 之间，用于逻辑回归和神经网络
降维	1.1	用更少的维度来表示多维输入数据
次数	2.2	多项式展开时出现的最大指数。例如 $w_0 + w_1x + w_2x^2$ 的次数为 2
自然语言处理	1.1	以日语、汉语或英语等自然语言的句子或单词为对象的任务。代表性应用有垃圾邮件的检测和搜索引擎
基尼系数	2.7	用在决策树的学习算法，用来计算不纯度
主成分	3.1	通过主成分分析计算得到的新轴。按方差从大到小的顺序，依次为第一主成分、第二主成分
主成分得分	3.1	在主成分轴上表示的值
深度学习	1.1	英文为 Deep Learning。一种机器学习算法。它在图像识别领域的突破性成果引发了人们的关注
缩放	3.3	对数据值进行变换，使其值在一定范围之内
类别标签数据	1.2	在有监督学习分类问题中，将属于每个分类的数据称为类别，数据所属的类别称为标签
正确率	4.1	预测正确的结果占总预测结果的比例
正则化	2.2	一种在损失函数中加入惩罚项的学习方法。用来防止过拟合
特征变量	1.1	代表事物特征的数据，是机器学习模型的输入，即特征值
线性核函数	2.5	一种核函数。当在核方法中应用线性核函数时，它等价于线性支持向量机
线性组合	3.6	系数和向量的乘积之和
潜在空间	3.3	由潜在变量表示的数据空间
潜在变量	3.2	虽然不能直接观察，但可以根据给定的数据点推测出来的变量
相关系数	1.2	衡量两个变量之间线性关系（相关性）的强弱的指标。越接近于 1，越表明数据之间是正相关关系；越接近于 -1，越表明数据之间是负相关关系

(续)

名词	首次出现章节	名词解释
软间隔	2.4	支持向量机中允许训练数据进入间隔内侧的情况
损失	2.1	表示模型的输出结果与目标变量之间的误差
损失函数	2.1	表示误差与学习参数之间关系的函数。即误差函数
对角矩阵	3.2	除对角线元素以外的元素的值都为 0 的行数和列数相同的矩阵
多项式核函数	2.5	一种核函数。它将多项式用作核函数
任务	1.1	算法要执行的处理。有监督学习的任务有分类和回归；无监督学习的任务有降维和聚类
多元分类	1.1	指的是分类问题中预测的类别在两个以上的情况
流形学习	3.7	用于非线性数据的降维方法
单词概率	3.4	表示 LDA 算法中在某个主题的文本中出现某个词的概率
简单感知机	2.8	一种机器学习模型，将加权特征值的结果应用于激活函数进行识别
精确率	4.1	在被预测为阳性的数据中，被正确预测为阳性的数据所占的比例
文本挖掘	3.3	对文本数据进行分析，从中提取重要信息的数据挖掘
奇异值	3.2	通过奇异值分解得到的对角矩阵的对角元素
奇异值分解	3.2	一种矩阵分解方法。它可以将原矩阵表示为两个正交矩阵和一个对角矩阵的乘积
特征值	1.1	它是机器学习模型的输入，是代表某种事物特征的数据。例如，当根据身高和体重这对组合中预测性别时，身高和体重被称为特征值
主题概率	3.4	表示在 LDA 算法中某个文本属于某个主题的概率
主题模型	3.4	假设文本数据是由主题等潜在变量组成的模型
二元分类	1.1	指的是分类问题中预测的类别为两个的情况
输入变量	1.2	它是机器学习模型的输入，是代表某种事物特征的数据。即特征值
节点	2.8	表示构成图形的一种元素——点。在神经网络的结构图中，它表示某个特征值
硬间隔	2.4	支持向量机中不允许训练数据进入间隔内侧的情况
偏置	2.3	只在计算输出时相加的学习参数。如果有数据 x 和学习参数 w_0 、 w_1 ，输出 $y = w_0 + w_1x$ ，那么其中的 w_0 就是偏置
超参数	2.2	不能从数据中学习，所以必须事先指定的机器学习模型的参数
惩罚项	2.2	在进行正则化时向损失函数增加的一项。它表示学习参数的大小
验证误差	2.2	在训练模型时未使用的数据上产生的误差

(续)

名词	首次出现章节	名词解释
评估方法	1.2	在评估机器学习模型时，对分类问题使用正确率、精确率、召回率、 F 值和AUC等，对回归问题使用均方误差和决定系数等多种指标进行评估
Bootstrap 方法	2.7	通过对单个训练数据进行多次随机的抽样放回，“虚增”数据的学习方法
不均衡数据	4.1	目标变量有偏差的数据称为不均衡数据。例如用于二元分类的数据集中大部分数据是 1，几乎没有数据是 0 的情况
不纯度	2.7	一种数据杂乱程度的数值表达。表示不纯度的具体指标有基尼系数等
方差	3.1	表示数据点的偏差（杂乱）程度的值
方差 - 协方差矩阵	3.1	矩阵形式的表示数据之间的关系的协方差
分类问题	1.1	在有监督学习中，指的是对性别等类别进行预测的问题
向量	2.3	有方向和大小的量。在机器学习的语境下，它是排列为一系列的数值
间隔	2.4	最接近决策边界的训练数据与决策边界之间的距离
间隔最大化	2.4	计算机器学习模型的参数，使决策边界的间隔最大。支持向量机通过间隔最大化来学习决策边界
目标变量	1.1	作为有监督学习输入的数据集中的答案。在分类问题中，它是代表类别的离散值；在回归问题中，它是连续值
预测概率	2.3	由机器学习模型根据输入数据计算出的概率。它是指预测结果为某个值的概率。在二元分类预测结果为 0 或 1 的情况下，如果预测 0 的概率为 10%，那么预测 1 的概率是 90%。这个概率叫作预测概率
预测模型	1.1	可以通过输入未知数据来“预测”和计算结果的模型。学习后的机器学习模型有时被称为预测模型
库	1.1	将多个程序组合起来以便于再次分发的形式
标签	1.1	即目标变量。尤其在分类问题中，有时目标变量被称为标签
随机搜索	2.4	一种超参数搜索方法。它随机地尝试预先定义的参数，并重复进行，直到获得良好的结果
离散值	1.1	代表类别的、大小没有意义的数值。例如，在用数值表示性别时，“男”可以数值化为 0，“女”可以数值化为 1，但这些数值的大小关系没有意义
相似度	3.2	表示数据点之间相似度的数值。数值越大，表明相似度越高
推荐	3.3	根据每位顾客的购买行为等数据，向顾客展示喜欢的产品和相关产品的方法
连续值	1.1	身高和体重等大小关系有意义的数值

参考文献

● Python 语言相关（参考图书）

岩崎圭, 北川慎治, 寺田学. スラスラわかる Python [M]. 東京: 翔泳社, 2017. [难度等级: 初级](#)

Brett Slatkin. Effective Python: 编写高质量 Python 代码的 59 个有效方法 [M]. 爱飞翔, 译. 北京: 机械工业出版社, 2016. [难度等级: 中级](#)

Michal Jaworski, Tarek Ziad é. Python 高级编程: 第 2 版 [M]. 张亮, 阿信, 译. 北京: 人民邮电出版社, 2017. [难度等级: 高级](#)

● 从预处理到机器学习

池内孝啓, 片柳薫子, 岩尾エマはるか, 等. Python ユーザのための Jupyter [実践] 入門 [M]. 東京: 技術評論社, 2017. [详解 pandas 基础知识和可视化](#)

寺田学, 辻真吾, 鈴木たかのり, 等. Python によるあたらしいデータ分析の教科書 (AI & TECHNOLOGY) [M]. 東京: 翔泳社, 2018. [详解从预处理到机器学习的流程](#)

Jake VanderPlas. Python 数据科学手册 [M]. 陶俊杰, 陈小莉, 译. 北京: 人民邮电出版社, 2018. [详解从预处理到机器学习的知识](#)

● 数学和算法

加藤公一. 機械学習のエッセンス - 実装しながら学ぶ Python, 数学, アルゴリズム - (Machine Learning) [M]. 東京: ソフトバンククリエイティブ, 2018. [用 Python 学算法](#)

斋藤康毅. 深度学习入门: 基于 Python 的理论与实现 [M]. 陆宇杰, 译. 北京: 人民邮电出版社, 2018. [用 Python 从零实现深度学习](#)

平井有三. はじめてのパターン認識 [M]. 東京: 森北出版, 2012. [学习模式识别的基础知识](#)

久保拓弥. データ解析のための統計モデリング入門——一般化線形モデル・階層ベイズモデル・MCMC (確率と情報の科学) [M]. 東京: 岩波書店, 2012. [用 R 学统计建模](#)

中井悦司．技術者のための基礎解析学 機械学習に必要な数学を本気で学ぶ [M]．東京：翔泳社，2018． [用数学方法学基础分析](#)

東京大学教養学部統計学教室．統計学入門 [M]．東京：東京大学出版会，1991． [统计学入门必读图书](#)

齋藤正彦．線型代数入門（基礎数学1）[M]．東京：東京大学出版会，1966． [线性代数必读图书](#)

佐藤洋行，原田博植，里洋平，等．改訂2版 データサイエンティスト養成読本 [プロになるためのデータ分析力が身につく！（Software Design plus）] [M]．東京：技術評論社，2016． [日本技术评论社的“养成读本”系列](#)

比戸将平，馬場雪乃，里洋平，等．データサイエンティスト養成読本 機械学習入門編（Software Design plus）[M]．東京：技術評論社，2015． [日本技术评论社的“养成读本”系列](#)

版 权 声 明

機械学習図鑑 見て試してわかる機械学習アルゴリズムの仕組み
(Kikai Gakushuu Zukan : 5565-4)

Copyright © 2018 Shinya Akiba, Asei Sugiyama, Manabu Terada

Original Japanese edition published by SHOEISHA Co., Ltd.

Simplified Chinese Character translation rights arranged with SHOEISHA Co., Ltd.

through CREEK & RIVER Co., Ltd. and CREEK & RIVER SHANGHAI Co., Ltd.

Simplified Chinese Character translation copyright © 2021 by Posts & Telecom Press.

本书中文简体字版由 SHOEISHA Co., Ltd. 授权人民邮电出版社有限公司独家出版。未经出版者事先书面许可，不得以任何方式或途径复制或传播本书内容。

版权所有，侵权必究。



微信连接



回复“算法”查看相关书单



微博连接

关注 @图灵教育 每日分享IT好书



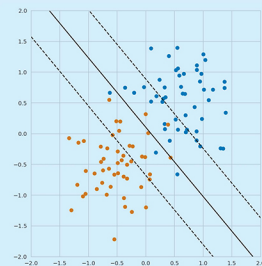
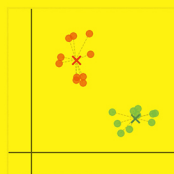
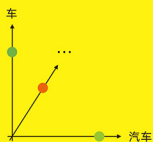
QQ连接

图灵读者官方群I: 218139230

图灵读者官方群II: 164939616

图灵社区
iTuring.cn

在线出版,电子书,《码农》杂志,图灵访谈



| 全面

网罗有监督学习和无监督学习的17种算法，涉及回归、分类、降维、聚类等问题。

| 直观

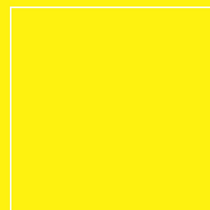
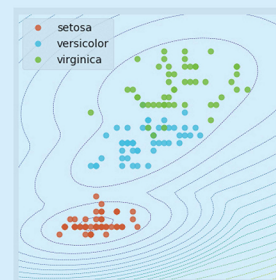
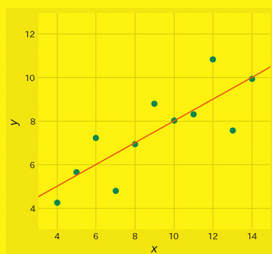
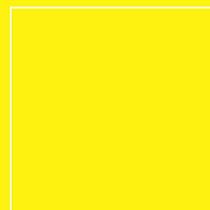
全彩图解，让原本抽象复杂的算法变得一目了然，方便读者快速了解各算法的特征，学会如何选择恰当的算法。

| 易懂

几乎没有数学公式和统计术语，零基础也可轻松读懂。

| 实践

各算法均用Python代码实现，读者可下载配套代码资源，边试边学。



图灵社区：iTuring.cn
分类建议：计算机 / 算法
人民邮电出版社网址：www.ptpress.com.cn



扫码领取
随书代码资料

ISBN 978-7-115-56356-9



9 787115 563569 >

定价：79.80 元